



The Ultimate Linux Bootcamp for DevOps, SRE and Cloud Engineers (Hands-On)





DevOps



Cloud Computing

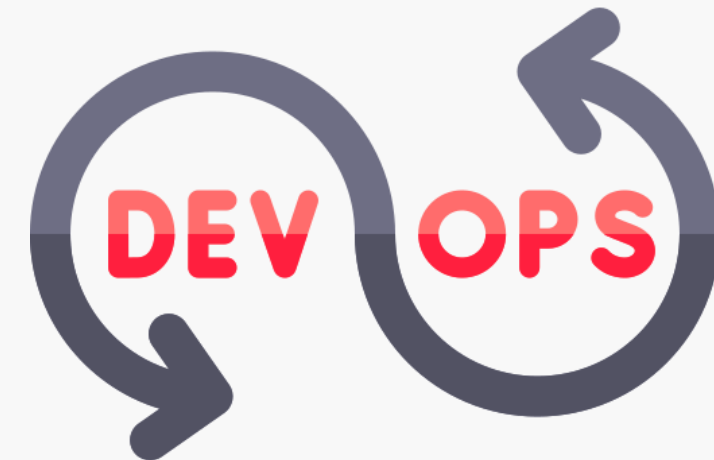


Site Reliability Engineering





Yogesh Raheja



Our Courses & Books



Puppet for the Absolute Beginners – Hands-On



Generative AI Essentials - Practical Use Cases



SaltStack for the Absolute Beginners – Hands-On



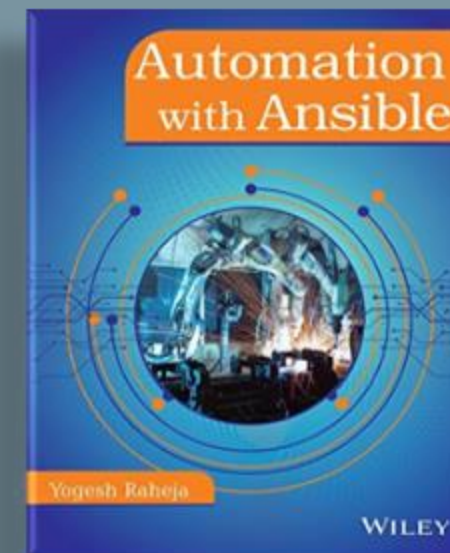
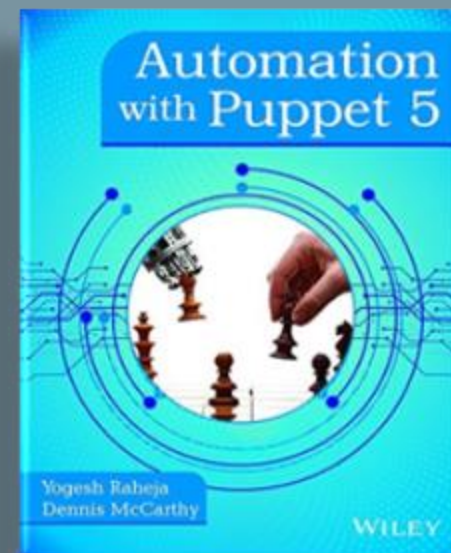
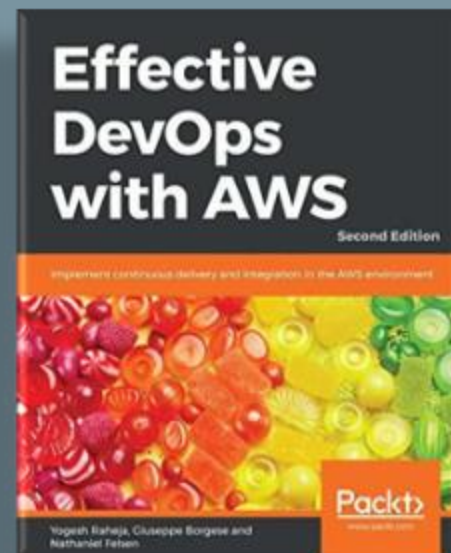
Infrastructure Automation with OpenTofu – Hands-On



AI Ecosystem for the Absolute Beginners - Hands-On



Mastering Prompt Engineering for GenAI





Yogesh Raheja



Thinknyx Team





Linux Architecture



Command-line
Operations



User & Group
Management



File
Permissions



Package
Management



Storage
Management



Process & Service
Management



Networking



SSH



System Monitoring



Scheduling



Lectures



Demonstrations

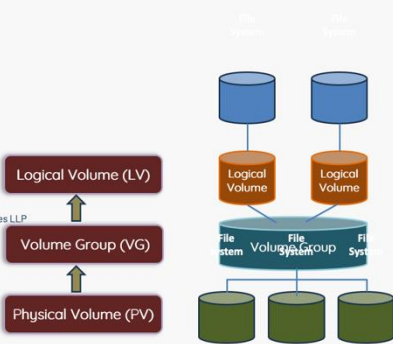


Practice Tests

LVM

- Key Components of LVM
 - ❑ Key Advantage of LVM
 - ✓ Filesystem resides on top of the Logical Volume (LV)
 - ✓ Allows on-the-fly resizing without downtime
 - ✓ Supports expanding or shrinking storage as needed
 - ✓ Ideal for production environments requiring uninterrupted access

Copyright © Thinknyx Technologies LLP



Advanced Commands

```
GAWK(1)                                Utility Commands                                GAWK(1)
NAME
    gawk - pattern scanning and processing language

SYNOPSIS
    gawk [ POSIX or GNU style options ] -f program-file [ -- ] file ...
    gawk [ POSIX or GNU style options ] [ -- ] program-text file ...

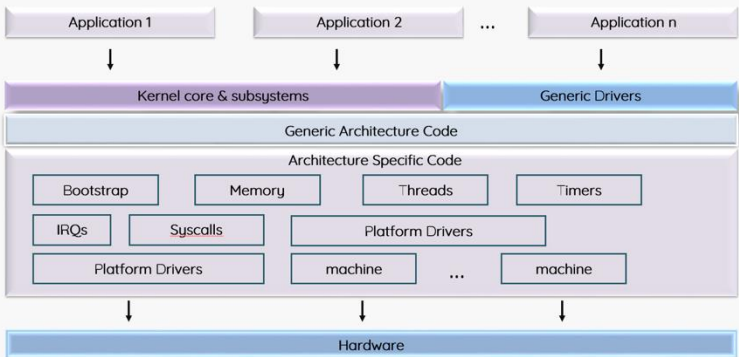
DESCRIPTION
    Gawk is the GNU Project's implementation of the AWK programming language. It conforms to the
    definition of the language in the POSIX 1003.1 standard. This version in turn is based on the
    description in The AWK Programming Language, by Aho, Kernighan, and Weinberger. Gawk provides
    the additional features found in the current version of Brian Kernighan's awk and numerous
    GNU-specific extensions.

    The command line consists of options to gawk itself, the AWK program text (if not supplied via
    the -f or --include options), and values to be made available in the ARGV and ARGV pre-defined
    AWK variables.
```



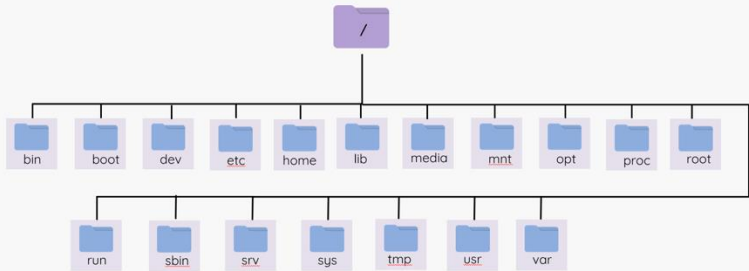
Copyright © Thinknyx Technologies LLP

Linux Architecture



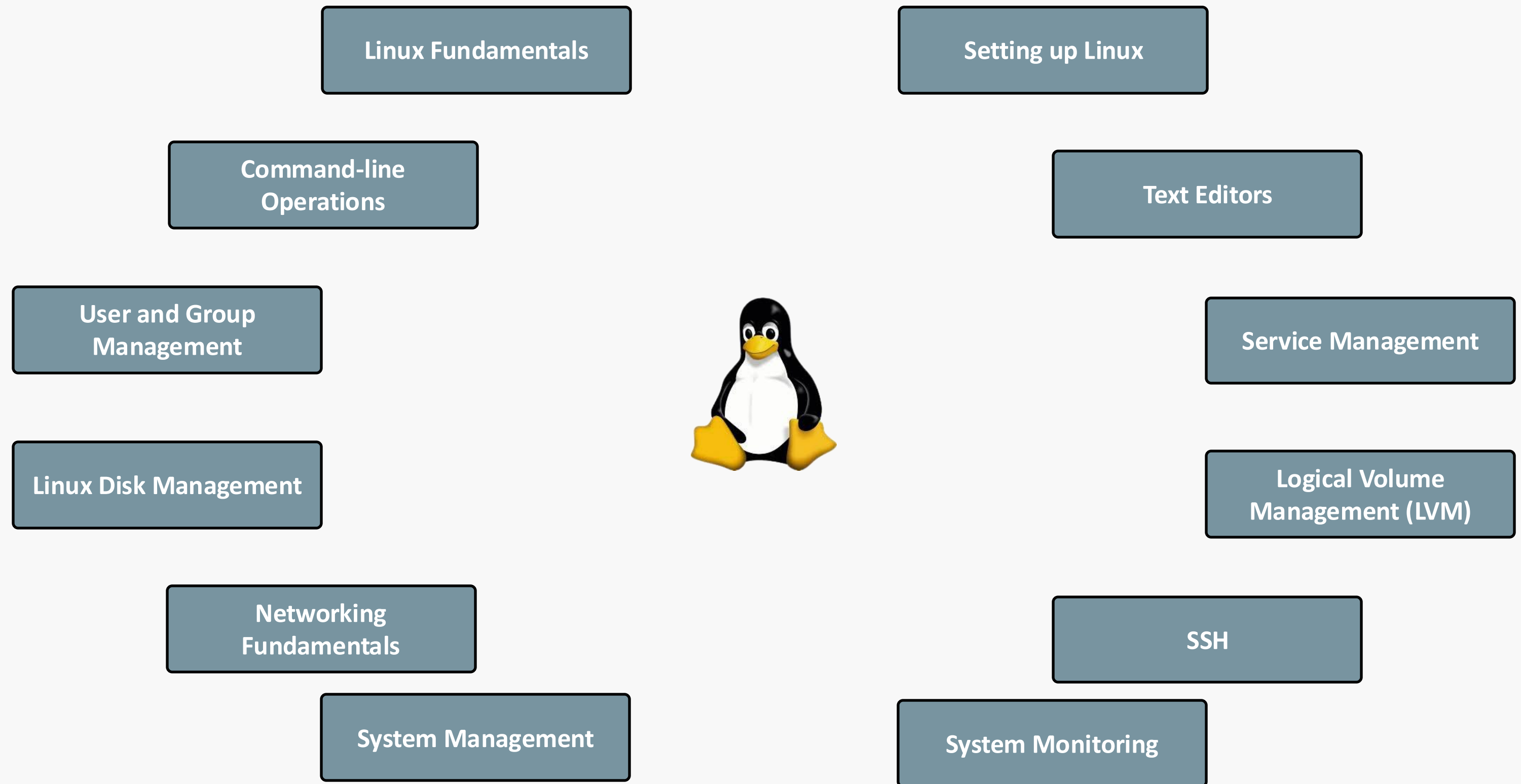
Copyright © Thinknyx Technologies LLP

Linux Directory Structure



Copyright © Thinknyx Technologies LLP







Thinknyx Technologies

Linux Course

Enhancing Strategy

Section - 1





Linux | Introduction to Linux



Introduction to Linux



- Evolution of Operating Systems
- Linux's Pivotal Role in Modern Computing
- Key Features of Linux
- Essential Terminologies
- Types of Linux Shells
- Linux Distributions



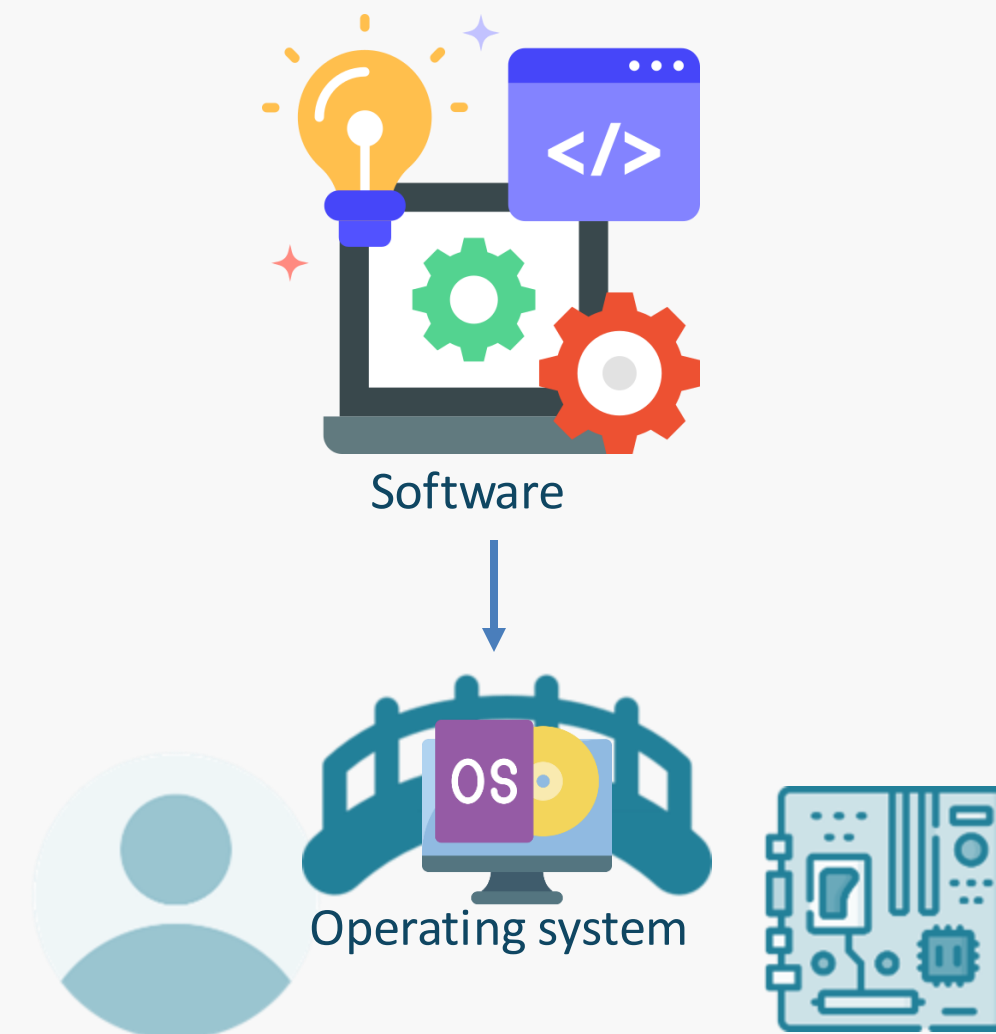
Linux | History of Operating Systems



History of Operating Systems



Computing devices



History of Operating Systems

Early Computing
(1940s - 1950s)

- ✓ Early computing ran one program at a time
- ✓ Programs were manually loaded using punch cards
- ✓ Early computing output was slow
- ✓ Development of the first operating systems
- ✓ Operating systems automated program loading and queue management



History of Operating Systems



History of Operating Systems

The Rise of Multitasking
(1960s)

- ✓ Introduction of multitasking
- ✓ Simultaneous program execution
- ✓ Emergence of the Atlas Supervisor in the 1960s
- ✓ CPU usage maximization
- ✓ Automated program switching for continuous computing



History of Operating Systems

Memory Management and Time-Sharing (1960s)

- ✓ Virtual memory in operating systems
- ✓ Dynamic memory allocation for programs
- ✓ Memory protection between programs
- ✓ Multi-user support through time-sharing
- ✓ Emergence of Multics as a timesharing Operating System



History of Operating Systems

The Birth of Unix (1969)

- ✓ Ken Thompson and Dennis Ritchie developed Unix in 1969 at Bell Labs
- ✓ Unix introduced a streamlined kernel that separated essential functions from user utilities
- ✓ The kernel manages system resources and facilitates communication between hardware and software
- ✓ Unix's modular design allowed it to be adapted for various hardware platforms
- ✓ Unix's open-source code led to influential versions like AT&T's System V and BSD
- ✓ This innovation enabled Unix variants like HP-UX, Solaris, and AIX to thrive in industries



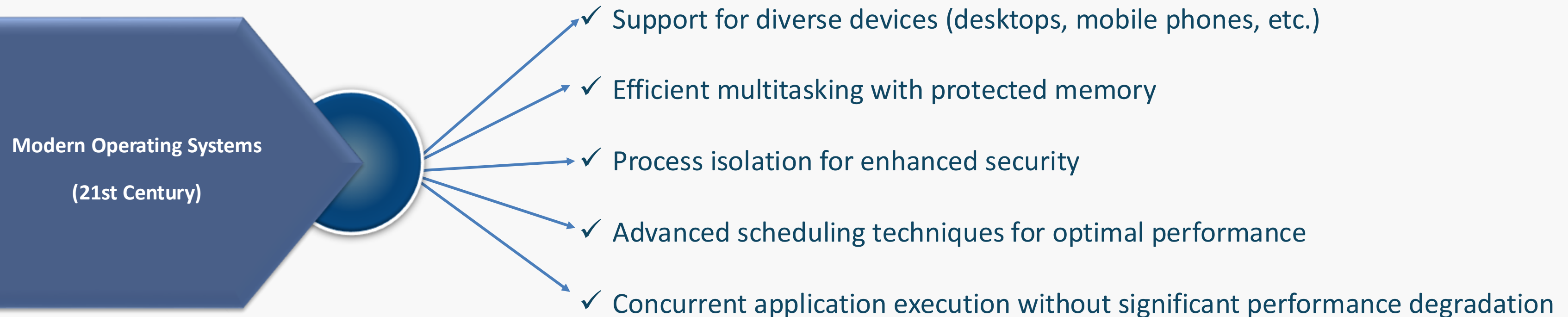
History of Operating Systems

The Rise of Linux (1991)

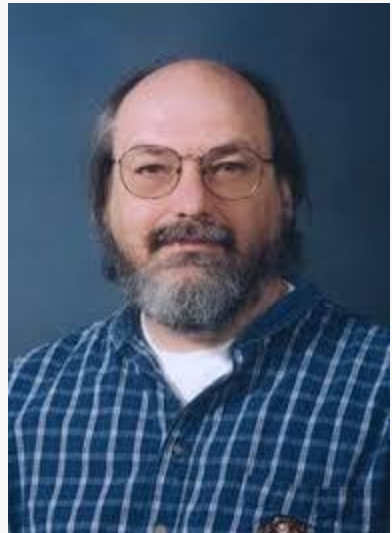
- ✓ Linus Torvalds released the first version of Linux in 1991
- ✓ Linux is a free and open-source operating system
- ✓ Inspired by Unix, Linux has a modular design
- ✓ Global contributions have led to a wide variety of Linux distributions
- ✓ Linux is used in data centers and Android smartphones



History of Operating Systems



History of Operating Systems



Ken Thompson



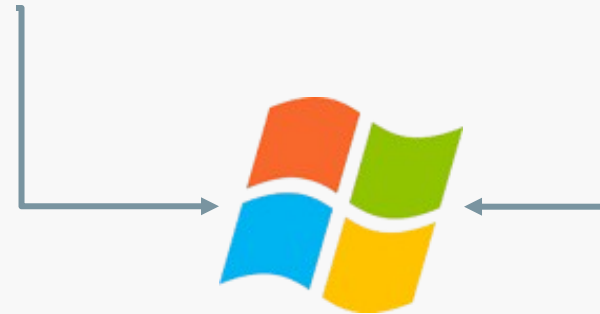
Dennis Ritchie



Bill Gates



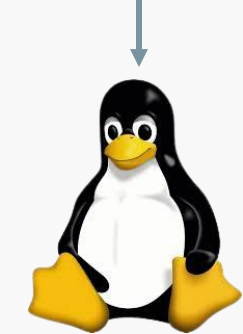
Paul Allen



Steve Jobs



Linus Torvalds



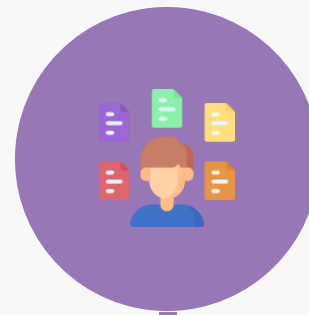
History of Operating Systems



Early Computing (1940s
- 1950s)



The Need for Compatibility (1960s)



The Rise of Multitasking
(1960s)



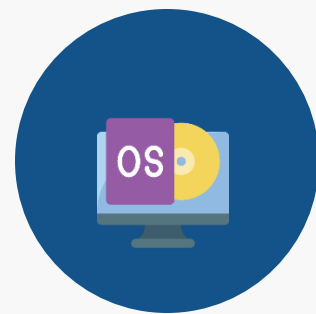
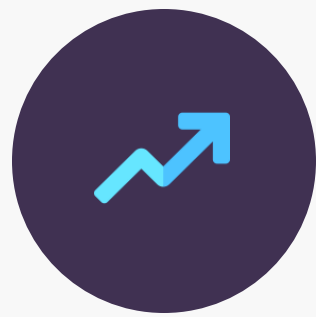
Memory Management and Time-
Sharing (1960s)



The Birth of Unix
(1969)



History of Operating Systems



The Rise of Linux
(1991)

Modern Operating Systems
(21st Century)





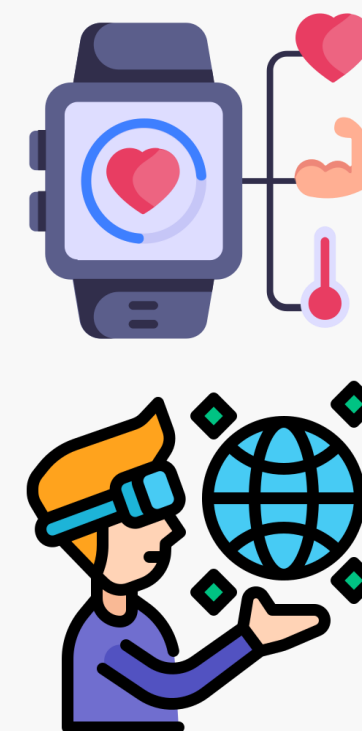
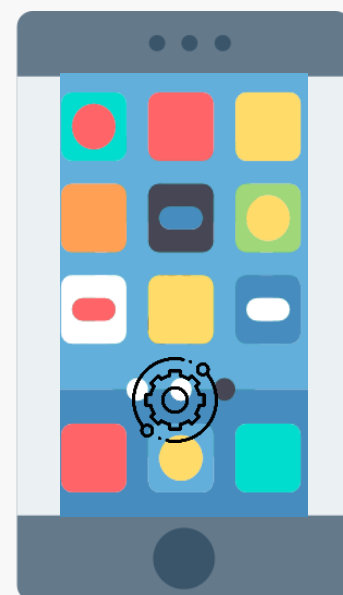
Linux | Getting Started



Getting Started with Linux



Getting Started with Linux



Getting Started with Linux

History of Linux

➤ Inception and Vision



Andrew S. Tanenbaum
1985



Minix

Tanenbaum preferred Minix's microkernel design over
Torvalds' monolithic kernel approach



Linus Torvalds
1990



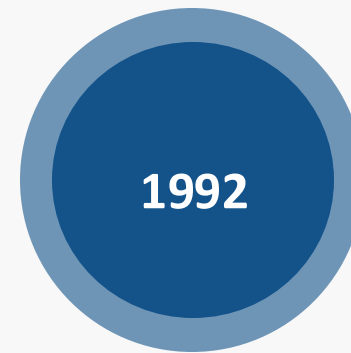
Linux



Getting Started with Linux

History of Linux

➤ The Open-Source Movement



- ✓ Linus Torvalds re-licensed the kernel
- ✓ General Public License (GPL)



- ✓ The licensing enabled global developer collaboration and contributions

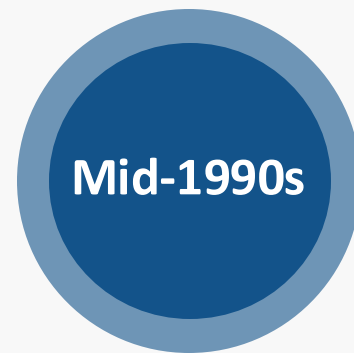
- ✓ Many Linux distributions combine the kernel with GNU tools for various needs
- ✓ A Linux distro combines the Linux kernel with software and tools



Getting Started with Linux

History of Linux

➤ A New Paradigm for Software Development



- ✓ Linux distributions offered powerful tools and promoted user freedom
- ✓ Open-source ethos enabled innovation through modification, sharing, and inclusivity



- ✓ Corporations like IBM invested in Linux development



Getting Started with Linux

History of Linux

➤ Linux Today and Its Impact

- ✓ Linux serves as the backbone of the digital ecosystem



- ✓ Linux manages over 90% of public cloud workloads
- ✓ Scalability and efficiency
- ✓ Linux-based instances



- In DevOps landscape



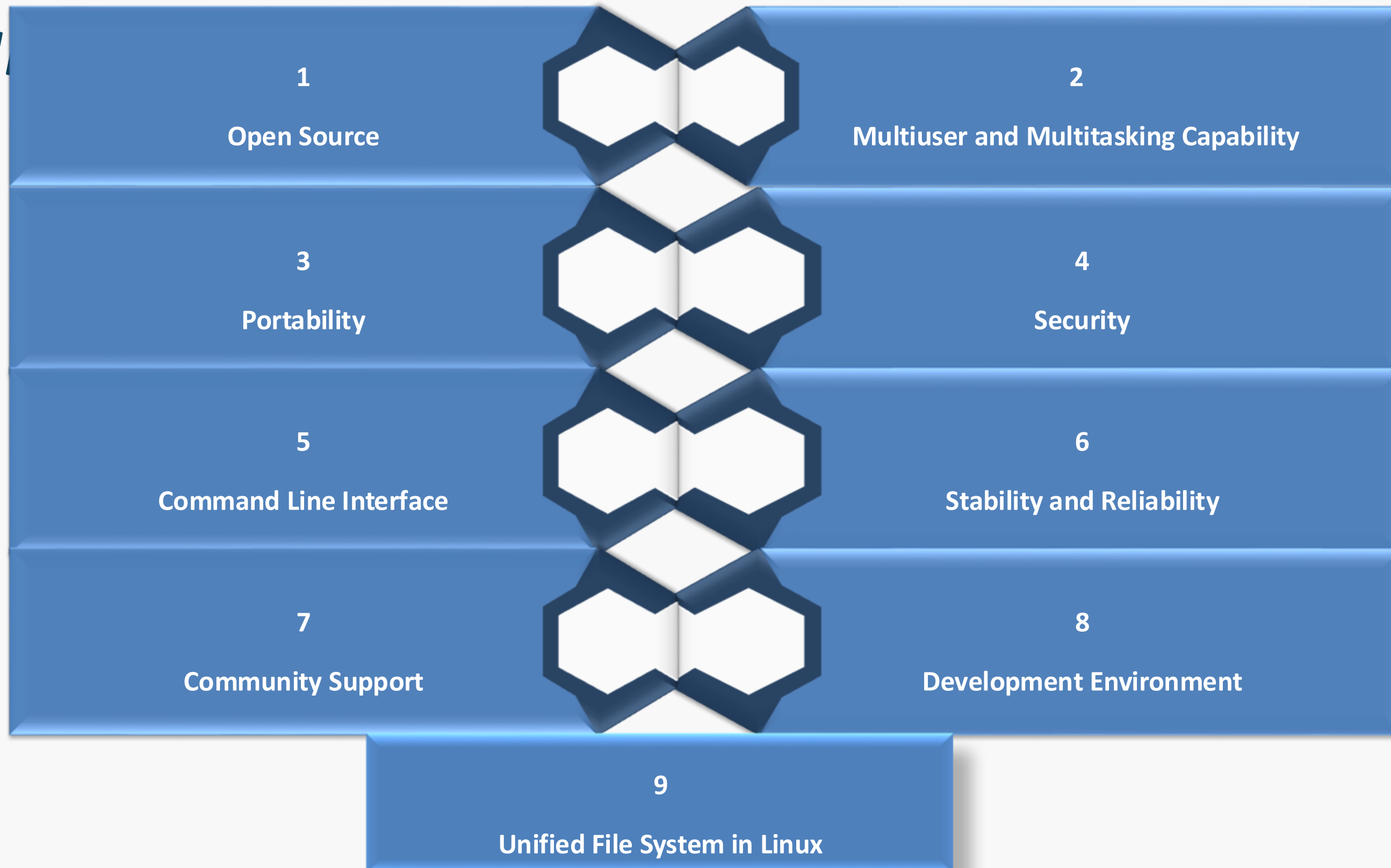


Linux | Features



Linux

Features





Linux | Terminologies



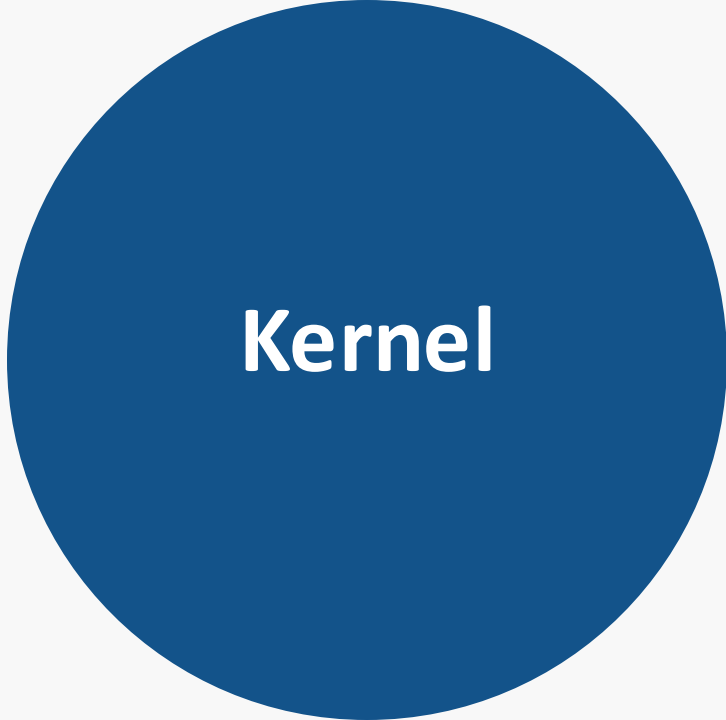
Linux Terminologies

The Shell or
command line
interface (CLI)

- ✓ **Operating System** (OS) is the software that manages the hardware and provides an interface for the user to interact with the hardware.
- ✓ **Application** is a program that runs on top of the OS and provides a specific functionality.
- ✓ **Shell** is a program that runs on top of the OS and provides a command line interface for the user to interact with the OS.
- ✓ **Process** is a program that is currently running on the system.
- ✓ Common shells: bash, zsh



Linux Terminologies



Kernel

- ✓ The "brain" of the Linux system, managing hardware and resources
- ✓ Acts as a bridge between hardware and applications
- ✓ Forms the core of a Linux-based OS
- ✓ Combined with libraries and utilities to create Linux distros



Linux Terminologies



Distributions

- ✓ Offer unique tools and packages built on the Linux kernel
- ✓ Examples: Ubuntu, Fedora, Red Hat
- ✓ Each distro customizes Linux to suit specific needs



Linux Terminologies



Process

- ✓ Instance of a running program
- ✓ Created when you execute a command or launch an application
- ✓ Each process has a unique Process ID (PID)
- ✓ The PID helps manage resources and track or terminate processes



Linux Terminologies

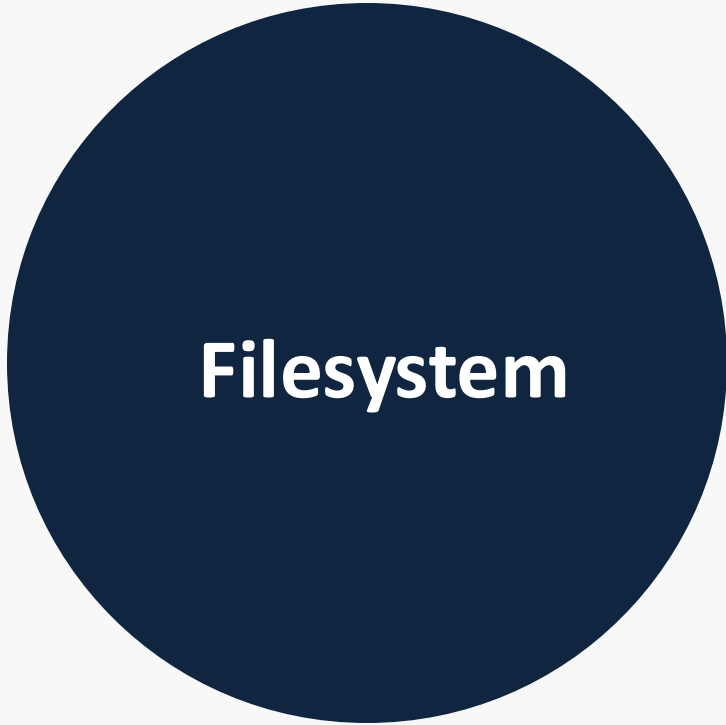


Services

- ✓ Background programs supporting applications or system functions
- ✓ httpd: Handles web services
- ✓ ftpd: Manages file transfers
- ✓ systemd: Supervises the system from boot to shutdown



Linux Terminologies



Filesystem

- ✓ Organizes and manages files in Linux
- ✓ Common types: ext3, ext4, FAT, XFS, Btrfs
- ✓ Different filesystems cater to various use cases
- ✓ Impacts performance, data integrity, and storage efficiency



Linux Terminologies

X Window System

- ✓ Provides tools for creating GUIs
- ✓ Enables interaction with Linux through graphical controls
- ✓ Desktop environments like GNOME, KDE, Xfce, and Fluxbox build on top of X to offer user-friendly visuals



Linux Terminologies

The Shell or Command Line Interface (CLI)

- ✓ The command line interface (CLI) in Linux
- ✓ Allows users to type commands to control the system
- ✓ Acts as an intermediary between the user and the OS, translating commands into instructions for the kernel
- ✓ Common shells: bash, zsh





Linux | Types of Shells



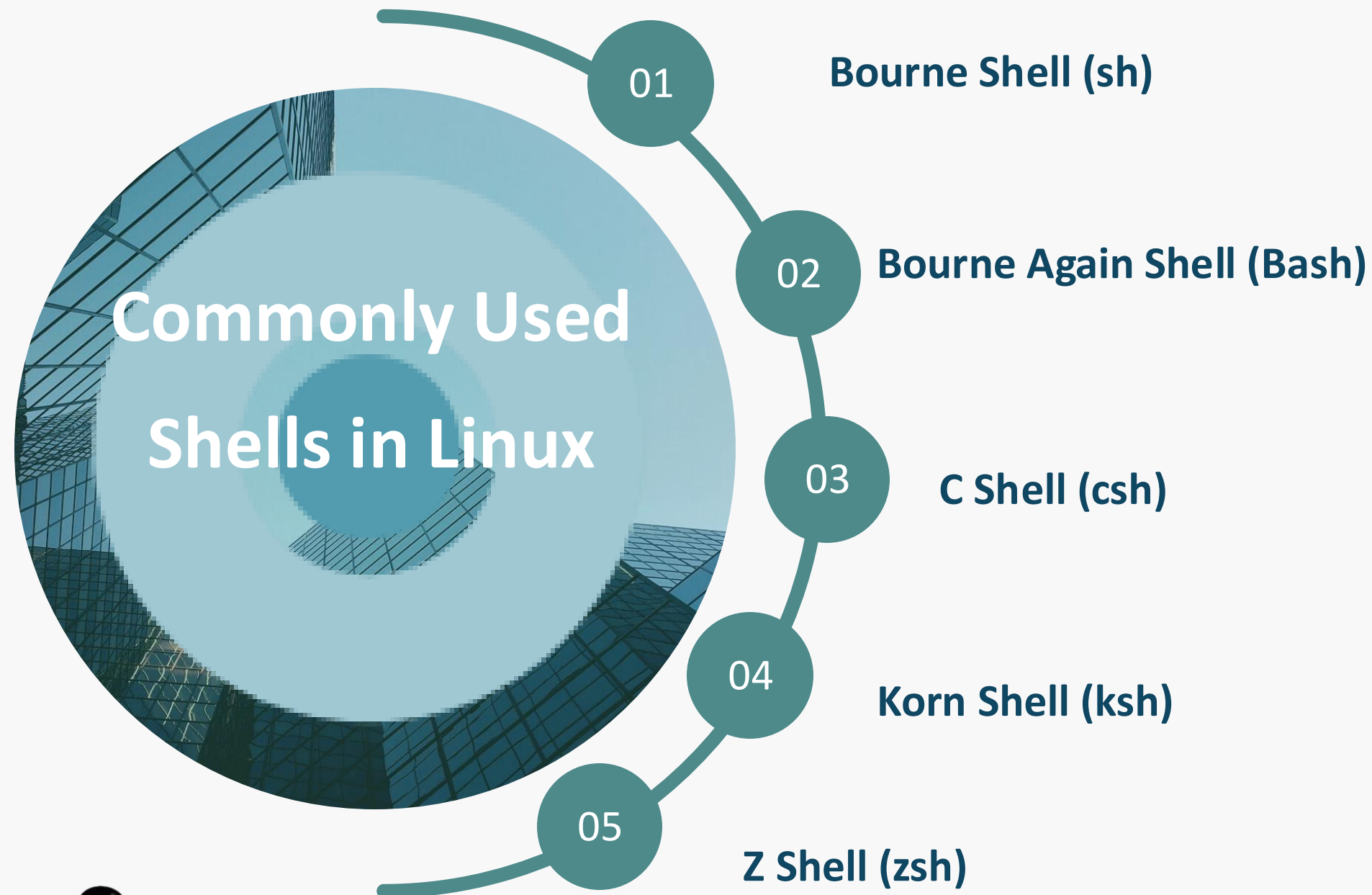
Types of Shells

Shell is a Command-line interface

It's a bridge between User and Operating System



Types of Shells



- ✓ One of the earliest shells along with Bourne and Korn Shells
- ✓ Designed for long scripting and command-line tasks
- ✓ Simple and efficient for managing control, scripting
- ✓ Popular among advanced users





Linux | Types of Distributions



Linux Distribution

- ✓ A Linux distribution (distro) is a Linux-based operating system
- ✓ Combines the Linux kernel with tools, libraries, and utilities
- ✓ Each distro has its own package management and repository
- ✓ Some distros offer unique desktop environments
- ✓ Designed for different users: beginners, administrators, or enterprises

Now hundreds of Linux
distributions

➤ **Three foundational families**



Types of Distributions

Slackware: The Old-School Stability

- ✓ One of the oldest Linux distributions
- ✓ Focuses on simplicity and stability
- ✓ Designed for advanced users and administrators
- ✓ Minimalist, no-frills experience
- ✓ Not widely adopted but has a dedicated following
- ✓ Follows the "Keep It Simple, Stupid" (KISS) philosophy
- ✓ Popular for servers and custom configurations



Types of Distributions

Debian: The Universal Operating System

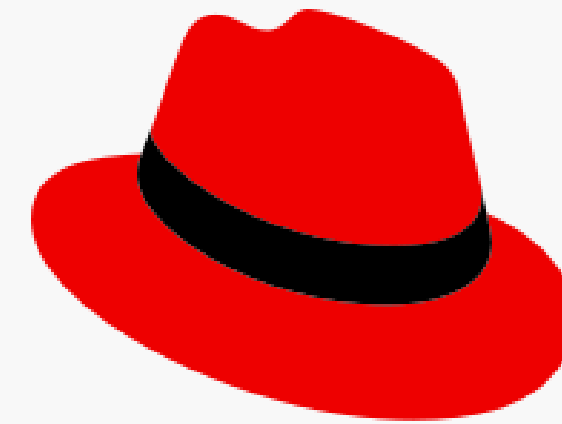
- ✓ Foundation for Popular Distros like Ubuntu
- ✓ Renowned Stability and Security
- ✓ Ideal for Servers and Critical Systems
- ✓ Developer and Administrator Friendly
- ✓ Robust Dependency Management



Types of Distributions

Red Hat Linux: The Enterprise Pioneer

- ✓ Discontinued in 2003, but paved the way for enterprise Linux
- ✓ Introduced the RPM (Red Hat Package Manager) system
- ✓ Legacy continues with Red Hat Enterprise Linux (RHEL)
- ✓ Widely adopted in businesses for reliability and professional support
- ✓ Set the standard for commercial Linux support



Types of Distributions

Popular distributions

➤ **Ubuntu: The Beginner-Friendly**

- ✓ Built on Debian, known for user-friendliness
- ✓ Ideal for beginners, students, and developers
- ✓ Offers stability and a vast software repository
- ✓ Excellent for cloud computing, AI, and machine learning
- ✓ Strong community support and regular updates
- ✓ One of the most popular Linux distros globally



Types of Distributions

Popular distributions

➤ **Fedora: The Cutting-Edge Innovator**

- ✓ Sponsored by Red Hat, focuses on cutting-edge technologies
- ✓ Ideal for developers and sysadmins seeking new features
- ✓ Popular in tech communities for containerization and virtualization
- ✓ Strong support for tools like Podman and KVM



Types of Distributions

Popular distributions

➤ CentOS: The Enterprise Workhorse

- ✓ Based on RHEL, stable without commercial support
- ✓ A server staple until its End-of-Life in 2024
- ✓ Rocky Linux is the successor, offering a community-driven, production-ready alternative

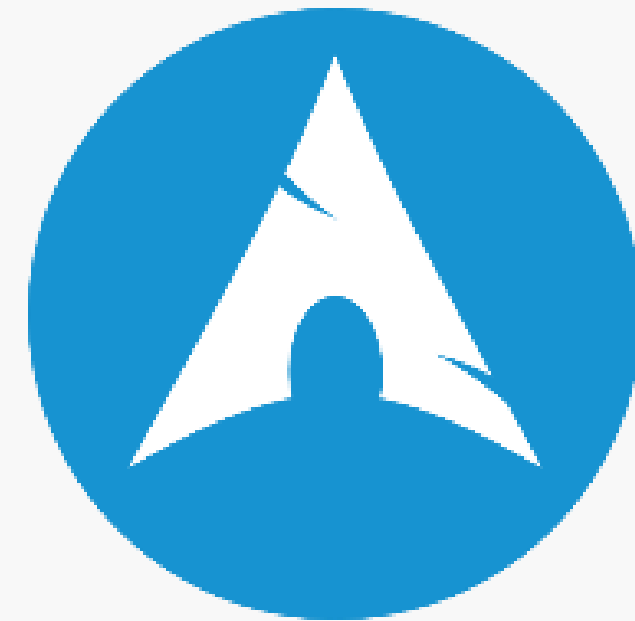


Types of Distributions

Popular distributions

➤ Arch Linux: The DIY Enthusiast's Choice

- ✓ Customizable and minimalist for advanced users
- ✓ Rolling-release for latest software
- ✓ Ideal for power users who configure from scratch



Types of Distributions

Popular distributions

➤ Linux Mint: The Elegant Desktop

- ✓ Built on Ubuntu user-friendly interface
- ✓ Ideal for casual users and home setups
- ✓ Features Cinnamon desktop and multimedia support
- ✓ Stable for daily tasks



Types of Distributions

Popular distributions

➤ openSUSE: The Swiss Army Knife

- ✓ Based on SUSE Linux, known for flexibility
- ✓ Popular among developers and system admins
- ✓ Two editions: Leap for stability, Tumbleweed for latest software
- ✓ Suited for both long-term and cutting-edge users



Types of Distributions

Popular distributions

➤ Manjaro: The User-Friendly Arch

- ✓ Based on Arch Linux, offers easier installation
- ✓ Ideal for gamers, developers, and those seeking a rolling-release model
- ✓ Access to Arch User Repository (AUR)
- ✓ Simplified system management tools for tech enthusiasts



Types of Distributions

Popular distributions

- Its own flavor and purpose, making Linux flexible for a wide range of use cases, from servers and desktops to specialized systems



Ubuntu: The Beginner-Friendly



Fedora: The Cutting-Edge Innovator



CentOS: The Enterprise Workhorse



Arch Linux: The DIY Enthusiast's Choice



Linux Mint: The Elegant Desktop



openSUSE: The Swiss Army Knife



Manjaro: The User-Friendly Arch

Summary



- ❖ **History of Operating Systems:** Brief overview and evolution
- ❖ **Introduction to Linux:** Unique features and core terminologies
- ❖ **Types of Shells:** Exploring the command-line interface
- ❖ **Linux Distributions:** Diversity and strengths of popular distros





Thinknyx Technologies

Linux Course

Enhancing Strategy

Section - 2





Linux | Inside Linux (Architecture, Directory Structure & Booting Process)



Inside Linux



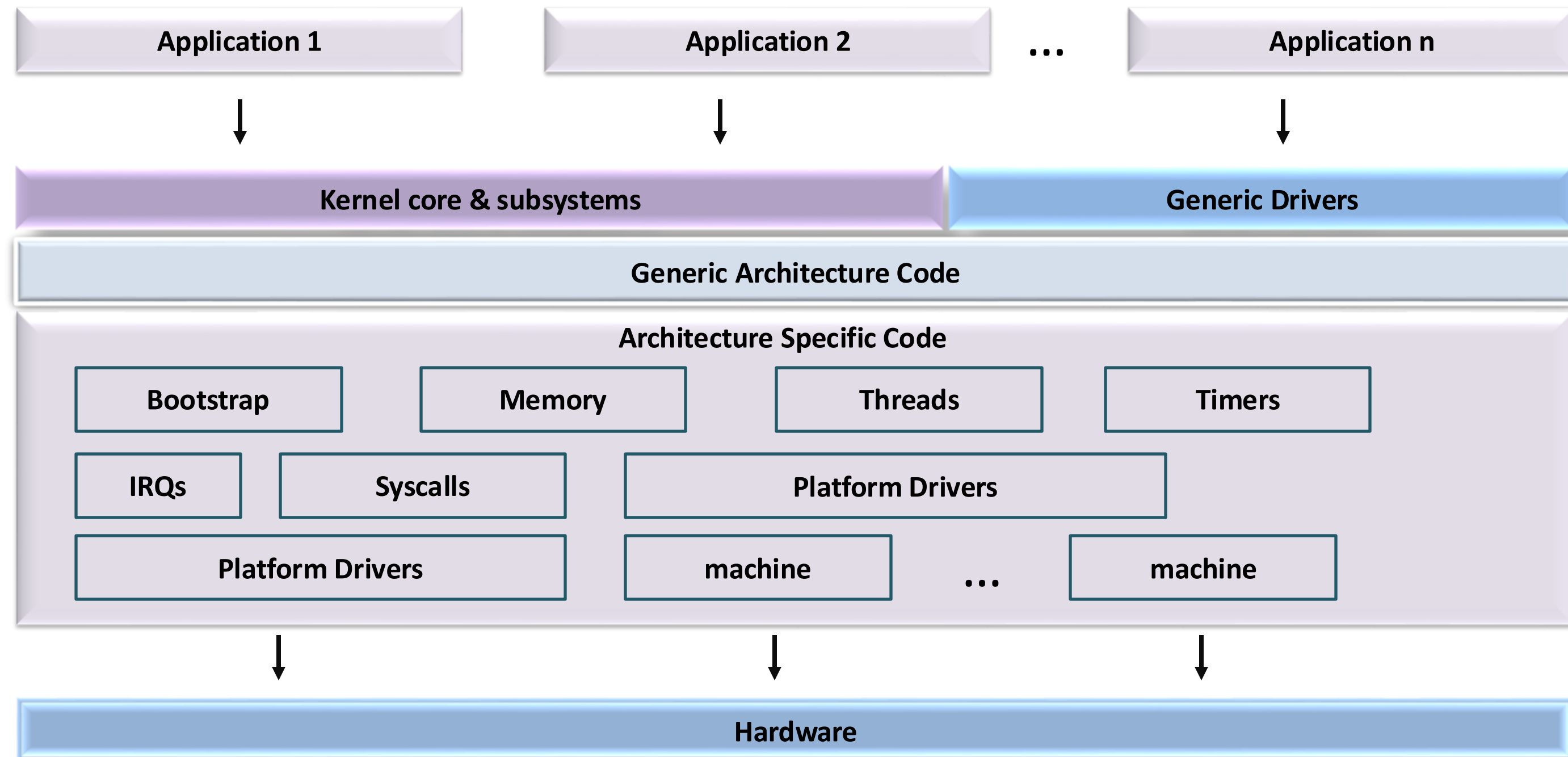
- Architecture of Linux
- Directory Structure
- Important Configuration files
- Bootup process



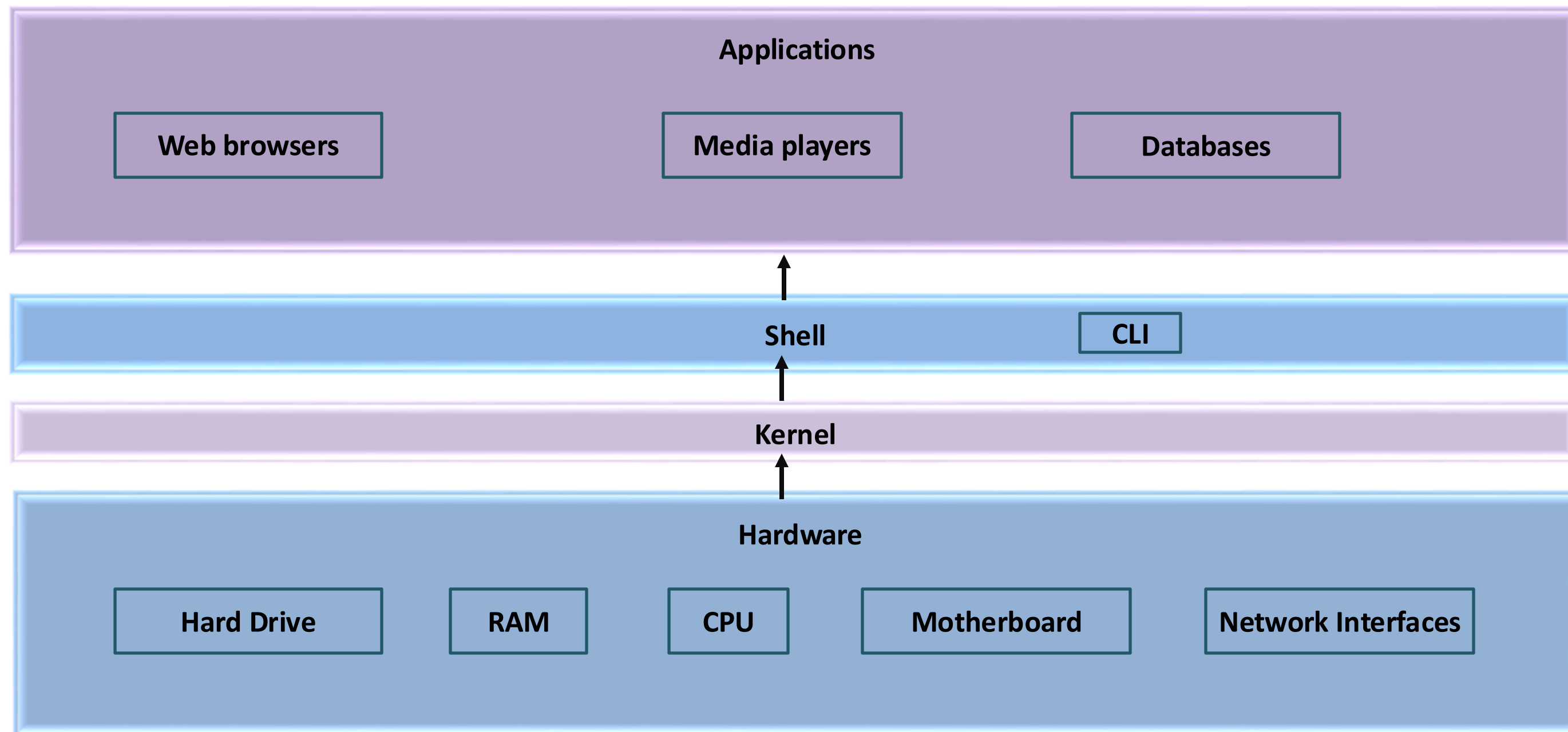
Linux | Architecture



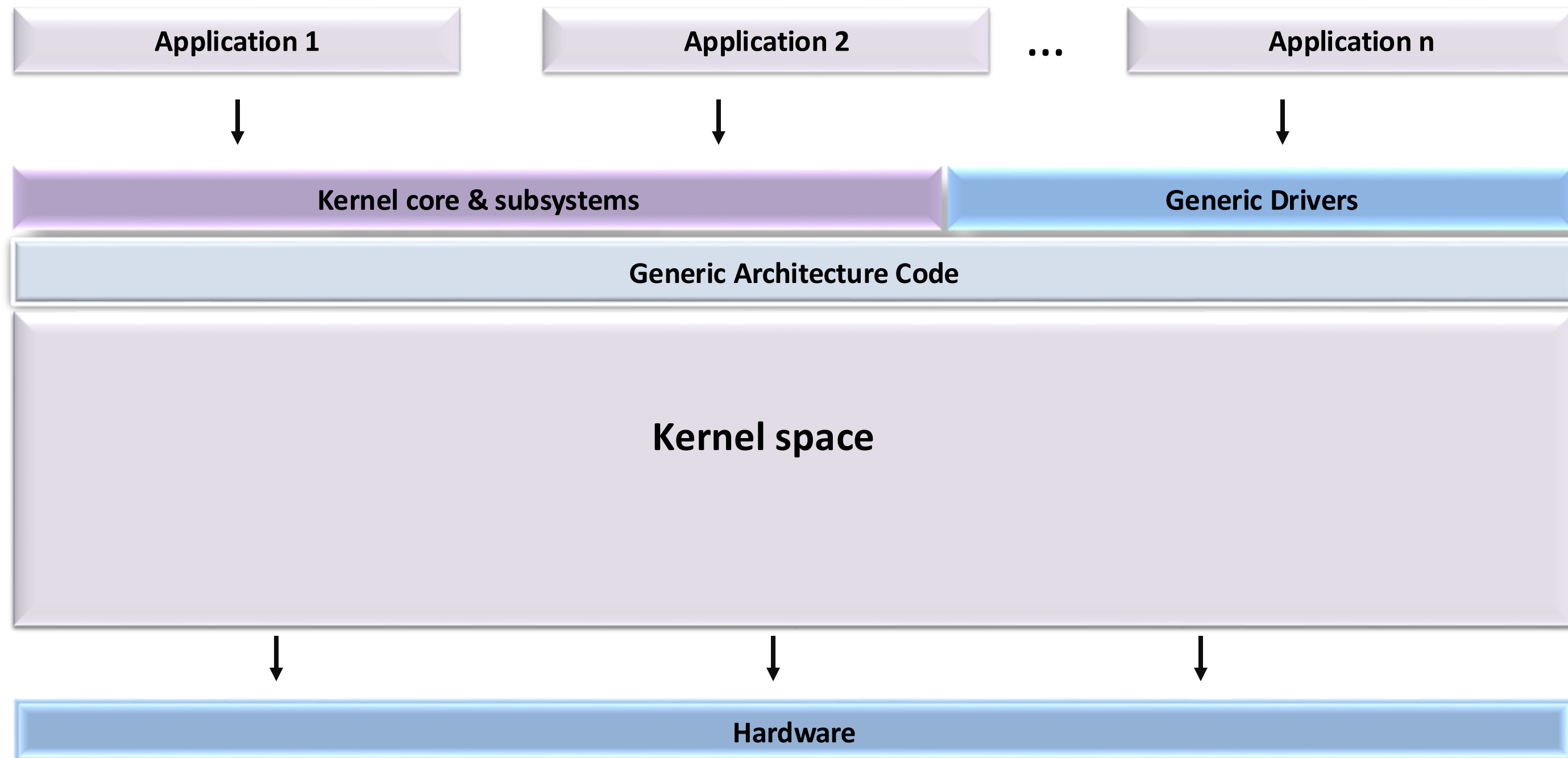
Linux Architecture



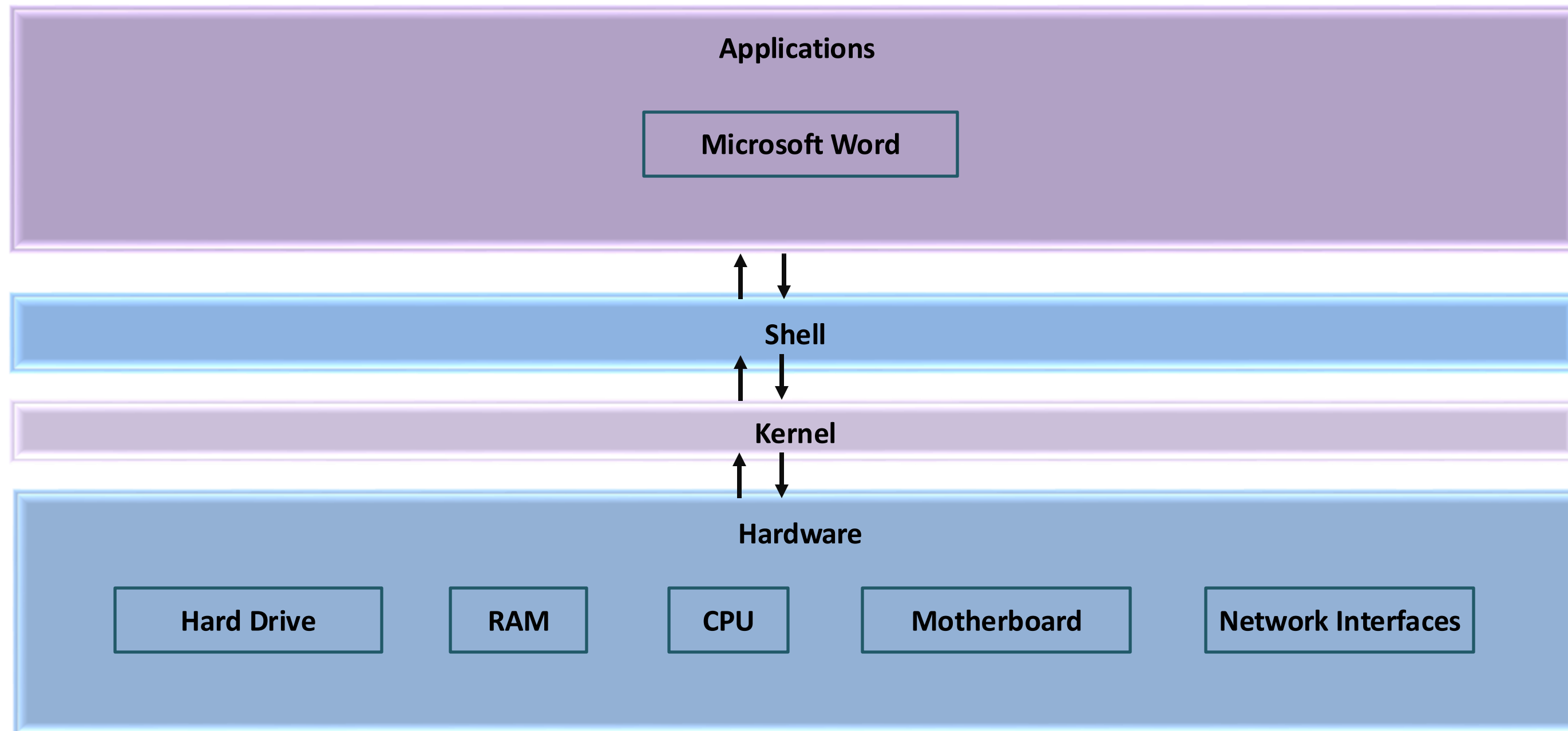
Linux Architecture



Linux Architecture



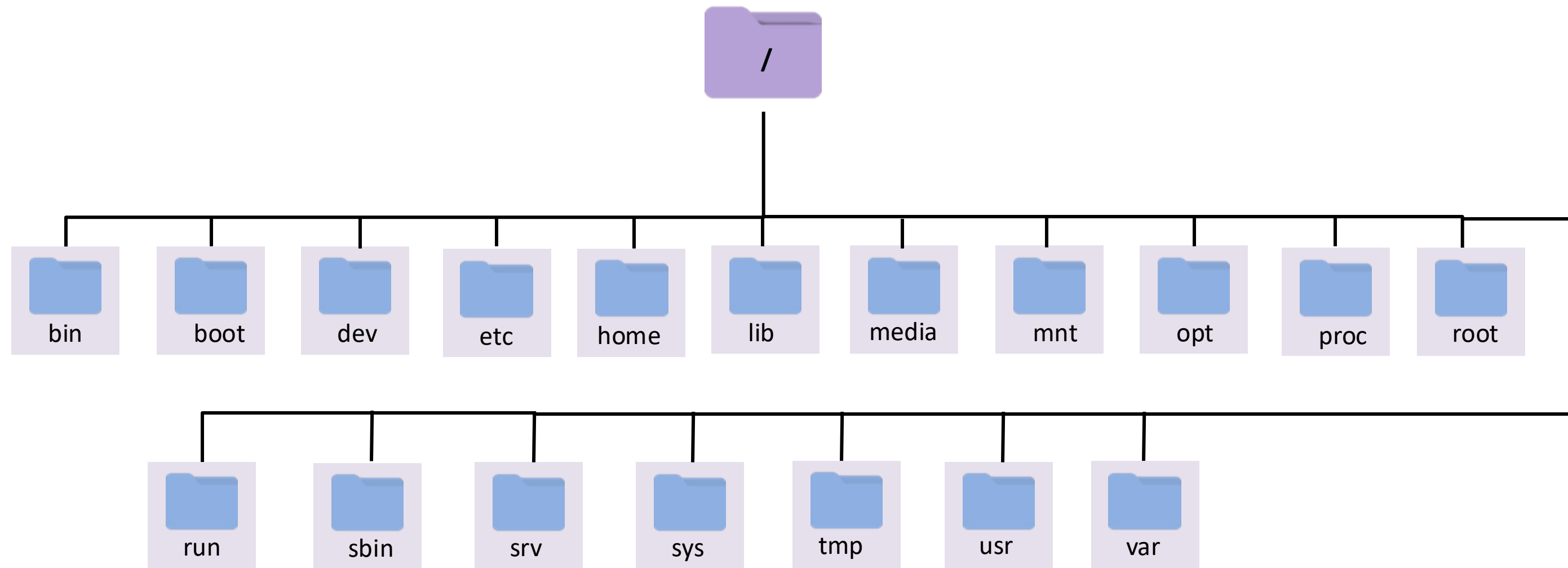
Linux Architecture



Linux Architecture



Linux Directory Structure





Linux | Directory Structure



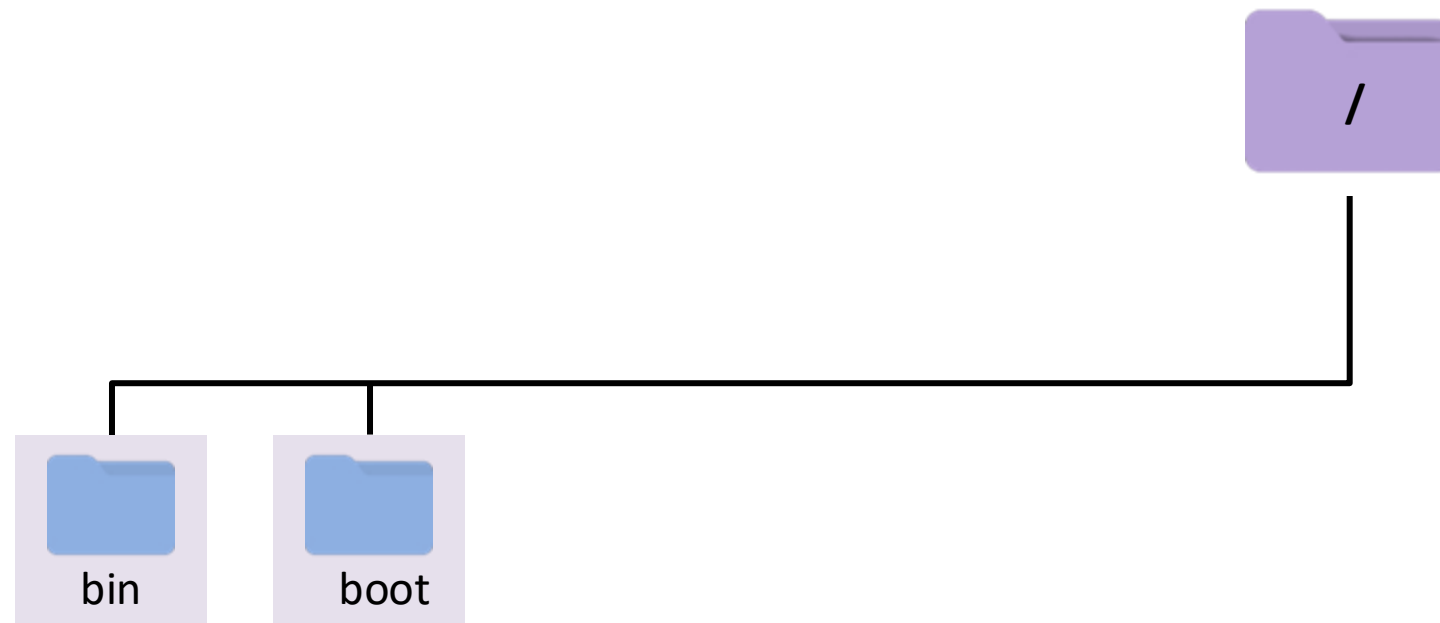
Linux Directory Structure



- Essential user command binaries



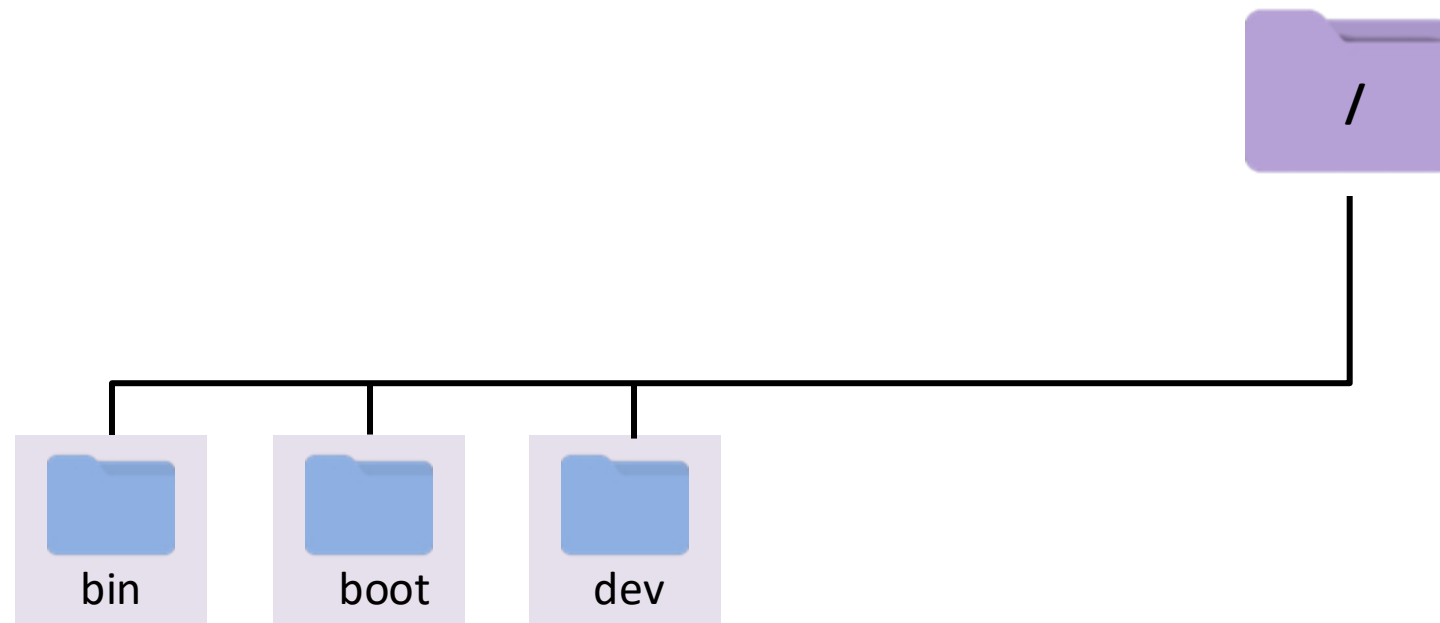
Linux Directory Structure



- Files required for booting the system



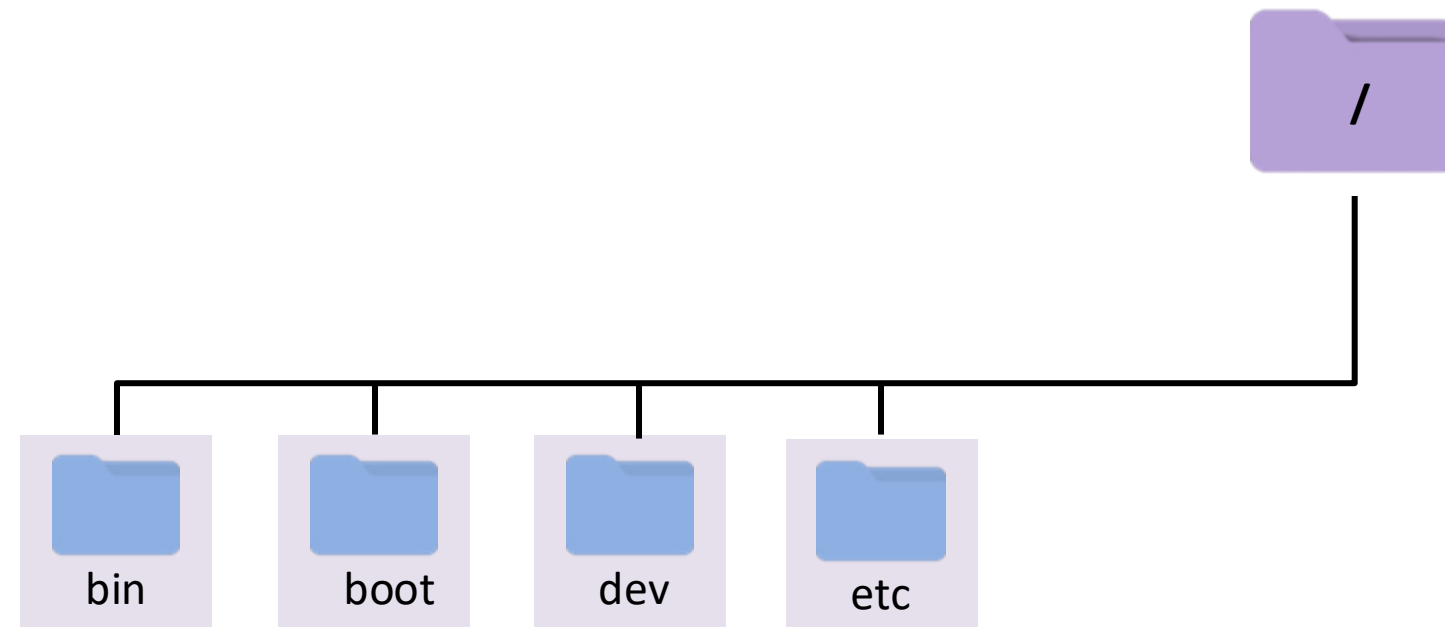
Linux Directory Structure



- Device files



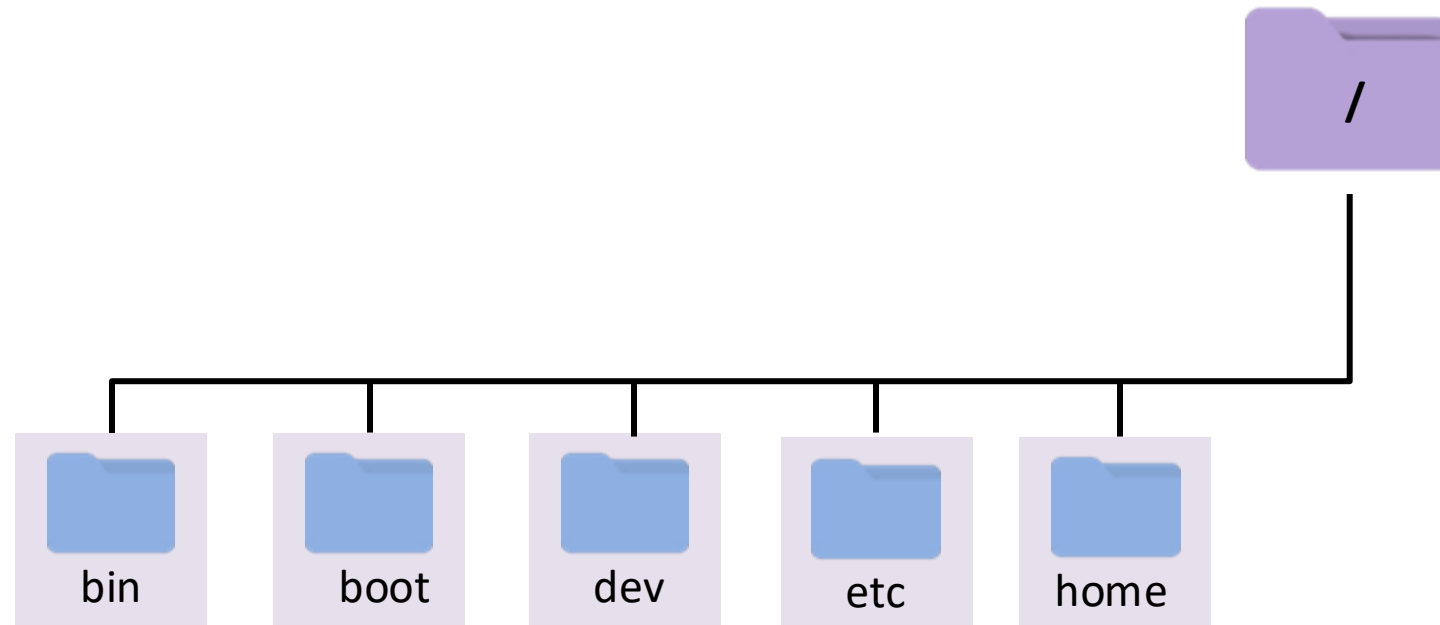
Linux Directory Structure



- Configuration files for the system



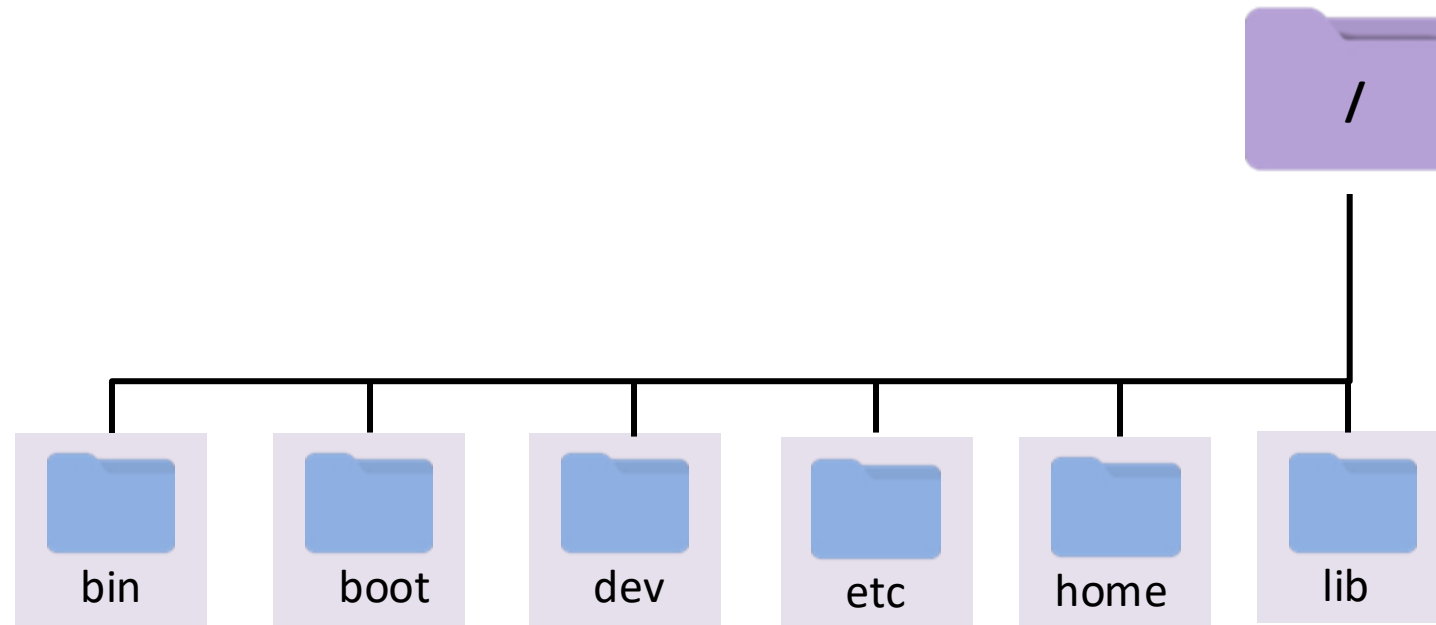
Linux Directory Structure



- Home directory for regular users



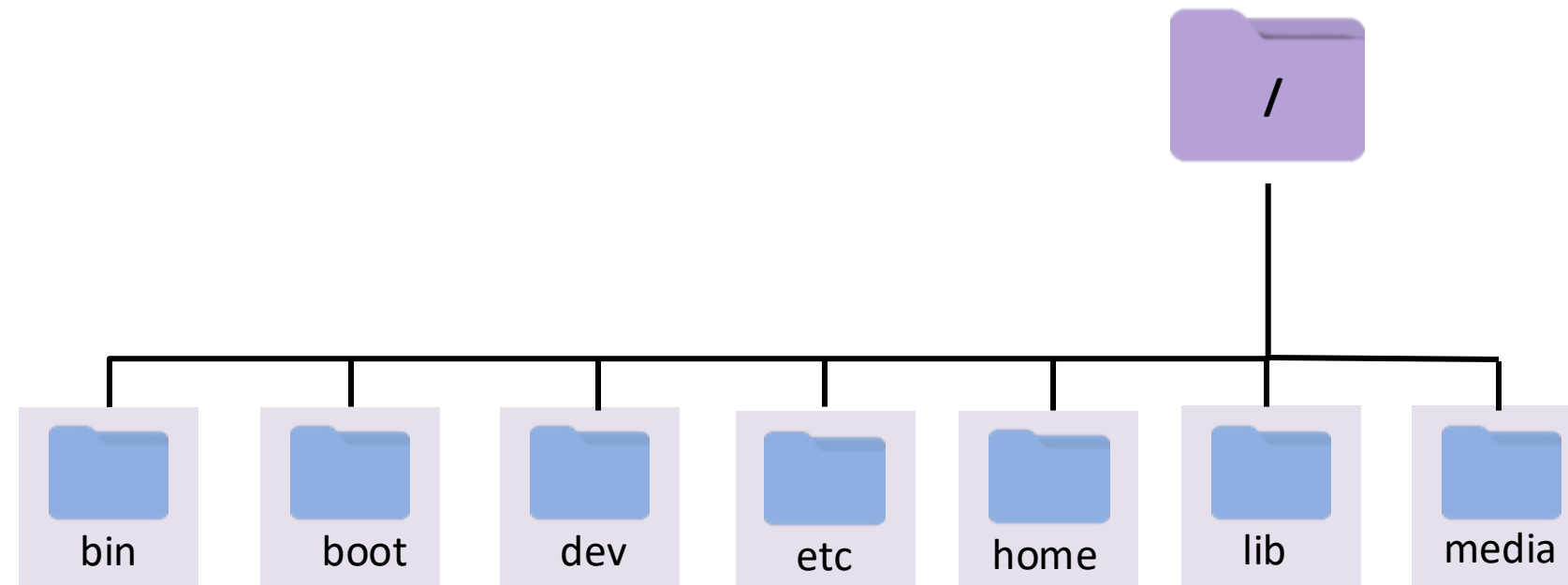
Linux Directory Structure



- Essential shared libraries and kernel modules



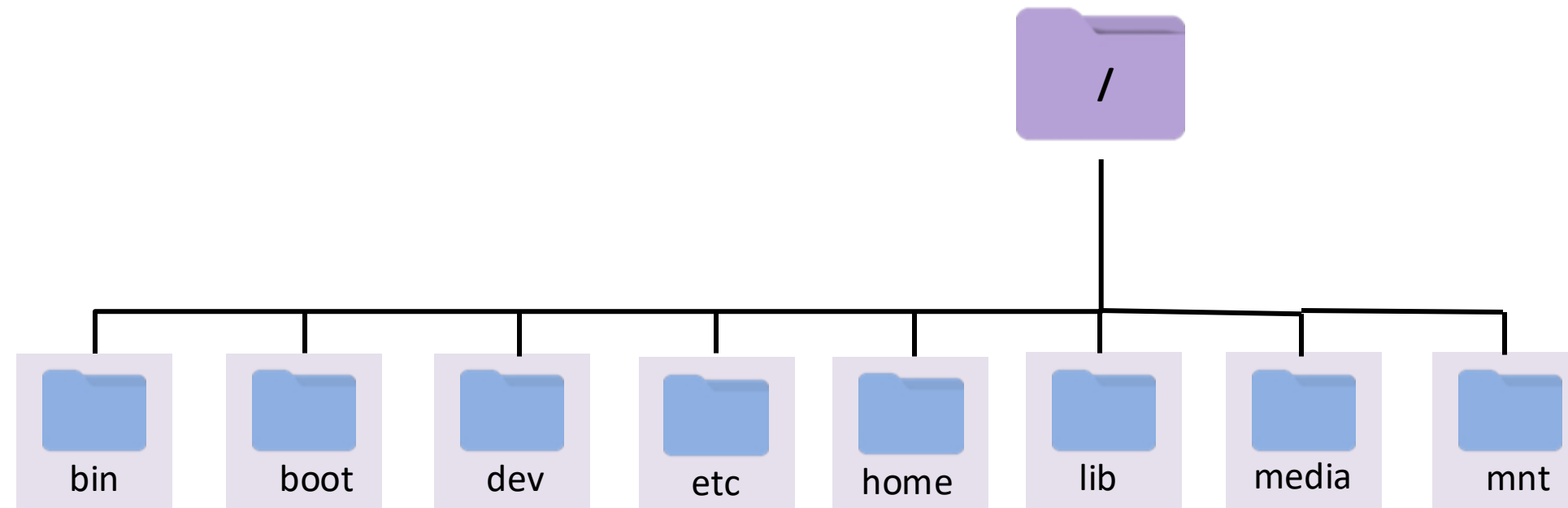
Linux Directory Structure



- Mount point for removable media



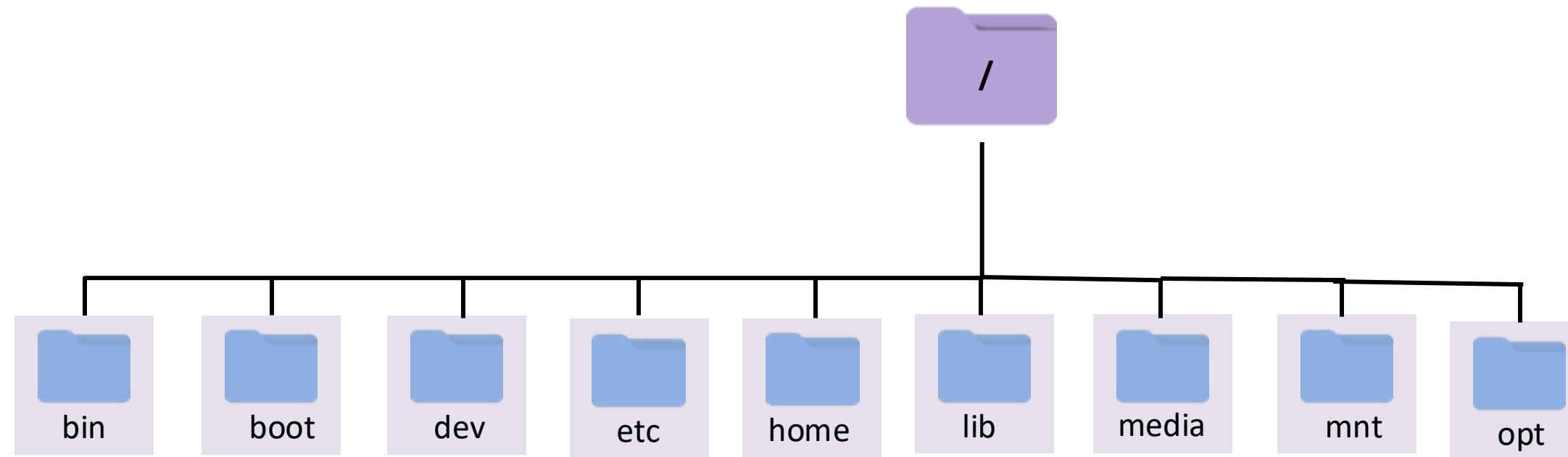
Linux Directory Structure



- Temporary mount point for mounting filesystems



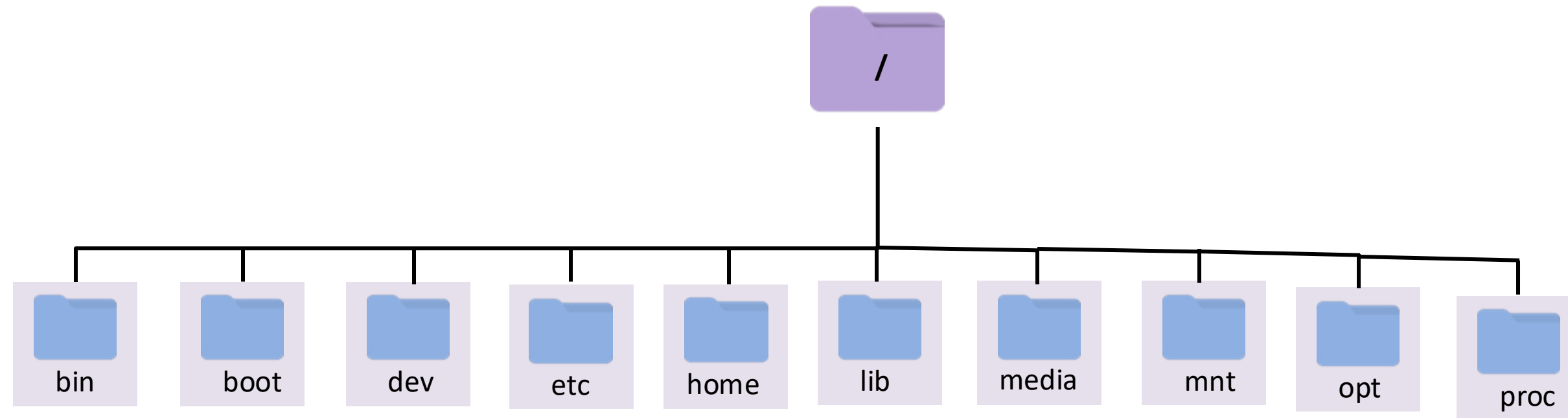
Linux Directory Structure



- Add-on software packages



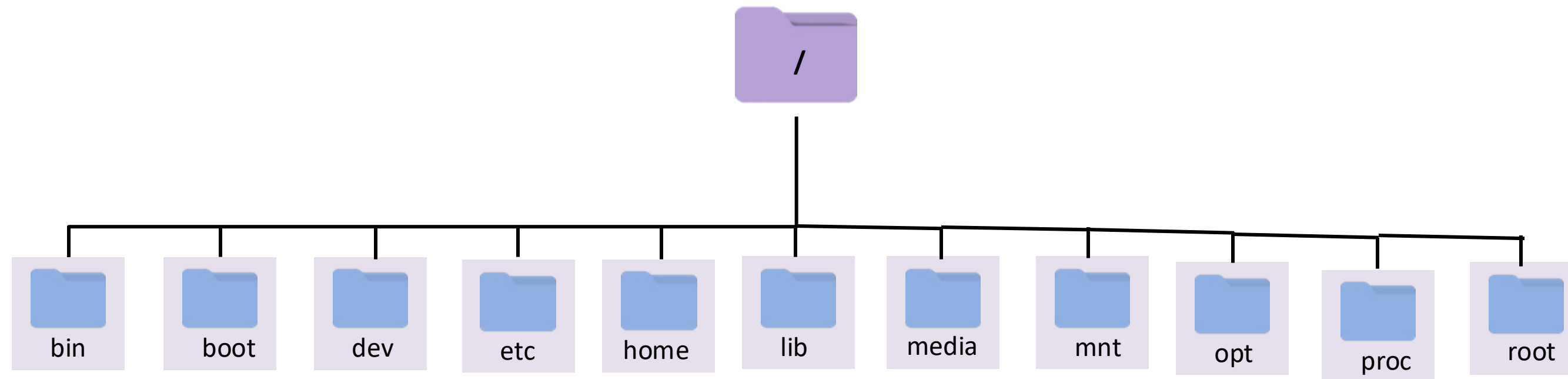
Linux Directory Structure



- Information about running processes and the kernel



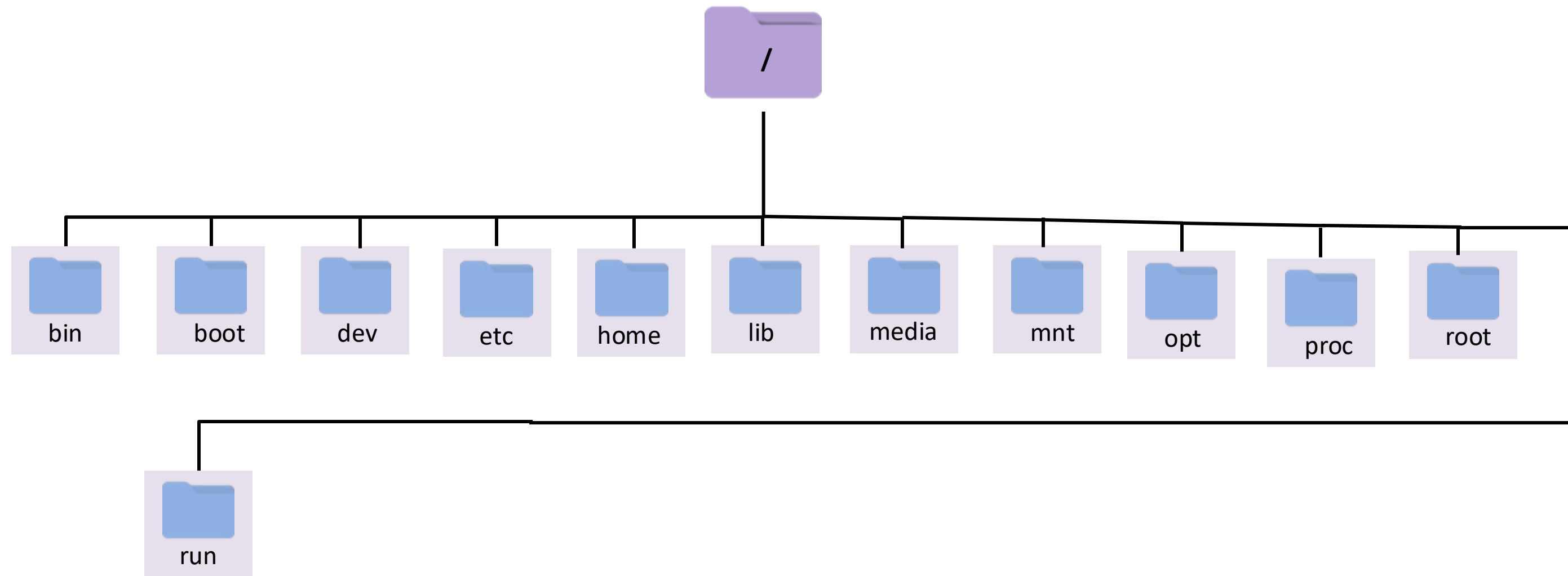
Linux Directory Structure



- Home directory for the root user



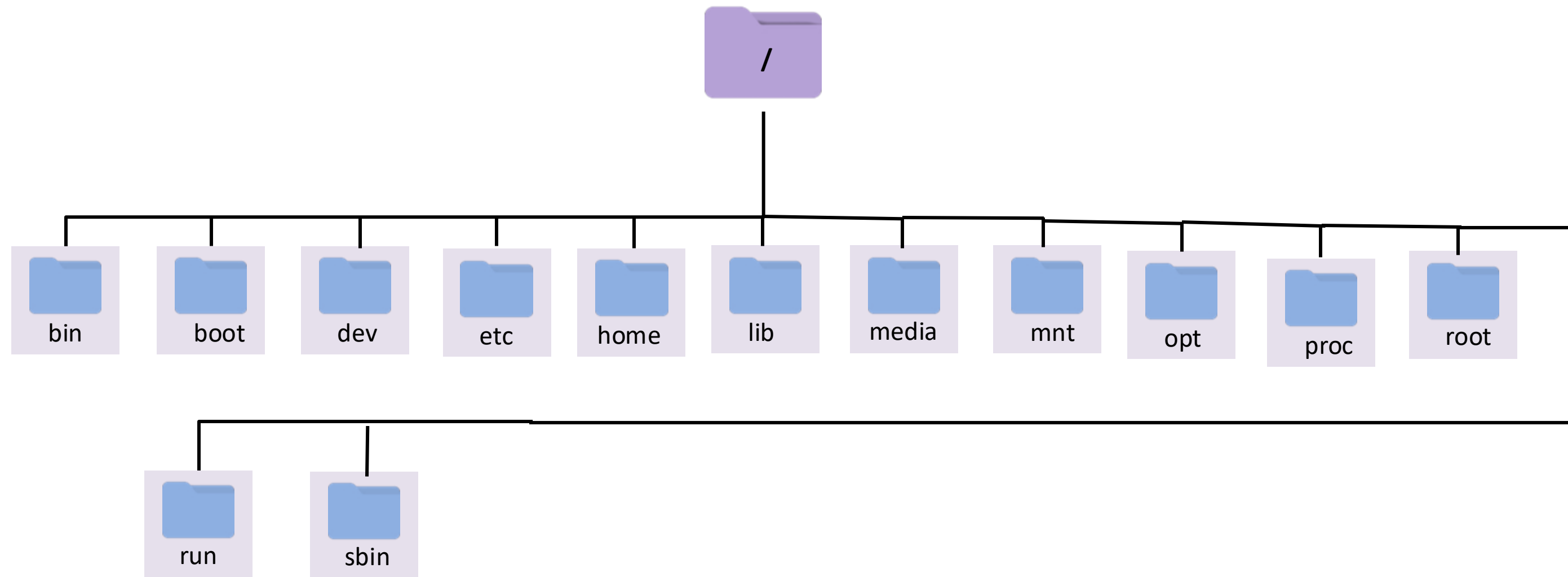
Linux Directory Structure



- Directory is used for information about the current state of the system



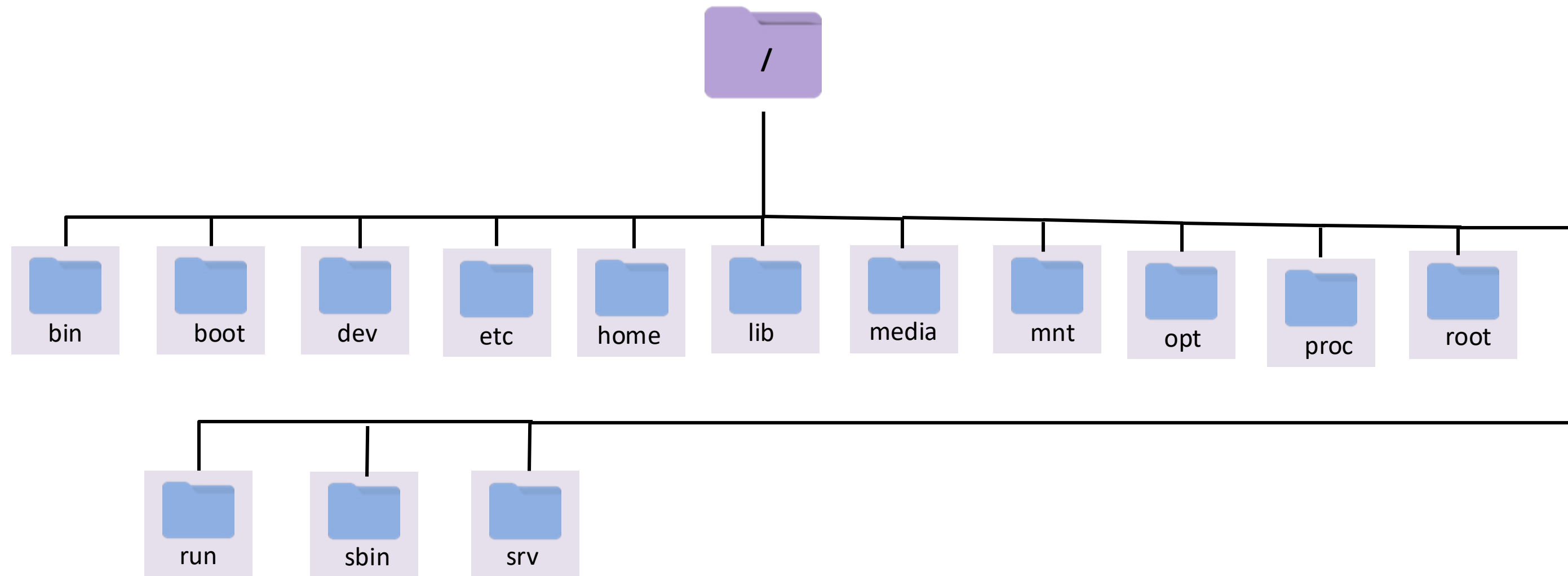
Linux Directory Structure



- System binaries



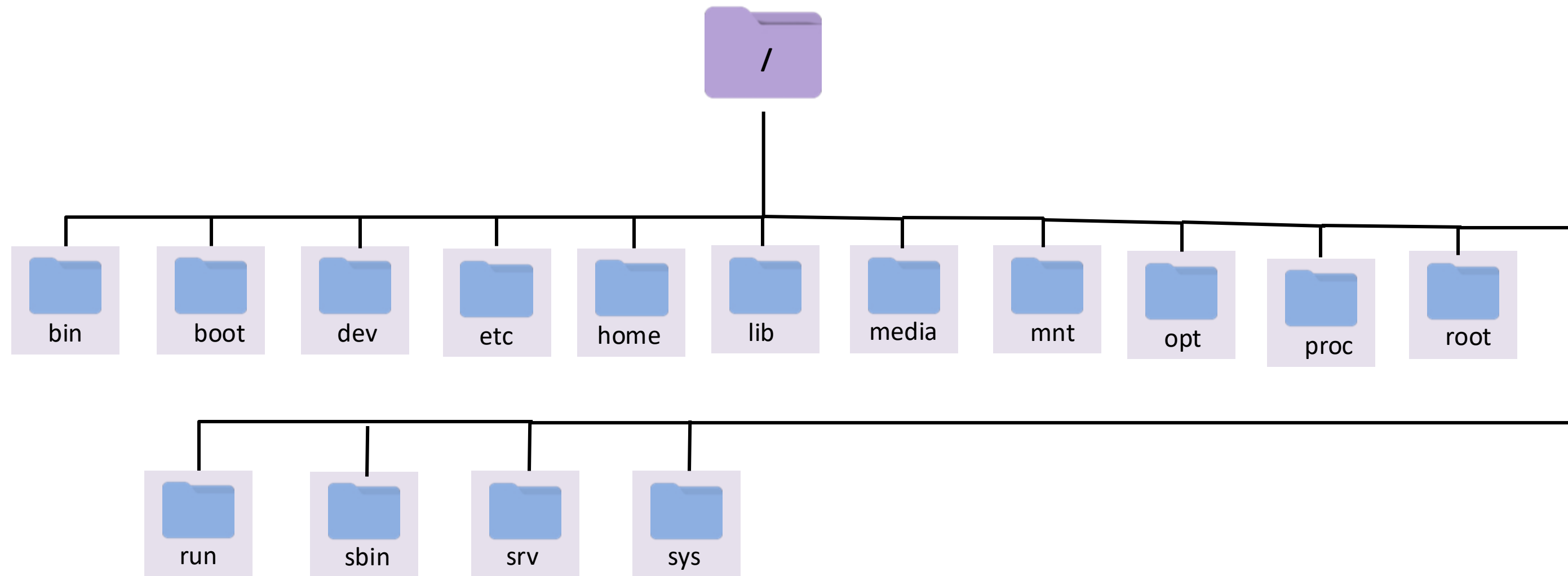
Linux Directory Structure



- Data for services provided by the system



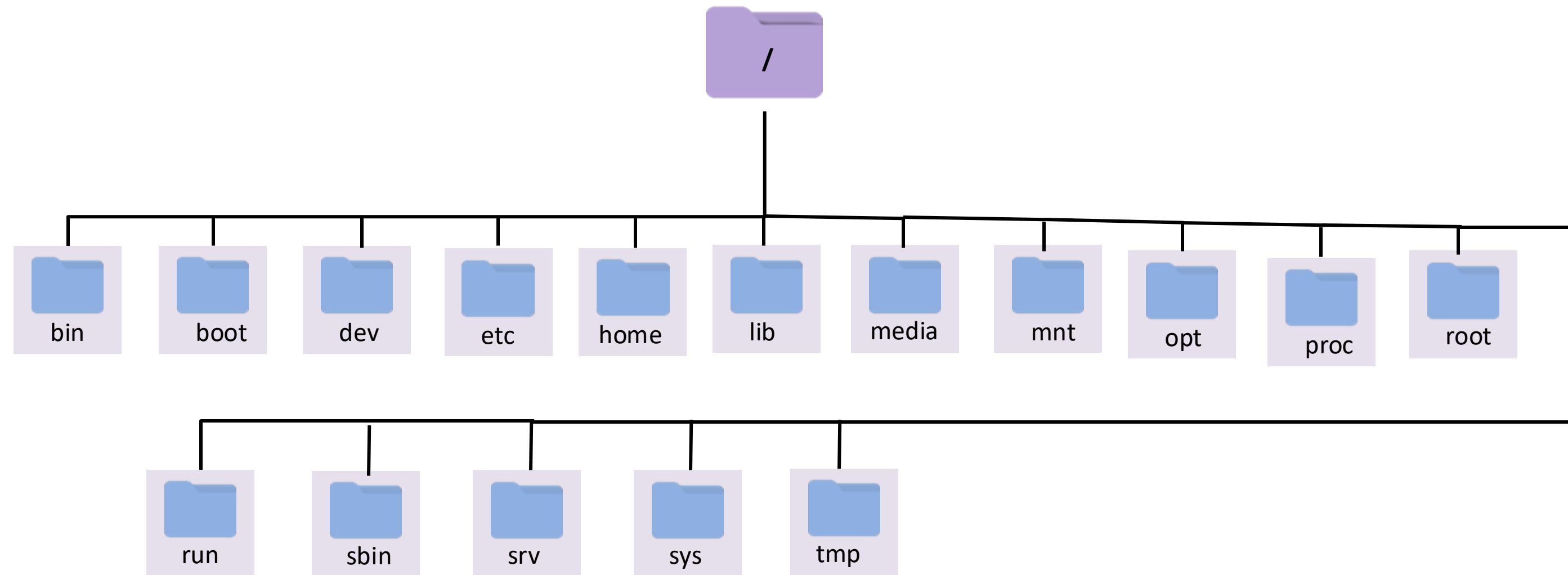
Linux Directory Structure



- Virtual filesystem and mount point for sysfs



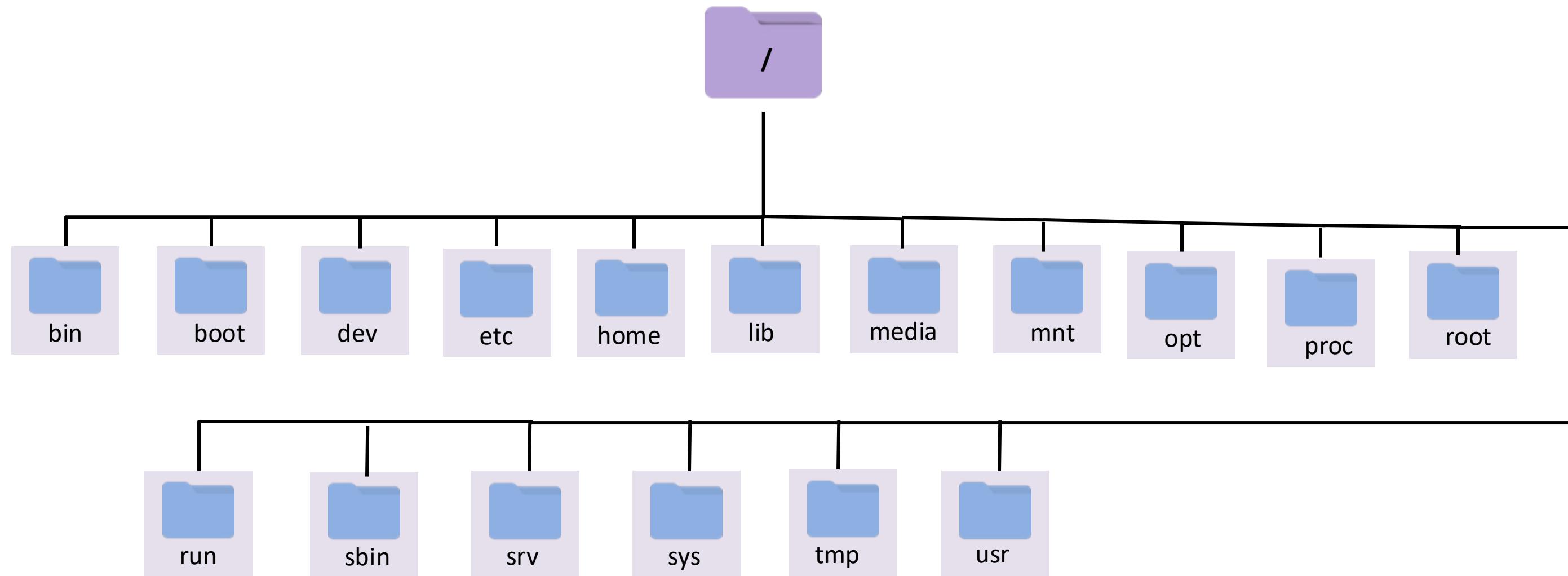
Linux Directory Structure



- Temporary directory



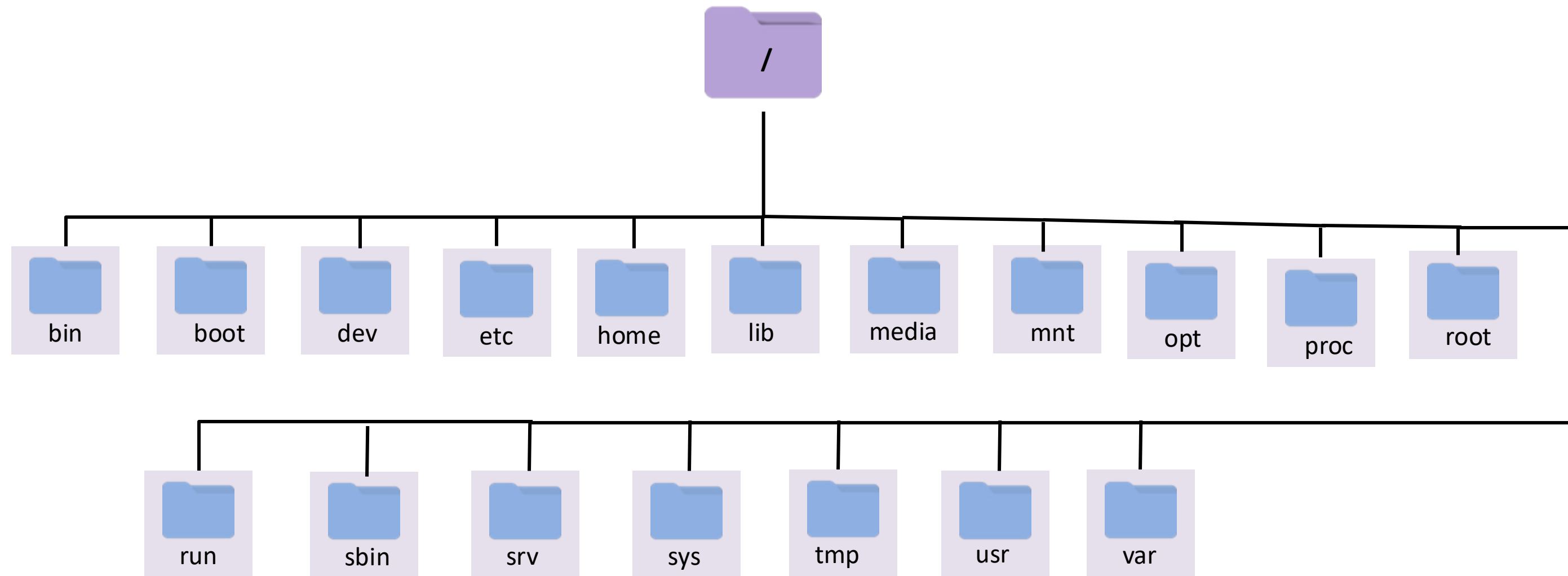
Linux Directory Structure



- Directory contains user programs and data



Linux Directory Structure



- Directory contains variable data files





Linux | Important Configuration Files



Important Configuration Files

Configuration folders	Description
/etc/fstab	All mounted filesystems can be found here and new can be added.
/etc/resolv.conf	To add DNS server entry here for resolving your hostname with it.
/etc/ssh/sshd_config	For enabling remote access via SSH port 22
/etc/passwd	Contains user account details, such as usernames, user IDs, and the default shell assigned to each user.
/etc/group	Stores group information.



Important Configuration Files

Configuration folders	Description
<code>/etc/nsswitch.conf</code>	The Name Service Switch (NSS) configuration file determines the sources from which to obtain name-service information in a range of categories and in what order.
<code>/etc/init.d</code>	Contains scripts linked to run-level directories.
<code>/etc/hosts</code>	Maps IP addresses to hostnames for local/ private networks.
<code>/etc/hosts.allow</code>	Lists host computers that are allowed to use certain TCP/IP services from the local computer.
<code>/etc/hosts.deny</code>	Lists host computers that are not allowed to use certain TCP/IP services from the local computer (doesn't exist by default).



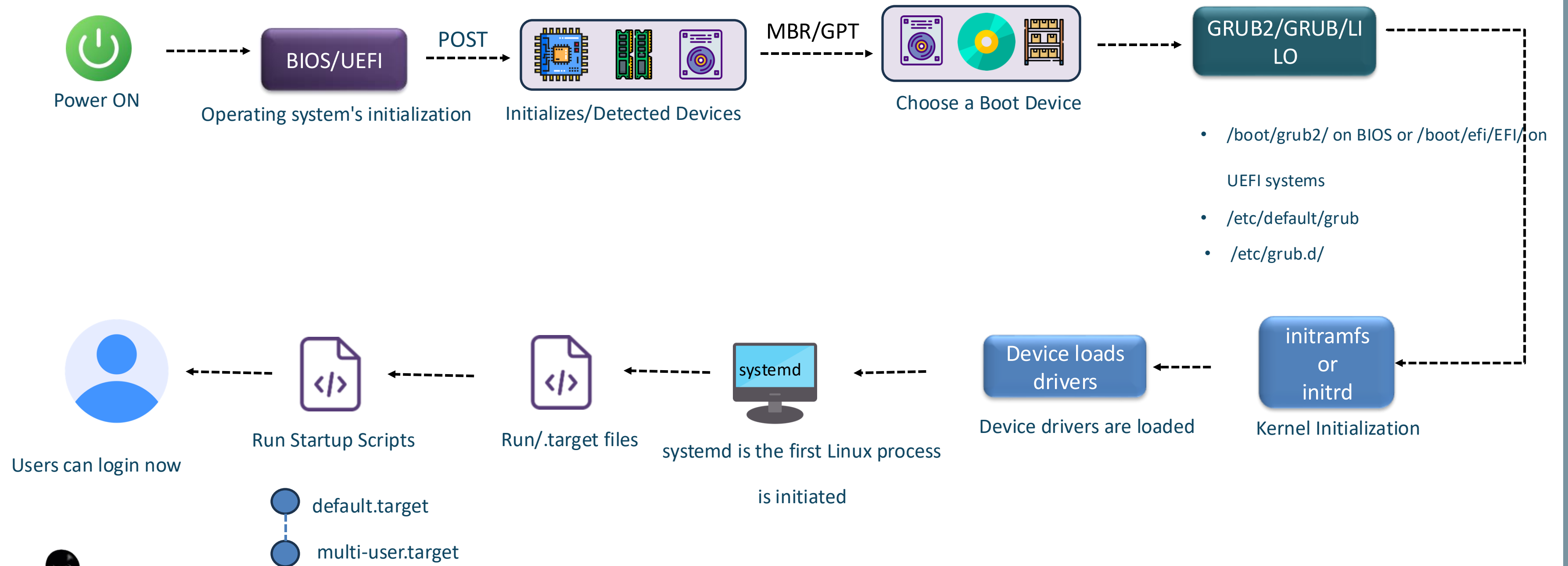


Linux | Bootup Process



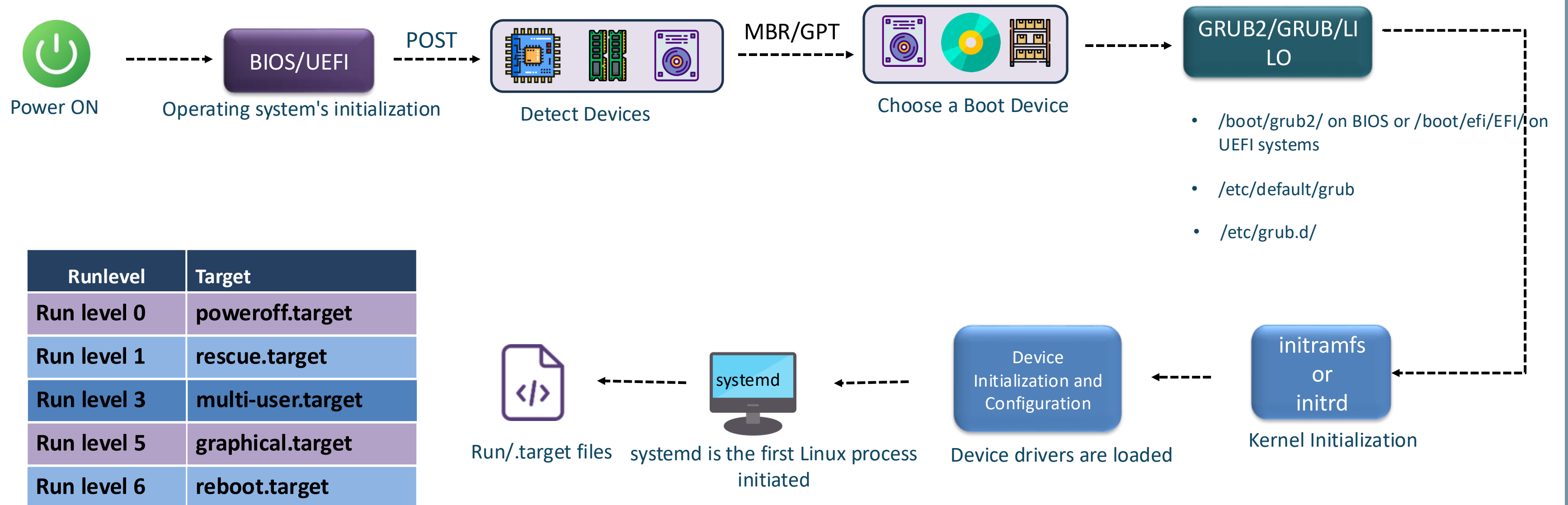
Bootup Process

➤ Generic Booting Process



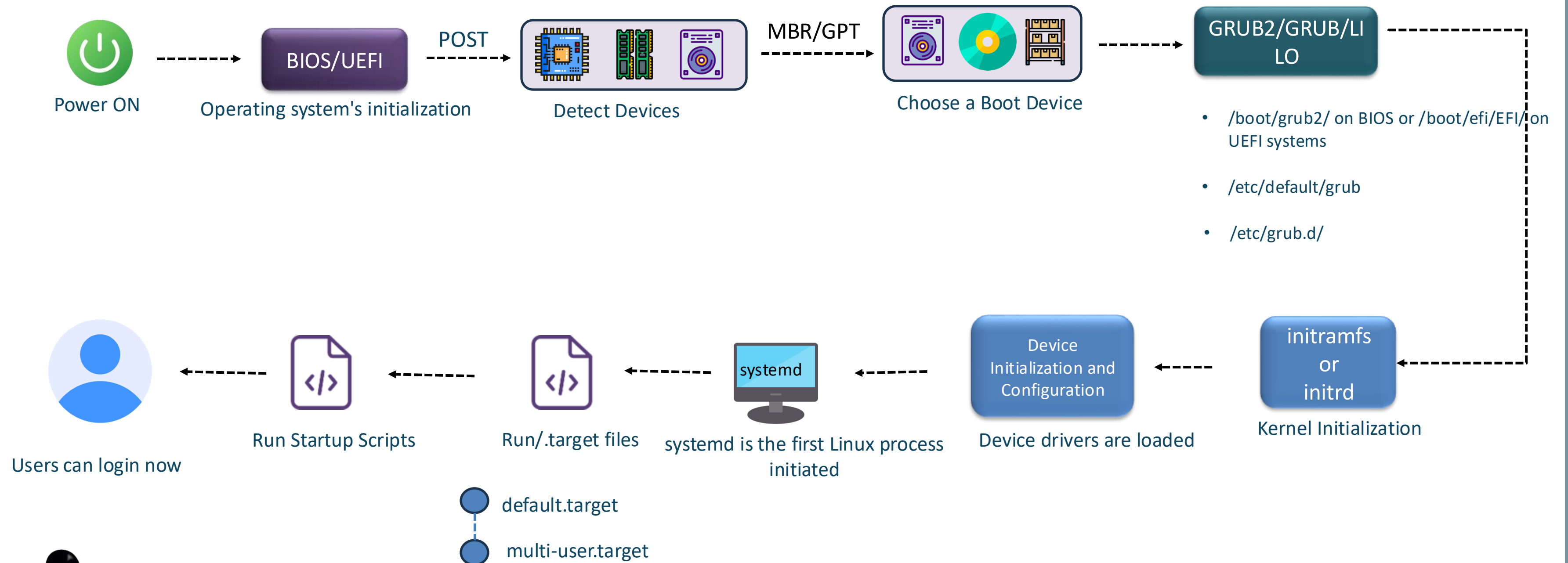
Bootup Process

➤ Generic Booting Process



Bootup Process

➤ Generic Booting Process



Summary



- ❖ **Linux Architecture:** Core components and their interactions
- ❖ **Directory Structure:** Organization and purpose of key directories
- ❖ **Boot Process:** Steps of how Linux boots up





Thinknyx Technologies

Linux Course

Enhancing Strategy

Section - 3





Linux | Setting up Linux



Setting up Linux





Linux | Setting up Linux in Diverse Environments



Setting up Linux in Diverse Environments



Virtual machines



Cloud instances



Containers



Ubuntu

Debian



RHEL 9

RPM



Setting up Linux in Diverse Environments

- Setting Up Linux on Oracle VirtualBox with an Ubuntu Image file



Setting up Linux in Diverse Environments

➤ Setting Up Linux on Oracle VirtualBox with Vagrant



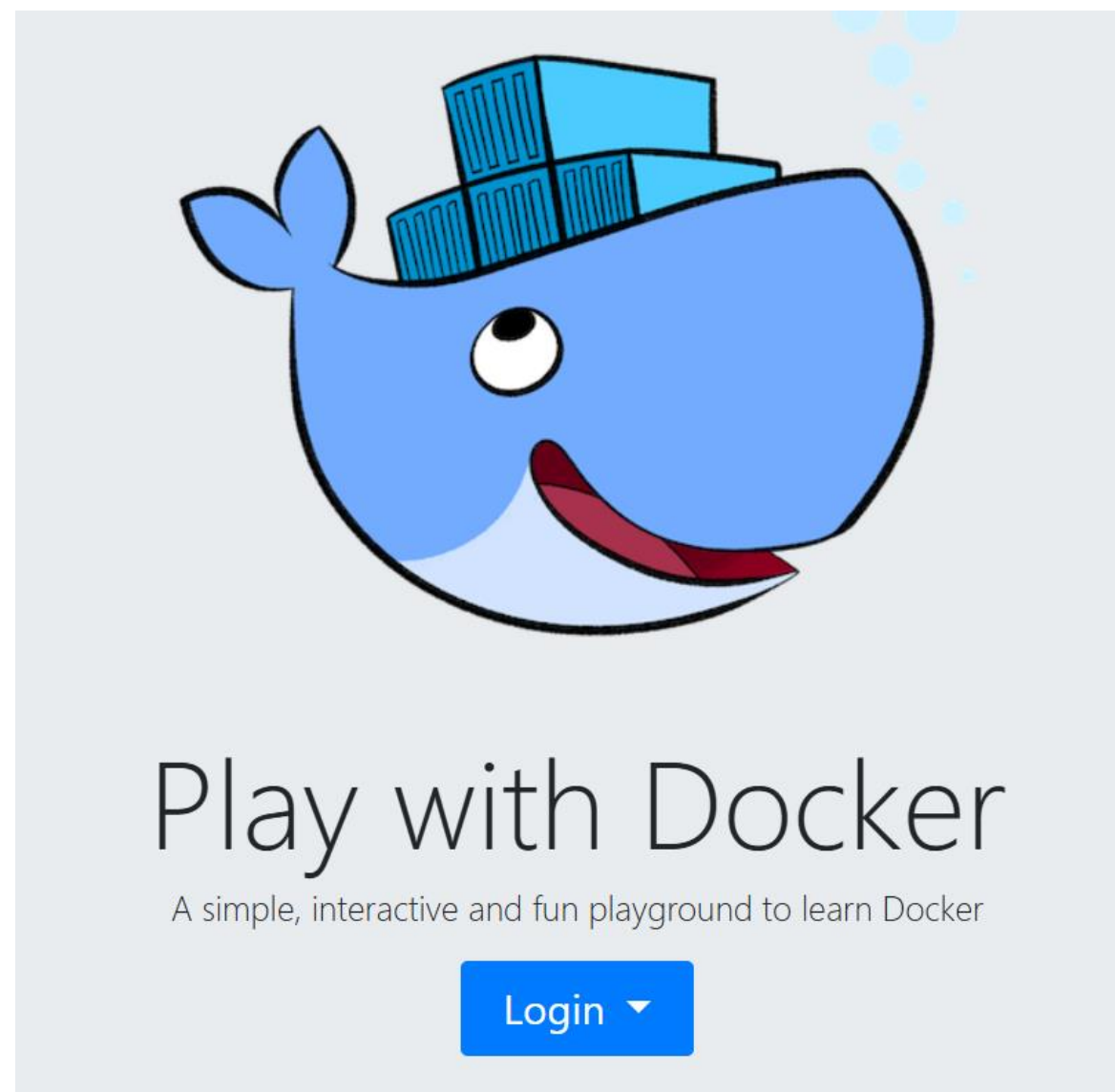
Setting up Linux in Diverse Environments

- Creating a Virtual Machine on the Cloud



Setting up Linux in Diverse Environments

➤ Quick Start with Play with Docker

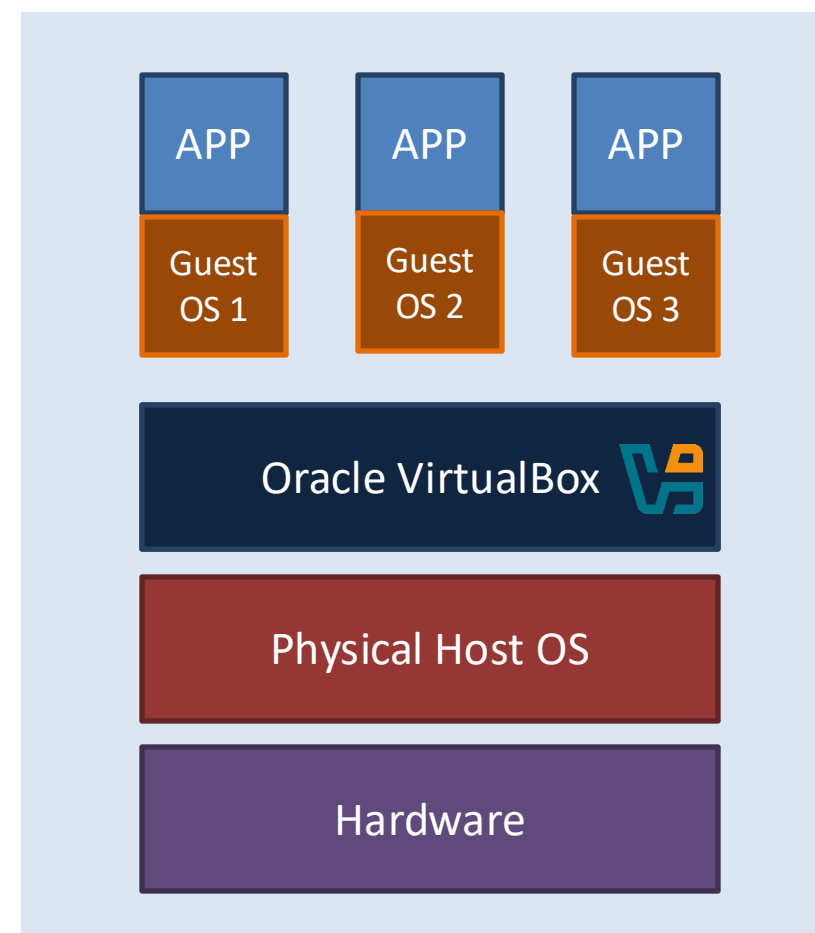




Demonstration | Getting started with Linux on VirtualBox

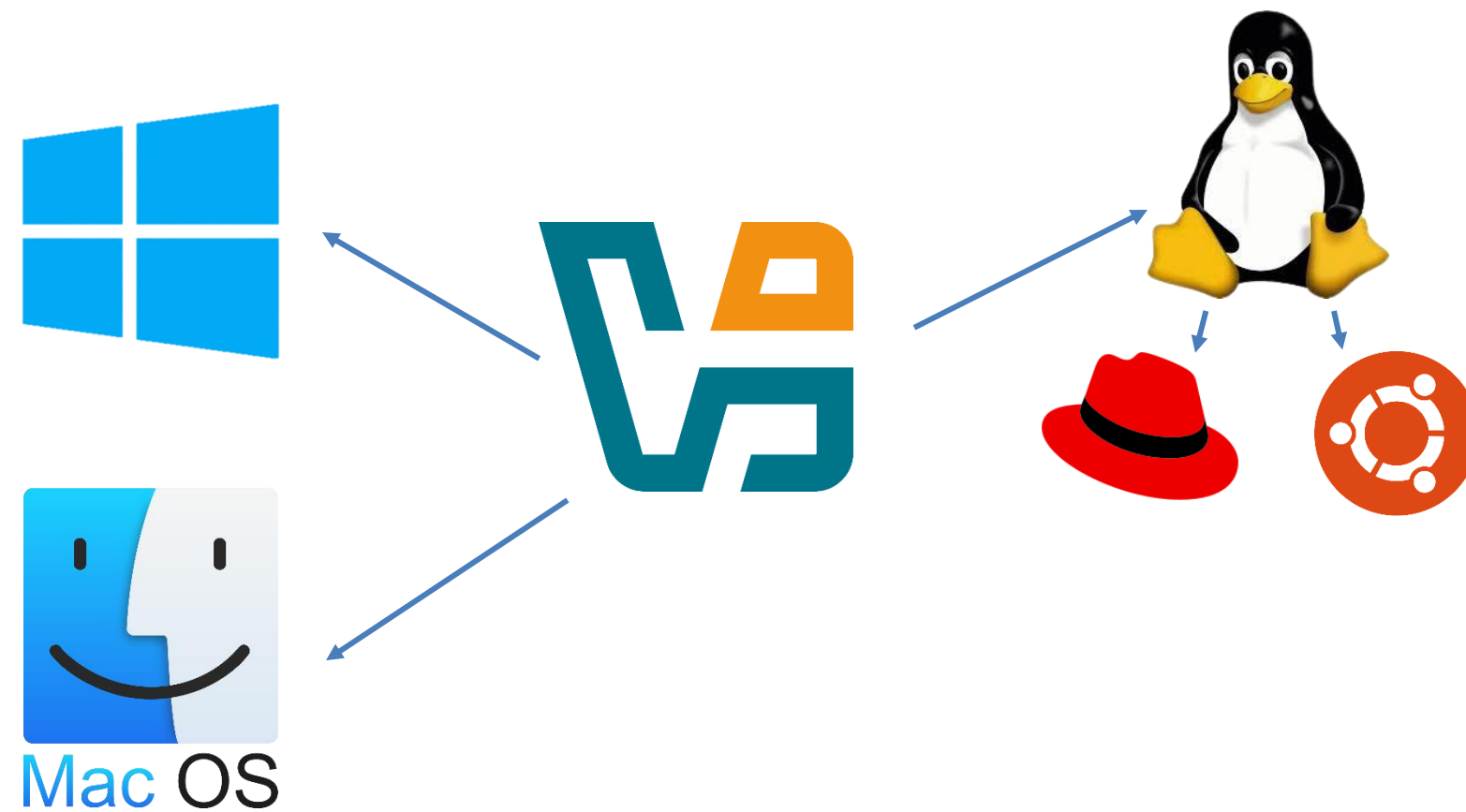
Getting started with Linux on VirtualBox

➤ Introduction to Virtualization



Getting started with Linux on VirtualBox

➤ Introduction to Virtualization



Getting started with Linux on VirtualBox

➤ Features of Oracle Virtual BOX

- ✓ Snapshot capabilities
- ✓ Flexible networking options
- ✓ Support for various guest operating systems

☐ List of Supported Host OS for VirtualBox

<https://www.virtualbox.org/manual/ch01.html#hostossupport>

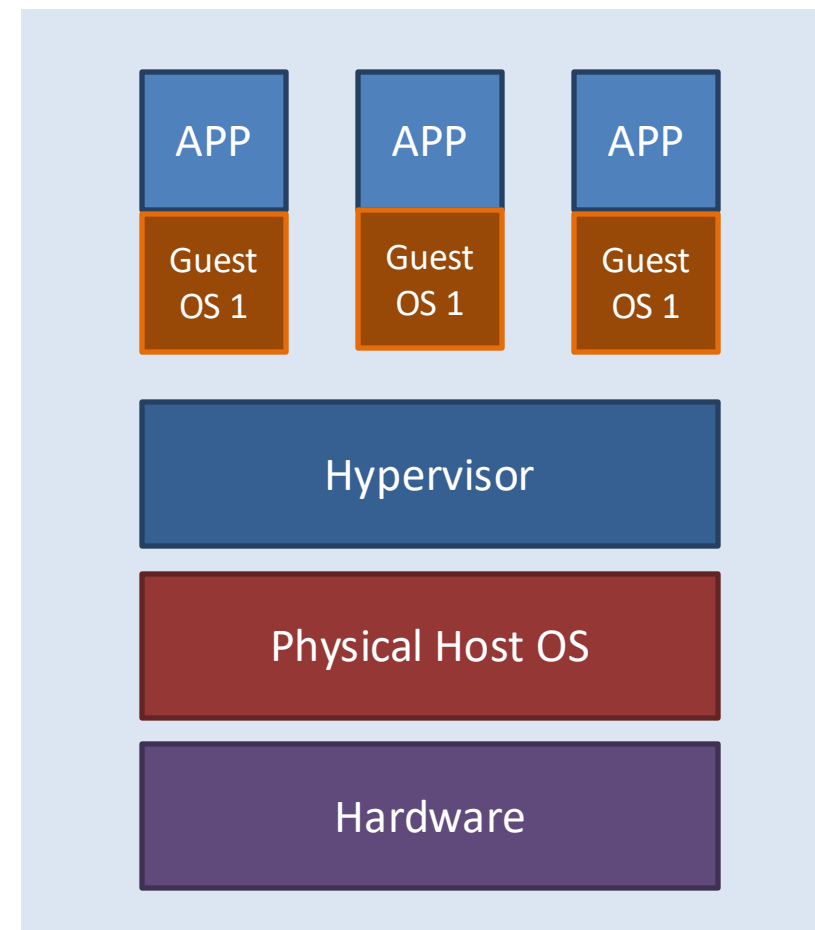
☐ List of Supported Guest OS for VirtualBox

<https://www.virtualbox.org/manual/ch03.html#guestossupport>



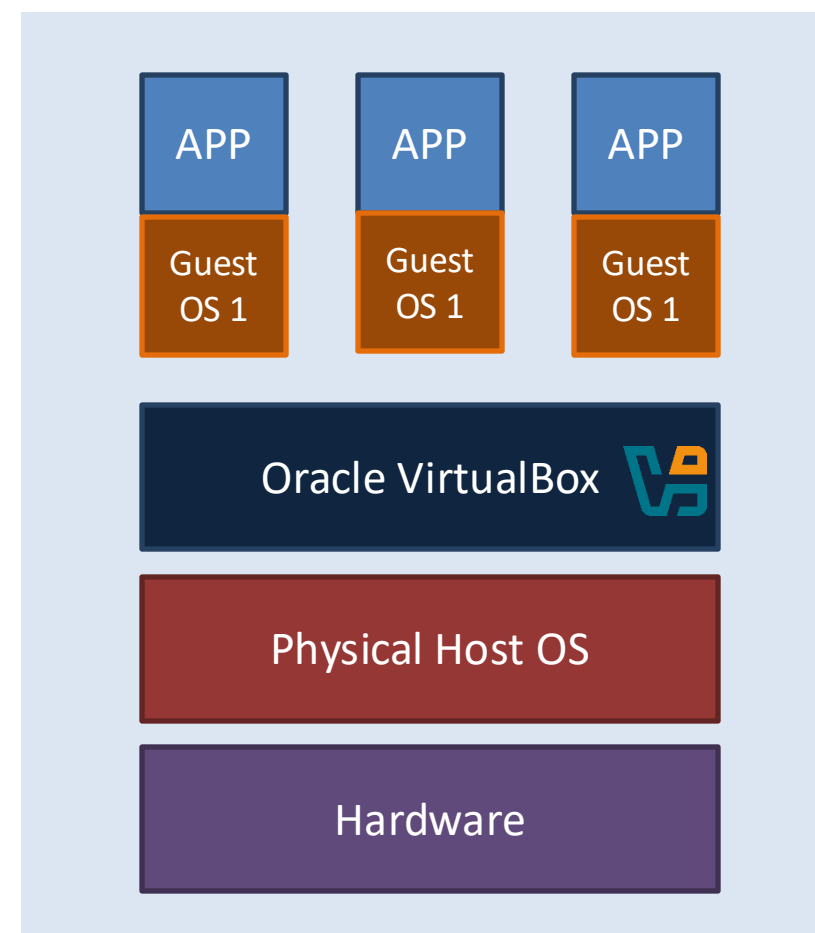
Getting started with Linux on VirtualBox

➤ Introduction to Virtualization



Getting started with Linux on VirtualBox

➤ Introduction to Virtualization





Demonstration | Accessing Linux Machine via Putty

Getting started with Linux on VirtualBox

➤ Accessing a Linux Machine via PuTTY

PuTTY



✓ Free open-source SSH client





Demonstration | Getting started with Linux on Vagrant



Demonstration | Getting started with Linux on Cloud



Demonstration | Getting started with Linux on Container

Summary



❖ Traditional VM Setup:

Oracle VirtualBox with Ubuntu ISO

Download, installation, and configuration

❖ Automated Setup with Vagrant:

Vagrant and Oracle VirtualBox

❖ Cloud-Based Environment:

Ubuntu instance on AWS

❖ Quick Container Setup:

Play with Docker





Thinknyx Technologies

Linux Course

Enhancing Strategy

Section - 4





Linux | Introduction to Essential Linux Commands



Essential Linux Commands



- Importance of Terminal
- Work with Directories, Files and File Contents

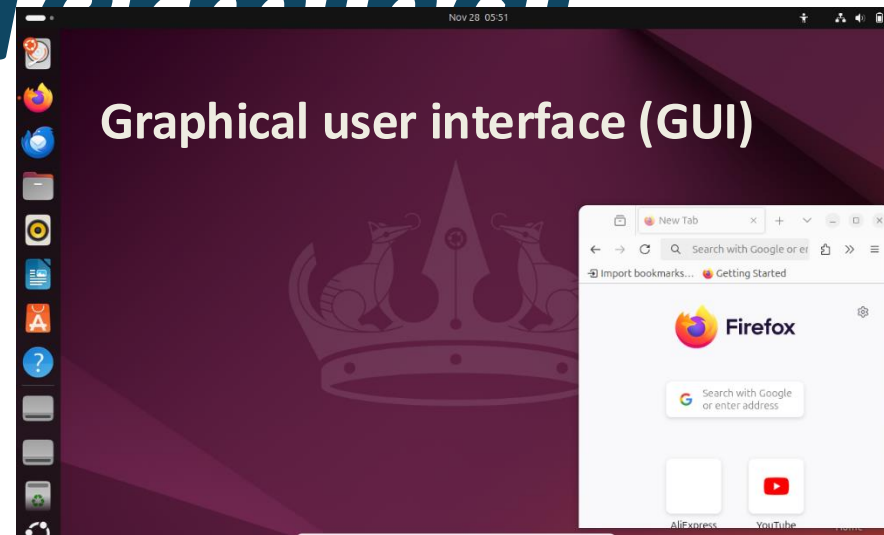




Linux | Getting Started with Terminal



Getting Started with Terminal



- ✓ Visual and Intuitive Navigation
- ✓ Beginner-Friendly
- ✓ Point-and-Click Interface



- ✓ Powerful Text-Based Approach
- ✓ Speed and Precision
- ✓ Greater Control
- ✓ Preferred by advanced users and system administrators



Getting Started with

Terminal

Basic Elements of a Shell Command

- ✓ The shell is a program that interprets user commands
- ✓ Commands are names of programs or scripts installed on the system
- ✓ Each command may include options and arguments



Getting Started with Terminal

Basic Elements of a Shell Command

Syntax:

Command [Options] [Arguments]

Command	Description
ls	Lists files and directories in the current directory
ls -l	Lists files and directories in a detailed (long) format in the current directory
ls -l /dev	Lists files and directories in the specified directory (/dev) with detailed information

```
ubuntu@thinknyxlinux:~$ ls -l /dev
total 0
crw-r--r-- 1 root root    10, 235 Nov 14 12:06 autofs
drwxr-xr-x 2 root root    300 Nov 14 12:07 block
crw----- 1 root root    10, 234 Nov 14 12:06 btrfs-control
drwxr-xr-x 2 root root  2620 Nov 14 12:07 char
crw--w---- 1 root tty      5,   1 Nov 14 12:07 console
lrwxrwxrwx 1 root root    11 Nov 14 12:06 core -> /proc/kcore
drwxr-xr-x 3 root root    60 Nov 14 12:06 cpu
crw----- 1 root root   10, 122 Nov 14 12:06 cpu_dma_latency
crw----- 1 root root   10, 203 Nov 14 12:06 cuse
drwxr-xr-x 6 root root   120 Nov 14 12:07 disk
drwxr-xr-x 2 root root    60 Nov 14 12:06 dma_heap
crw----- 1 root root   10, 125 Nov 14 12:06 ecryptfs
lrwxrwxrwx 1 root root    13 Nov 14 12:06 fd -> /proc/self/fd
crw-rw-rw- 1 root root    1,   7 Nov 14 12:06 full
crw-rw-rw- 1 root root   10, 229 Nov 14 12:06 fuse
crw----- 1 root root   10, 228 Nov 14 12:06 hpet
drwxr-xr-x 2 root root    10, 128 Nov 14 12:06 hugepages
vcsu3
vcsu4
vcsu5
vcsu6
vfio
vga_arbiter
vhost-net
vhost-vsock
xen
xvda
xvda1
xvda14
xvda15
xvda16
zero
zfs
```



Getting Started with Terminal

Basic Commands

Command	Description	Syntax	Usage
echo	Prints the string(s) to standard output	echo <string(s)>	echo "Hello, Linux!"
uname	Prints the Systems Information	uname [options]	uname -a
whoami	Prints the username associated with the current effective user ID	whoami	whoami
who	Prints information about users who are currently logged in	who	who
uptime	Tells how long the system has been running	uptime	uptime
date	Prints or sets the system date and time	date	date





Linux | Working with Terminal Utilities



Working with Terminal Utilities

➤ Basic Commands

Command	Description	Syntax	Usage
clear	Clears the terminal screen	clear	clear
history	Lists previously executed commands	history	history
man	System's manual pager	man <command>	man clear
whatis	Displays one-line manual page descriptions	whatis <command>	whatis clear
whereis	Locates the binary, source, and manual page files for a command	whereis <command>	whereis clear
which	Locates the path of binary, source file for a command	which <command>	which ls





Linux | Working with Directories



Working with Directories

➤ Basic Commands

Command	Description	Syntax	Usage
pwd	Prints the full filename of the current working directory	pwd	pwd
cd	Changes directory	cd <directory>	cd /tmp
ls	List information about the FILES of the current directory by default	ls	ls
mkdir	Makes/Creates directory	mkdir <directory>	mkdir thinknyx
rmdir	Removes empty directories	rmdir <directory>	rmdir thinknyx



Working with Directories

➤ Understanding Absolute and Relative Paths

Absolute and Relative Paths:

- ✓ Two ways to locate files and directories
- ✓ Simplify system navigation

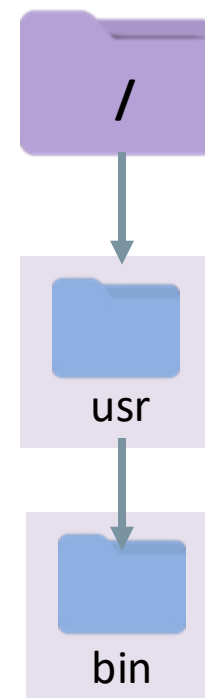


Working with Directories

➤ Understanding Absolute and Relative Paths

Absolute Paths:

- ✓ Starts from the root directory (/)
- ✓ Provides the full path to a file or folder
- ✓ Exact location, independent of the current directory
- ✓ Use cd to navigate



```
cd /usr/bin
```



Working with Directories

➤ Understanding Absolute and Relative Paths

Relative Paths:

- ✓ Starts from the current directory
- ✓ Doesn't begin with (/)

/home/user/documents

Current directory

/home/user

Parent directory

cd ..

.	Current directory
..	Parent directory
~	Home directory

Deeper directory

/home/user/documents/work

/home/user

cd ../../



Working with Directories

➤ Understanding Absolute and Relative Paths

When to Use Which?

- **Relative Path:** Quick navigation between nearby directories

/home/user/documents **to** /home/user/projects

```
cd ../projects
```

- **Absolute Path:** Reliable for deeper or fixed locations

/usr/bin

```
cd /usr/bin
```



Working with Directories

Character	Type
d	Directory
-	Regular file
l	Symbolic links
b	Block devices
c	Character devices
p	Named pipes(FIFOs)
s	Sockets





Linux | Working with Files



Working with Files

➤ Basic Commands

Command	Description	Syntax	Usage
touch	Creates an empty file;	touch <filename>	touch thinknyx.txt
file	Determines file type	file <filename>	file thinknyx.txt
cp	Copies files and directories	cp <src_filename> <dest_filename>	cp thinknyx.txt test.txt
mv	Moves or renames files	mv <src_filename> <dest_filename>	mv test.txt result.txt
rm	Removes files or directories	rm <filename>	rm result.txt





Linux | Working with File Contents



Working with File Contents

➤ Basic Commands

Command	Description	Syntax	Usage
cat	Concatenates files and prints on the standard output	cat <file1> <file2>	cat file1.txt file2.txt
tac	Concatenates files and prints the contents on the standard output in reverse order	tac <file1> <file2>	tac file1.txt file2.txt
head	Prints the first 10 lines of the file to standard output by default	head <filename>	head file1.txt
tail	Prints the last 10 lines of the file to standard output by default	tail <filename>	tail file1.txt
more	Prints file content page-by-page	more <filename>	more file1.txt
less	Prints file content page-by-page in reverse	less <filename>	less file1.txt



Summary



- ❖ **Foundational Shell Commands:** echo, who, clear, history
- ❖ **How to Work With:** Directories, files, and file contents





Thinknyx Technologies

Linux Course

Enhancing Strategy

Section - 5





Linux | More Essential Linux Commands



Essential Linux Commands



- Focus on filter, utility, archival, and compression commands
- Powerful tools to manage and process data efficiently
- Enhance productivity and streamline tasks



Linux | Filter Commands-Part1



Filter Commands Part-1

➤ Basic Commands

Command	Description	Syntax	Usage
grep	Searches for patterns in files using regular expressions	grep <pattern> <filename>	grep "hello" thinknyx.txt
egrep	Similar to grep, but supports extended regular expressions	egrep <pattern> <filename>	egrep "^[A-Z]" thinknyx.txt
wc	Counts the number of lines, words, and characters in a file	wc <filename>	wc thinknyx.txt
pipe	Passes output from one command as input to another	command1 command2	grep -i "developer" details.txt wc -l
comm	Compares two sorted files and outputs their differences and common lines	comm <file1> <file2>	comm file1.txt file2.txt
find	Searches for files and directories based on specified criteria	find [path] [expression]	find /etc -name "nsswitch.conf"



Filter Commands Part-1

➤ **grep: (global regular expression print)**

- ✓ search for patterns in text files and display the lines that contain those patterns

Syntax:

```
grep <pattern> <filename>
```





Linux | Filter Commands-Part2



Filter Commands-Part2

➤ Basic Commands

Command	Description	Syntax	Usage
sort	Sorts lines of text from input or a file in a specified order	sort <filename>	sort thinknyx.txt
uniq	Removes duplicate lines from sorted input	uniq <filename>	uniq thinknyx.txt
nl	Numbers the lines of a file or input	nl <filename>	nl thinknyx.txt
cut	Prints selected parts of lines from each file to standard output	cut [options] <filename>	cut -d ',' -f 1,3 file.csv
tr	Translates or replaces characters in input	tr set1 [set2]	cat employees.txt tr ',' '\t'
column	Formats input into a well-aligned table	column [options] <filename>	column -t employees.txt
pr	Converts text files for printing	pr [options] <filename>	pr -t -2 employees.txt
tee	Reads from standard input and writes to standard output and files	tee <filename>	pr -t -2 employees.txt tee formatted_employees.txt





Linux | Miscellaneous Utility Commands



Miscellaneous Utility

Commands

Basic Commands

Command	Description	Syntax	Usage
date	Used to display or set the system date and time	date	date
sleep	Pauses execution for a specified time	sleep <duration>	sleep 5
time	Measures the time taken by a command to execute	time <command>	time ls
timeout	Terminates a running command if it exceeds a specified time limit	timeout <duration> <command>	timeout 3 ls





Linux | Compressing and Archiving Commands



Compressing and Archiving

Commands

Compression	Archiving
Reduces file size to save space and speed up transfers	Combines multiple files into one without reducing size
Common Compression Tools: zip, gzip	Common Archiving Tool: tar , zip

- ✓ tar and gzip: Creates .tar.gz files for both archiving and compression



Compressing and Archiving Commands

zip	gzip
Common on Windows, supports cross-platform sharing	Standard on Unix, Linux, and macOS systems
Compresses both files and directories into a .zip file	Compresses individual files only

➤ tar and gzip

- ✓ Creates .tar.gz archives for directories and files
- ✓ Preserves file permissions, metadata, and directory structures
- ✓ Linux backups and managing large file sets efficiently



Compressing and Archiving

Commands

basic syntax is:

```
zip [options] archive_name.zip file1 file2 ...
```



Summary



- ❖ **Filter Commands:** grep, find, sort, uniq
- ❖ **Utility Commands:** date, sleep
- ❖ **Archival & Compression Commands:** zip, gzip, tar





Thinknyx Technologies

Linux Course

Enhancing Strategy

Section - 6





Linux | Text Editors and Advanced Commands





Text Editors and Advanced Commands

- Nano and Vi editors
- Sed and Awk for advanced text processing



Linux | Text Editors



Text Editors

- ✓ Text editors are crucial for editing files directly in the terminal
- ✓ Two popular text editors: Nano and Vi
 - **Nano:** Beginner-friendly and straightforward
 - **Vi:** Advanced features for power users



Text Editors

➤ Nano Editor

- ✓ Lightweight and easy-to-use text editor
- ✓ Ideal for quick edits and commonly used by beginners
- ✓ Open a file by using: **nano filename.txt**

Action	Shortcut
Save a file	CTRL + O
Exit Nano	CTRL + X
Search for text	CTRL + W
Cut a line	CTRL + K
Paste a line	CTRL + U



Text Editors

➤ Vi Editor

- ✓ Widely used by advanced Linux users for efficient text editing
- ✓ Vi stands for Visual Editor
- ✓ Vim (Vi Improved) is the enhanced version of vi, adding more features, flexibility, and functionality
- ✓ Ideal for editing, navigating, and manipulating large amounts of text

Mode	Description
Normal Mode Or Command mode	Default mode for navigation and commands
Insert Mode	For adding or editing text
Ex-Command Mode or Last line mode	For executing operations like saving, quitting & searching



Text Editors

/pattern	Searches forward for the pattern from cursor position
?pattern	Searches backward for pattern from cursor position
:%s/pattern/replacement/g	Replace all occurrences of 'pattern' with 'replacement' in the file
:%s/pattern/replacement/gc	Replace all occurrences of 'pattern' with confirmation before each change



Text Editors

➤ Advanced Vim Features

Action	Command
Delete a line	dd
Delete 3 lines	3dd
Delete character at cursor	x
Undo changes	u
Redo changes	CTRL + R

Action	Command
Copy	yy
Paste below cursor	p
Paste above cursor	P
Append to end of line	Shift + A



Text Editors

Command	Functionality	Timestamp Behavior
:wq	Saves changes and exits the editor	Always updates the timestamp to current time
:x	Saves changes and exits, but only updates the timestamp if changes were made	Updates timestamp only if changes were made





Linux | Advanced Commands



Advanced Commands

Command	Description	Syntax	Usage
sed	Stream editor for filtering and transforming text	sed [options] 'script' filename	sed 's/World/Linux/' thinknyx.txt
awk	Pattern scanning and processing language	awk [options] 'pattern {action}' filename	awk '/ERROR/ {print}' samplelog.txt



Advanced Commands

```
GAWK(1)                                Utility Commands                                GAWK(1)

NAME
    gawk - pattern scanning and processing language

SYNOPSIS
    gawk [ POSIX or GNU style options ] -f program-file [ -- ] file ...
    gawk [ POSIX or GNU style options ] [ -- ] program-text file ...

DESCRIPTION
    Gawk is the GNU Project's implementation of the AWK programming language. It conforms to the
    definition of the language in the POSIX 1003.1 standard. This version in turn is based on the
    description in The AWK Programming Language, by Aho, Kernighan, and Weinberger. Gawk provides
    the additional features found in the current version of Brian Kernighan's awk and numerous
    GNU-specific extensions.

    The command line consists of options to gawk itself, the AWK program text (if not supplied via
    the -f or --include options), and values to be made available in the ARGV and ARGV pre-defined
    AWK variables.
```



Advanced Commands

```
sed [options] 'script' filename
```



Summary



- ❖ **Nano and Vi editors:** Text editing
- ❖ **Sed and Awk:** Advanced text processing





Thinknyx Technologies

Linux Course

Enhancing Strategy

Section - 7





Linux | User & Group Management



User & Group Management



- Introduction to users
- Managing users and groups



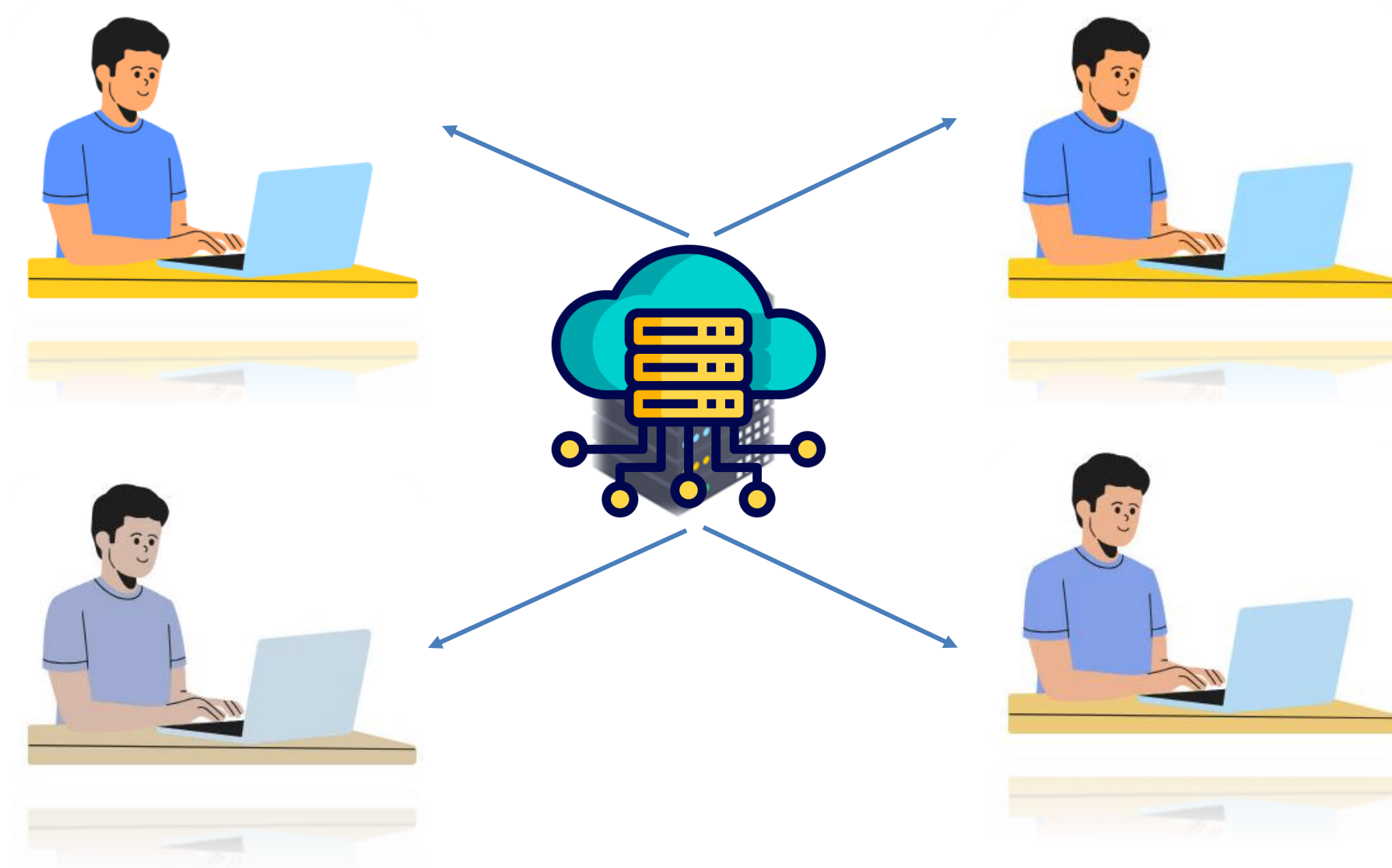


Linux | Introduction to Users

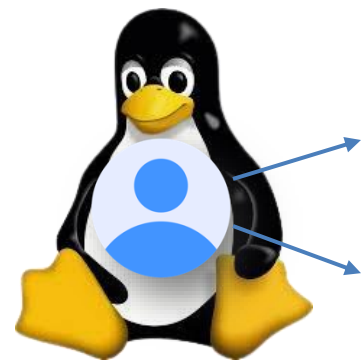


Introduction to Users

- ✓ Users are the backbone of any Linux system



Introduction to Users



Unique user id (UID)

At least one group

➤ Two primary types of users in Linux

- ✓ Root User
- ✓ Regular Users



New user

- Linux automatically places them in a group with the same name as the username

Primary group



Introduction to Users

➤ Key Commands Overview

- ✓ **id** : Displays the current user's UID, GID, and groups
- ✓ **su** : Switches the current user to another user, commonly the root user
- ✓ **sudo** : Allows a user to run commands with root privileges temporarily





Linux | User Management



User Management

File	Description
/etc/passwd	Contains user account details like username, UID, and home directory
/etc/shadow	Stores hashed passwords and password-related settings, accessible only by root
/etc/skel	Contains default files and directories copied to a new user's home directory
/etc/sudoers	This file is a sensitive file, as it determines who can gain root access



User Management

UID Range	Description
0	Superuser (root) account
1-200	System accounts for specific processes
201-999	Unprivileged system processes
1000+	Regular users



User Management

Command	Description	Syntax	Usage
adduser	Creates a new user	sudo adduser <username>	sudo adduser alex
passwd	Change user passwords updating password aging policies & unlocking accounts	sudo passwd [options] [username]	sudo passwd alex
usermod	Modifies an existing user	sudo usermod <options> <username>	sudo usermod -d /home/directory alex
deluser	Deletes a user	sudo deluser <username>	sudo deluser alex





Linux | Group Management



Group

Management

➤ Core concepts involved in group management

- ✓ **addgroup** : Creates a new group
- ✓ **groupmod** : Modifies existing groups
- ✓ **/etc/group** : Lists all groups on the system





Demonstration | User & Group Management

Group

Management

➤ Core concepts involved in group management



prod_user



dev_user



Group : programmer



Group

Management

➤ Core concepts involved in group management

- ✓ Create two users : prod_user and dev_user
- ✓ Create a group : programmer
- ✓ Add prod_user to the programmer group





Demonstration | su VS su - Commands

su VS su -

Command	When to Use
su <username>	Quick access to another user Retains the current environment No need for a full user session
su - <username>	Loads the user's full environment Acts like a fresh login Preferred in production for a clean session



su VS su -

➤ Quick Rule:

- ✓ **Use su - <username>** for a full login shell
- ✓ **Use su <username>** for quick user switching



Summary



❖ User Concepts:

Commands: id, su, sudo, and root for user management

❖ User Management:

Key Files:

/etc/passwd: Stores user details

/etc/shadow: Manages encrypted passwords

/etc/skel: Default home directory templates

Summary



❖ Commands:

adduser: Create users

usermod: Modify user details

passwd: Change user passwords

deluser: Delete users

❖ Group Management:

Key File:

`/etc/group`

Summary



❖ Commands:

addgroup: Create new groups

groupmod: Modify existing groups

❖ Key Focus:

Managing permissions

Ensuring system security





Thinknyx Technologies

Linux Course

Enhancing Strategy

Section - 8





Linux | File Permissions



File Permissions



- File Ownership covering both Octal Mode and Symbolic Mode
- Managing Special Permissions



Linux | File Ownership



File Ownership

```
ubuntu@ip-172-31-37-18:/tmp$ ls -l
total 24
drwx----- 2 root root 4096 Nov 19 08:36 snap-private-tmp
drwx----- 3 root root 4096 Nov 19 08:36 systemd-private-86398170d5c448128a49d3f65fbddcfe-ModemManager.service-QvKYeo
drwx----- 3 root root 4096 Nov 19 08:36 systemd-private-86398170d5c448128a49d3f65fbddcfe-chrony.service-QZNxxE
drwx----- 3 root root 4096 Nov 19 08:36 systemd-private-86398170d5c448128a49d3f65fbddcfe-polkit.service-UAnXSe
drwx----- 3 root root 4096 Nov 19 08:36 systemd-private-86398170d5c448128a49d3f65fbddcfe-systemd-logind.service-jjDUtR
drwx----- 3 root root 4096 Nov 19 08:36 systemd-private-86398170d5c448128a49d3f65fbddcfe-systemd-resolved.service-TYd6lT
ubuntu@ip-172-31-37-18:/tmp$
```

File type

Link count

Group

Permissions

Owner

File size

Last modification time

File name



File Ownership

```
ubuntu@ip-172-31-37-18:/tmp$ ls -l
total 24
drwx----- 2 root root 4096 Nov 19 08:36 snap-private-tmp
drwx----- 3 root root 4096 Nov 19 08:36 systemd-private-86398170d5c448128a49d3f65fbddcfe-ModemManager.service-QvKYeo
drwx----- 3 root root 4096 Nov 19 08:36 systemd-private-86398170d5c448128a49d3f65fbddcfe-chrony.service-QZNxxE
drwx----- 3 root root 4096 Nov 19 08:36 systemd-private-86398170d5c448128a49d3f65fbddcfe-polkit.service-UAnXSe
drwx----- 3 root root 4096 Nov 19 08:36 systemd-private-86398170d5c448128a49d3f65fbddcfe-systemd-logind.service-jjDUtR
drwx----- 3 root root 4096 Nov 19 08:36 systemd-private-86398170d5c448128a49d3f65fbddcfe-systemd-resolved.service-TYd6lT
ubuntu@ip-172-31-37-18:/tmp$
```

File size

Last modification time

File name



File Ownership

```
ubuntu@thinknyxlinux:/tmp$ ls -lh
total 24K
drwx----- 2 root root 4.0K Nov 22 08:44 snap-private-tmp
drwx----- 3 root root 4.0K Nov 22 08:45 systemd-private-311f0da5651842b38945cbabfffd9098-ModemManager.service-qXDUSo
drwx----- 3 root root 4.0K Nov 22 08:45 systemd-private-311f0da5651842b38945cbabfffd9098-chrony.service-9qEvUa
drwx----- 3 root root 4.0K Nov 22 08:45 systemd-private-311f0da5651842b38945cbabfffd9098-polkit.service-TZQXIV
drwx----- 3 root root 4.0K Nov 22 08:45 systemd-private-311f0da5651842b38945cbabfffd9098-systemd-logind.service-Ep9JBB
drwx----- 3 root root 4.0K Nov 22 08:44 systemd-private-311f0da5651842b38945cbabfffd9098-systemd-resolved.service-E4GrOu
```

File size



File Ownership

```
ubuntu@thinknyxlinux:/tmp$ ls -lh
total 24K
drwx----- 2 root root 4.0K Nov 22 08:44 snap-private-tmp
drwx----- 3 root root 4.0K Nov 22 08:45 systemd-private-311f0da5651842b38945cbabfffd9098-ModemManager.service-qXDUSo
drwx----- 3 root root 4.0K Nov 22 08:45 systemd-private-311f0da5651842b38945cbabfffd9098-chrony.service-9qEvUa
drwx----- 3 root root 4.0K Nov 22 08:45 systemd-private-311f0da5651842b38945cbabfffd9098-polkit.service-TZQXIV
drwx----- 3 root root 4.0K Nov 22 08:45 systemd-private-311f0da5651842b38945cbabfffd9098-systemd-logind.service-Ep9JBB
drwx----- 3 root root 4.0K Nov 22 08:44 systemd-private-311f0da5651842b38945cbabfffd9098-systemd-resolved.service-E4GrOu
```

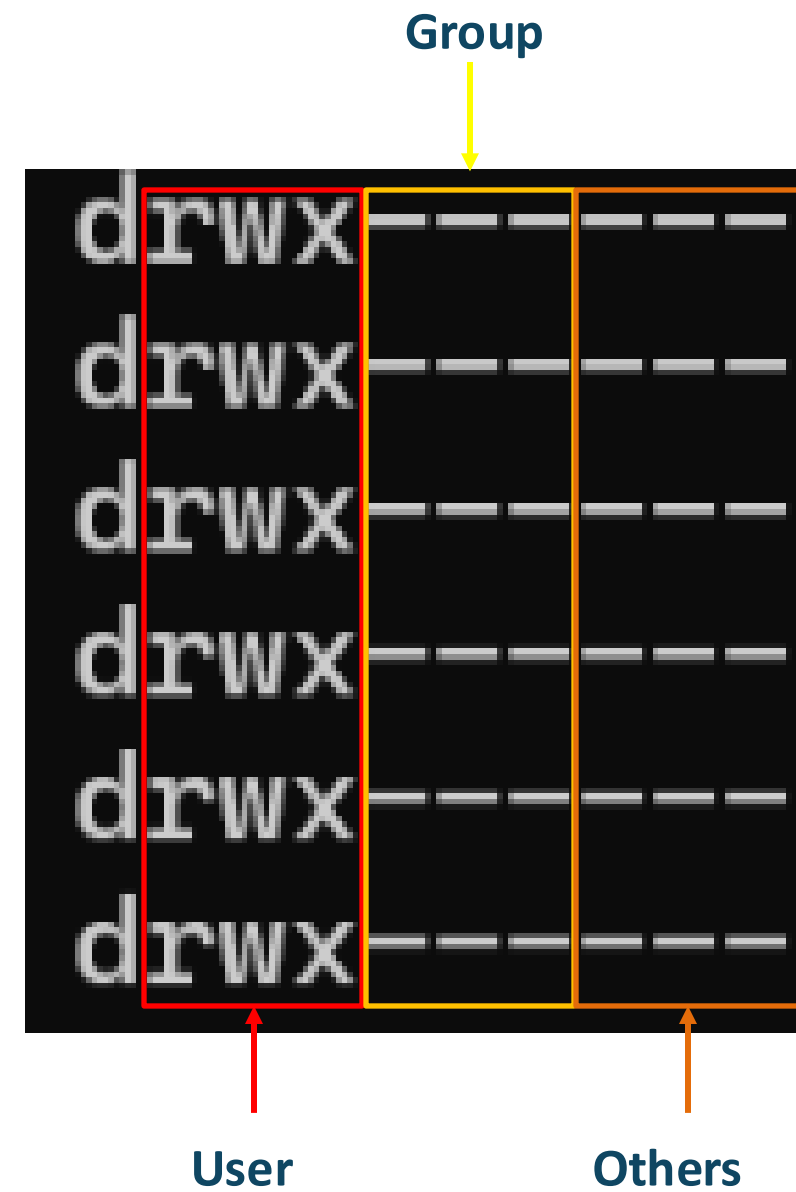
Permissions



File Ownership

➤ File permissions

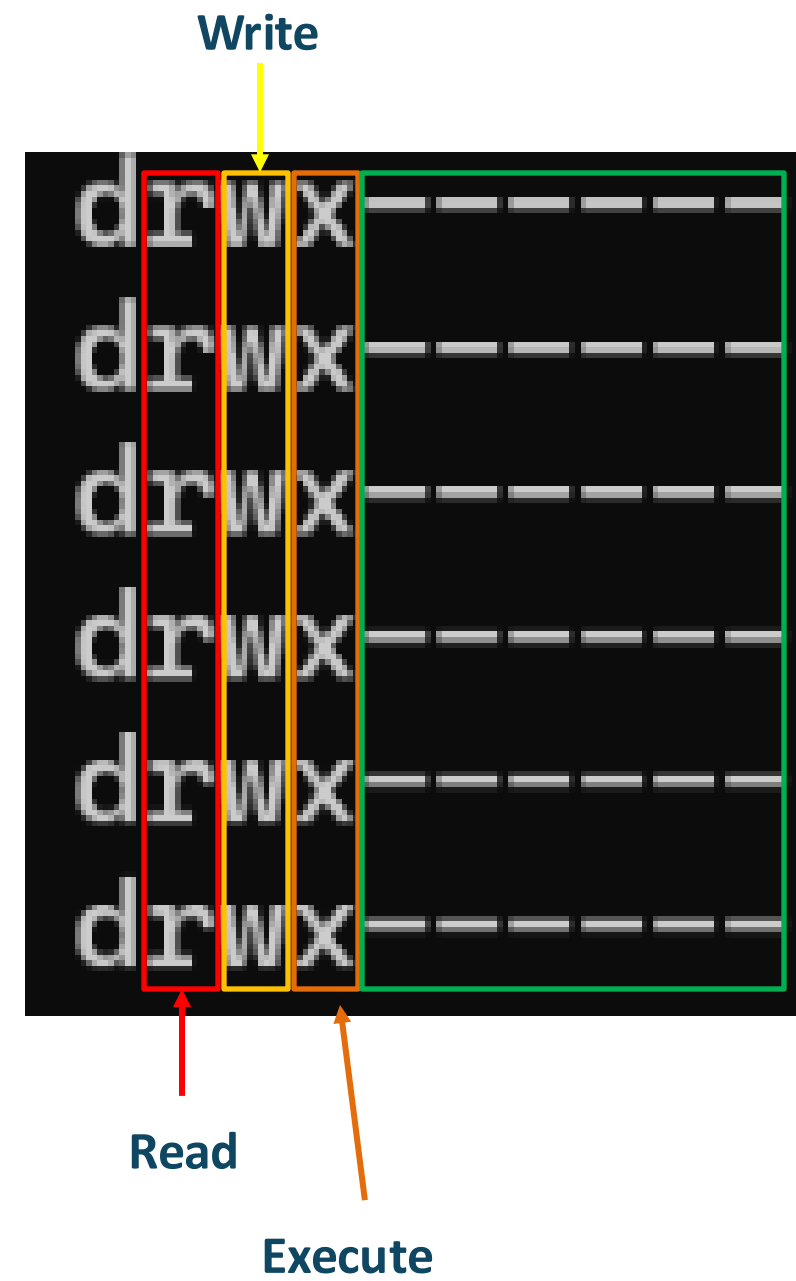
- ✓ **User** : Owner of the file
- ✓ **Group** : Set of users
- ✓ **Others** : Everyone else



File Ownership

➤ File permissions

- ✓ **User** : Owner of the file
- ✓ **Group** : Set of users
- ✓ **Others** : Everyone else



File Ownership

➤ File permission management commands

Command	Description	Syntax	Usage
chmod	Changes file mode bits	chmod [options] <mode> <file>	chmod u+rx thinknyx.txt
chown	Changes file owner and group	chown [options] <owner[:group]> <file>	chown alex:developers filename
chgrp	Changes group ownership	chgrp [options] <group> <file>	chgrp developers thinknyx.txt



File Ownership

➤ Different ways to use chmod

❑ Symbolic and Octal or Numeric Method

Symbolic Method

Permission groups

u	user
g	group
o	others
a	all



File Ownership

➤ **Different ways to use chmod**

- ❑ **Symbolic and Octal or Numeric Method**

Symbolic Method

Permission types

r	read permission
w	write permission
x	execute permission



File Ownership

➤ **Different ways to use chmod**

❑ **Symbolic and Octal or Numeric Method**

Symbolic Method

Operators

+	add permission
-	remove permission
=	set permission explicitly, replacing others



File Ownership

➤ Different ways to use chmod

```
chmod u+x thinknyx.txt
```

```
chmod g-r thinknyx.txt
```

```
chmod u=rw,g=r,o=r thinknyx.txt
```



File Ownership

➤ **Different ways to use chmod**

- ❑ **Symbolic and Octal or Numeric Method**

Octal Method

Permission

4	read permission (r)
2	write permission (w)
1	execute permission (x)



File Ownership

➤ Different ways to use chmod

❑ Symbolic and Octal or Numeric Method

Octal Method

Octal Representation

0	000	-	-	-	No permissions
1	001	-	-	x	Only Execute
2	010	-	w	-	Only Write
3	011	-	w	x	Write and Execute
4	100	r	-	-	Only Read
5	101	r	-	x	Read and Execute
6	110	r	w	-	Read and Write
7	111	r	w	x	Read, Write and Execute



File Ownership

➤ Numeric value

- ✓ **7 means Full access:** Read (4) + Write (2) + Execute (1)
- ✓ **6 means Read and Write:** Read (4) + Write (2)
- ✓ **5 means Read and Execute:** Read (4) + Execute (1)

```
chmod 765 thinknyx.txt
```

Binary Conversion

$$\begin{array}{ccc} 1 & 1 & 1 \\ \hline 4 & 2 & 1 \end{array}$$

$$4 + 2 + 1 = 7$$

(u) user

$$\begin{array}{ccc} 1 & 1 & 1 \\ \hline r & w & x \end{array}$$

$$7$$

(g) group

$$\begin{array}{ccc} 1 & 1 & 0 \\ \hline r & w & x \end{array}$$

$$6$$

(g) group

$$\begin{array}{ccc} 1 & 0 & 1 \\ \hline r & w & x \end{array}$$

$$5$$



File Ownership

➤ **Effects of Permissions on Files and Directories**

Permissions	File Permissions	Directory Permissions
Read (r)	Allows the user to viewing the contents of the file	Allows listing the directory's contents
Write (w)	Allows the user to modifying the contents of the file	Allows creating or deleting files within the directory
Execute (x)	Allows the user to executing the file as a program or command	Allows traversing the directory and accessing files and subdirectories inside the directory



File Ownership

➤ **Special Cases of Directory Permissions**

- ✓ **Read & Execute:** User can list and access file contents in read-only mode but cannot modify files
- ✓ **Read-Only:** User can list file names but cannot view permissions or timestamps
- ✓ **Execute-Only:** User can cd into the directory but cannot list file names; file access requires exact names



File Ownership

➤ Directory Ownership and File Deletion

- ✓ **Write Permissions:** Directory owners or users with write access can delete files inside, regardless of file ownership or permissions
- ✓ **Sticky Bit:** Restricts file deletion to file owners, even if others have write access to the directory





Demonstration | File Ownership

File Ownership



thinknyx

- ✓ **Create Users and Groups**
- ✓ **Set Ownership**
- ✓ **Configure Permissions**
- ✓ **Test Access**



Alex

Owner, Developers
group



Bob

Developers
group



Charlie

Others



File Ownership

➤ Effects of Permissions on Files and Directories

- ✓ Alex -> Full access
- ✓ Developers Group -> Full access
- ✓ Others -> No Access





Linux | Manage Special Permissions



Manage Special Permissions

u+s

Setuid



rwX

g+s

Setgid



rwX

o+t

Sticky



rwX



Manage Special Permissions

Special Permission	Effect On Files	Effect On Directories	Use Cases
Setuid (u+s)	The file runs with the permissions of the file's owner, rather than the user executing it	No impact	Used for programs requiring elevated privileges (e.g., passwd)
Setgid (g+s)	The file runs with the permissions of the file's group	New files created in the directory inherit the group ownership of the directory	Ensures group consistency or shares files among group members
Sticky Bit (o+t)	No impact	Users with write access can only delete or rename files they own; they cannot modify files owned by others	Prevents users from deleting others' files in shared directories like /tmp



Manage Special Permissions



thinknyx



Alex

Owner, Developers

group



Bob

Developers

group



Charlie

Others



Manage Special

Permissions

- ✓ ACLs provide fine-grained control over file and directory permissions
- ✓ Allow management of access for specific users or groups, even within the owning group
- ✓ **Example:** You can revoke access to a particular user from the owning group
- ✓ ACLs support default rules for new files and directories to ensure consistent permissions
- ✓ ACLs can be set during directory creation for immediate application
- ✓ Grants user charlie read (r) and write (w) access for new files/subdirectories
- ✓ ACLs offer flexibility beyond traditional rwx permissions
- ✓ They are powerful in multi-user environments



Manage Special Permissions

➤ Default initial permissions

File Type – whether it's a regular file or a directory

umask – a bitmask that modifies these default permissions

Regular Files	0666	Read and Write for everyone
Directories	0777	Read, Write, and Execute for everyone

special permissions

```
alex@thinknyxlinux:/home/shared$ umask
0002
```

- The umask is an octal bitmask that subtracts permissions from these defaults when creating files or directories



Manage Special Permissions

➤ Regular file

	Symbolic	Numeric octal	Numeric binary
Initial file permissions	rw-rw-rw-	0666	000 110 110 110
umask	-----w-	0002	000 000 000 010
Resulting file permissions	rw-rw-r--	0664	000 110 110 100



Manage Special Permissions

	Symbolic	Numeric octal	Numeric binary
Initial file permissions	rw-rw-rw-	0777	000 111 111 111
umask	-----w-	0002	000 000 000 010
Resulting file permissions	rw-rw-r-x	0775	000 111 111 101



Manage Special Permissions

- ✓ The regular file is created with 0666 permissions (read and write for all users)
- ✓ The directory is created with 0777 permissions (read, write, and execute for all users)



Manage Special Permissions

- ✓ **Regular files:** Only read and write for the owner (0600)
- ✓ **Directories:** Only read, write, and execute for the owner (0700)



Summary



- ❖ **File Ownership:** user, group, and others
- ❖ **chmod:** Explored octal and symbolic modes
- ❖ **Special Permissions:** SUID, SGID, and sticky bit
- ❖ **ACLs & Umask:** manage file access and default permissions





Thinknyx Technologies

Linux Course

Enhancing Strategy

Section - 9





Linux | Advanced Topics on Files



Advanced Topics on Files



- Hard links and soft links
- User/Shell startup files
- Environment variables



Linux | Hard Links and Soft Links



Hard Links and Soft Links

➤ inodes

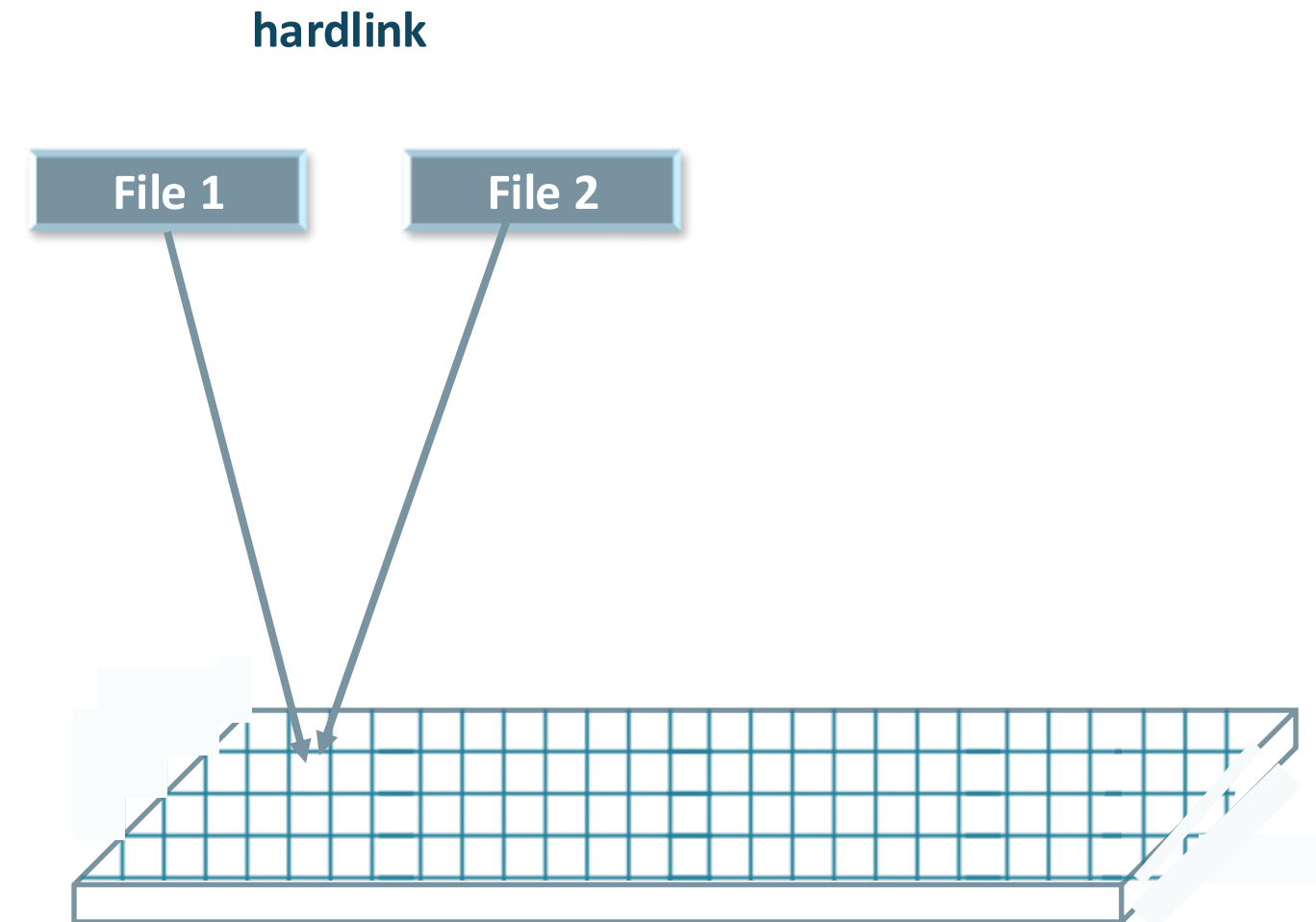
- ✓ **Records file metadata:** permissions, ownership, timestamps, and data pointers
- ✓ Data pointers locate the file's data on disk
- ✓ Each file/directory has a unique inode number
- ✓ File names are stored in directory entries, not inodes



Hard Links and Soft Links

➤ hardlink

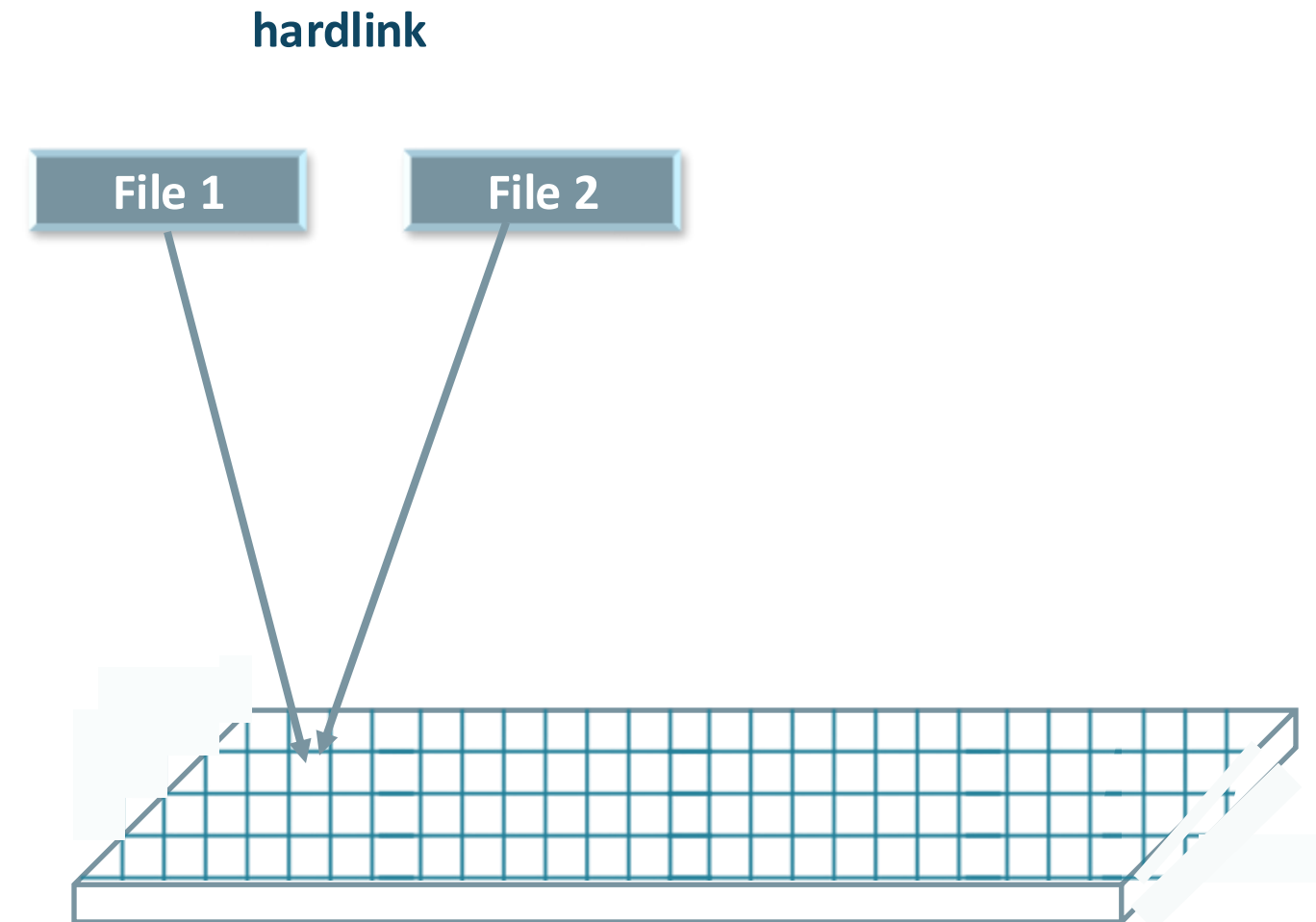
- ✓ Comparable to database entries pointing to the same data row
- ✓ Changes to one entry maintain data consistency
- ✓ Directly reference the inode of a file
- ✓ Enable shared access without duplicating data, saving disk space



Hard Links and Soft Links

➤ In hardlink

- ✓ Original file and hard link share the same inode number
- ✓ Changes to one reflect in the other
- ✓ Deleting one does not affect other hard links
- ✓ Cannot span file systems or link to directories

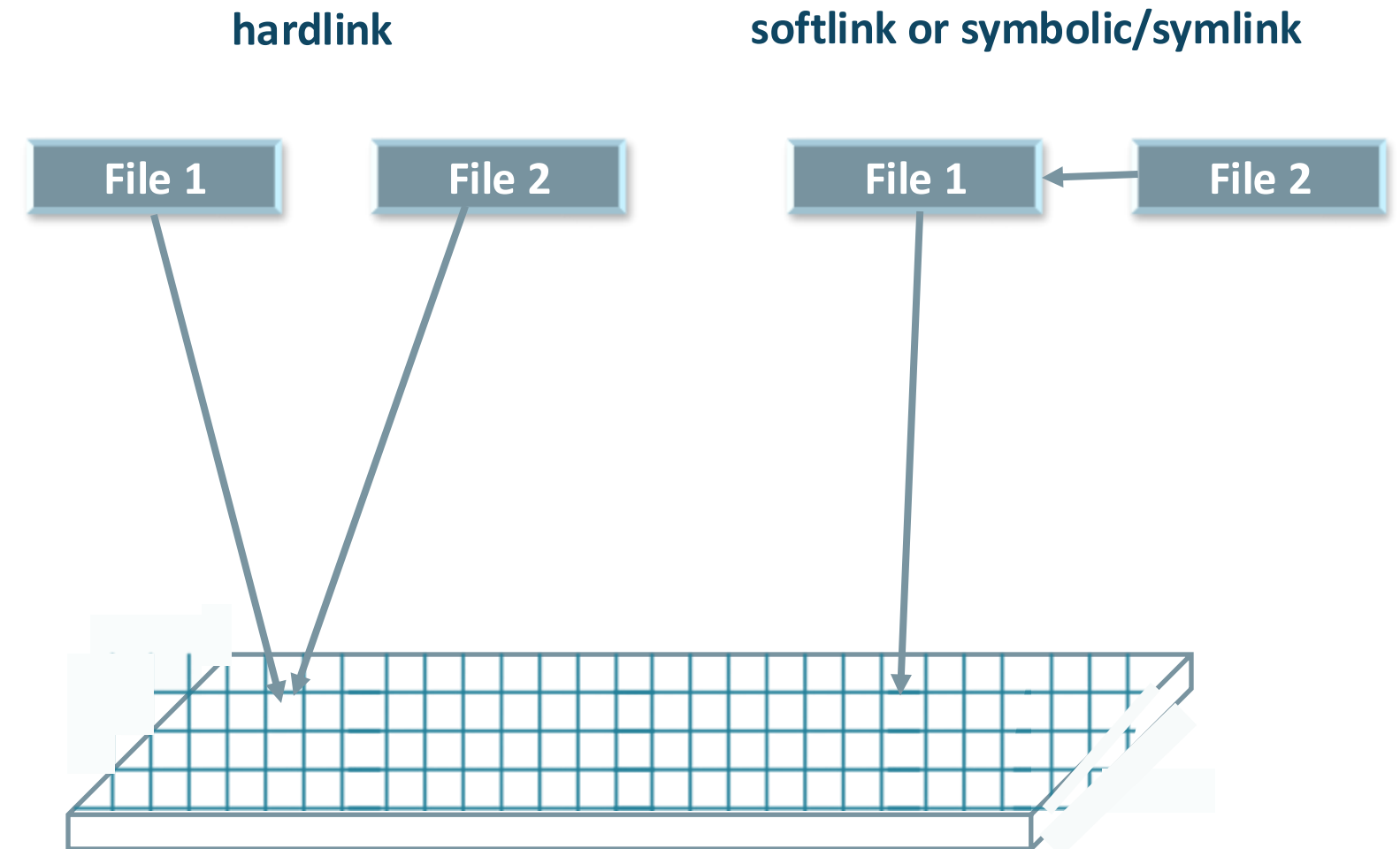


Hard Links and Soft Links

➤ softlink or symbolic/symlink



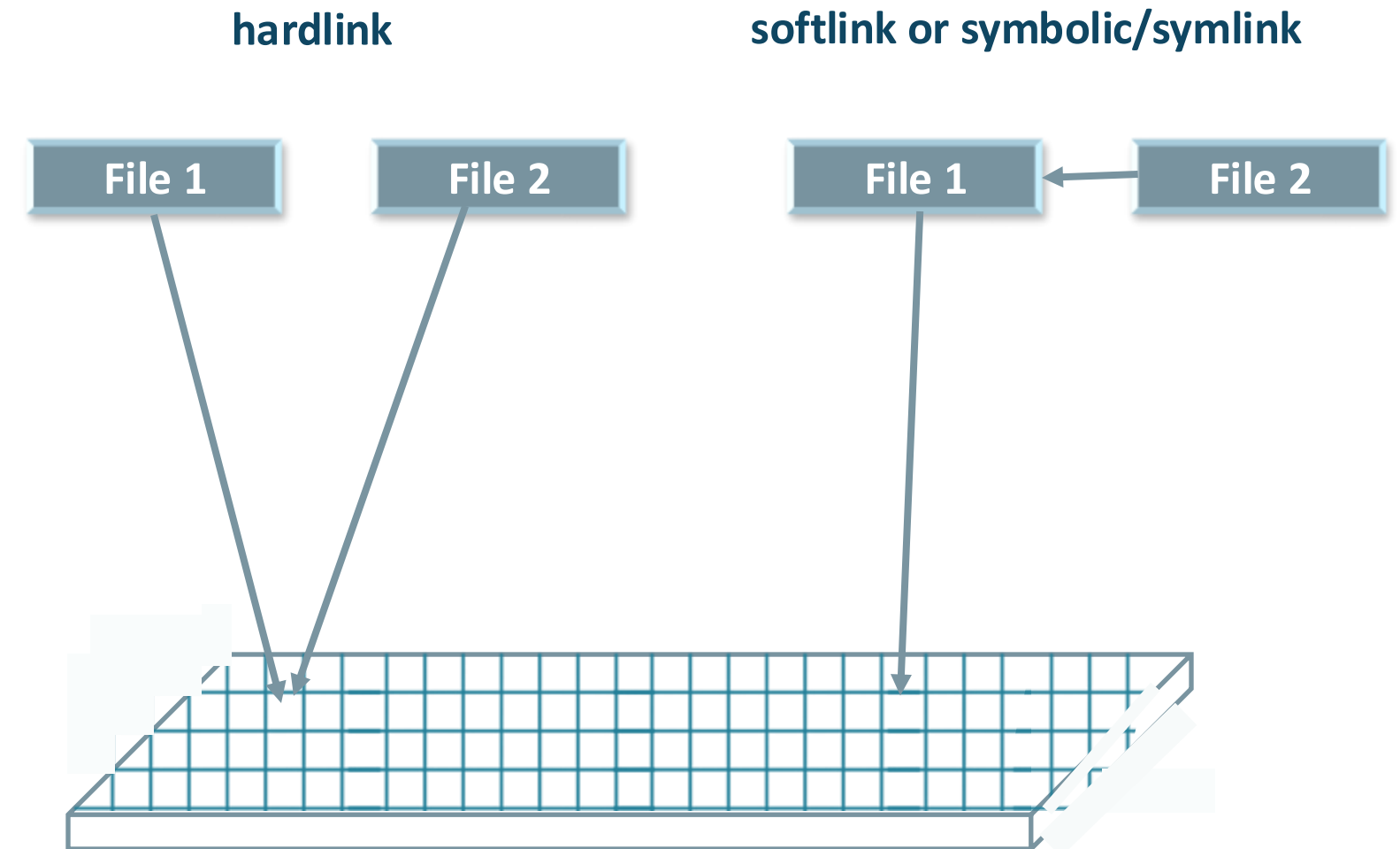
- ✓ Does not store actual data, only its location
- ✓ Commonly used in website hosting to connect directories (e.g., /var/www/html)
- ✓ Enables flexible redirection without additional storage



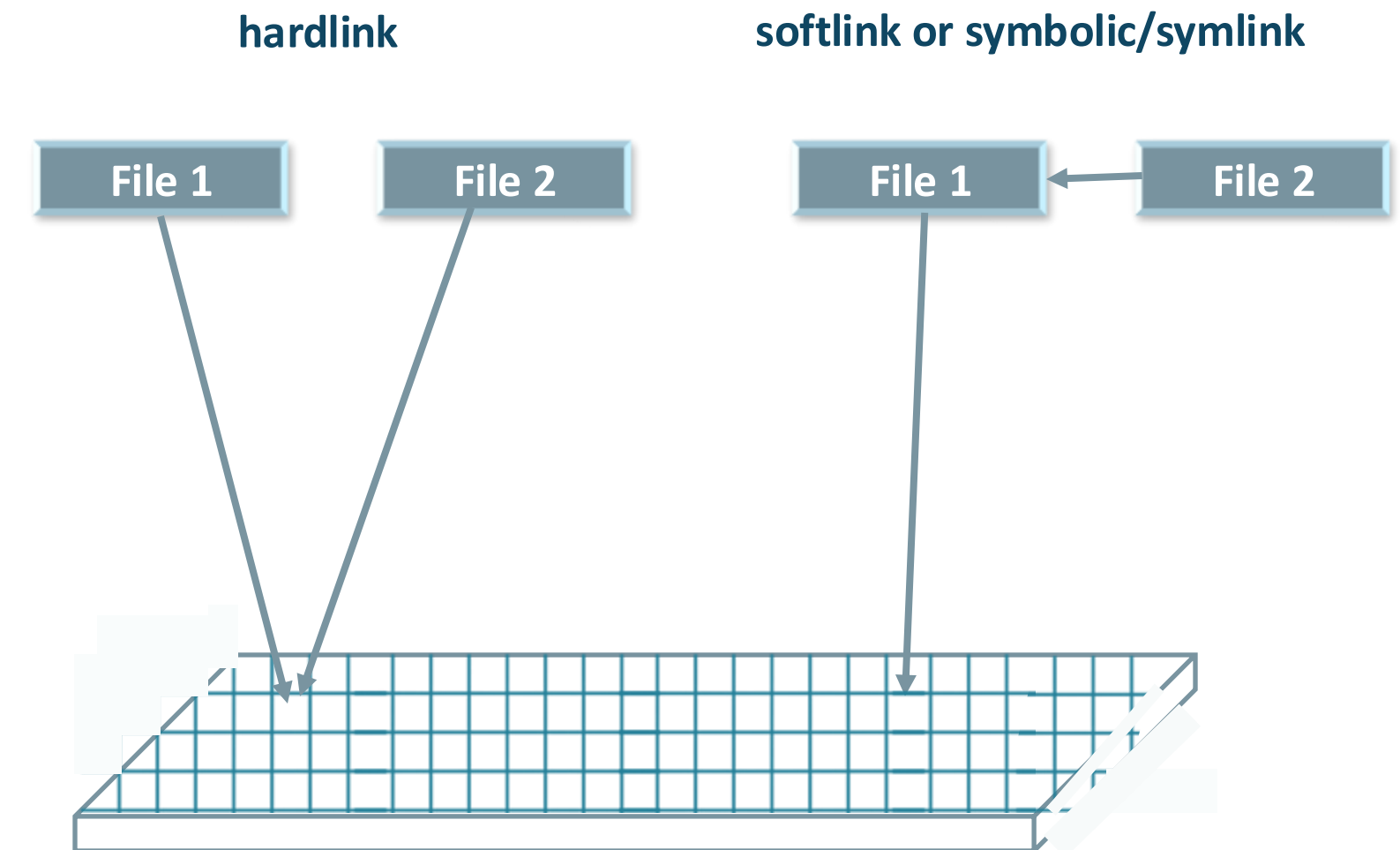
Hard Links and Soft Links

➤ In softlink

- ✓ Have a unique inode number
- ✓ Can span file systems and reference directories
- ✓ Become broken if the original file is deleted
- ✓ Serve as a reference, not a duplicate



Hard Links and Soft Links





Linux | User/Shell Startup Files



User/Shell Startup Files

.bashrc file

/etc/skel directory



User/Shell Startup Files

- ✓ Define aliases, variables, and functions for ease of use
- ✓ Automatically executed by the shell during startup
- ✓ Serve as setup instructions for the shell environment



User/Shell Startup Files

➤ Categories of Settings in Linux

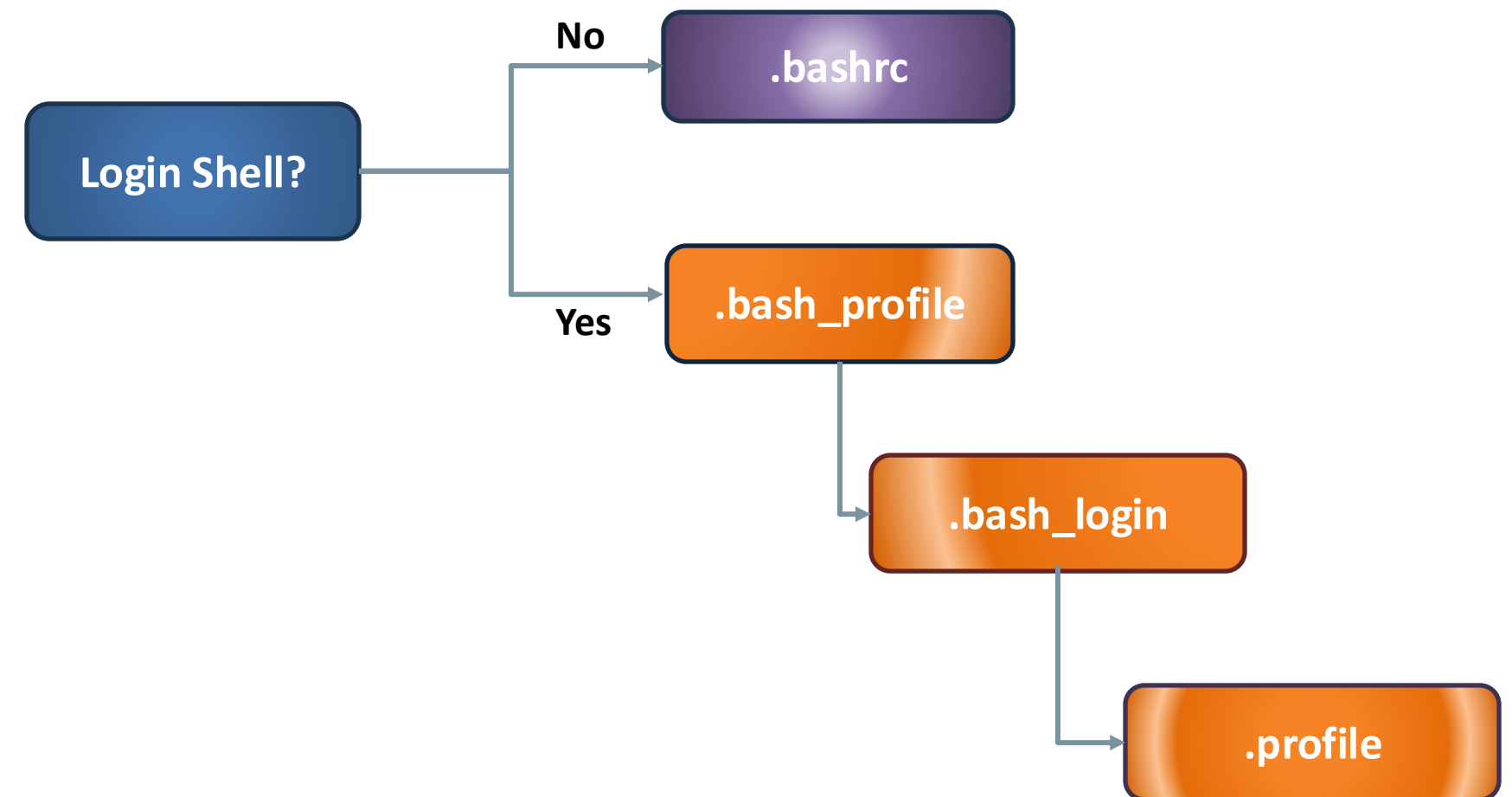
Category	Description	Location
Global Settings	Apply to all users on the system	/etc directory (e.g., /etc/profile)
User-Specific	Customize or override global settings for an individual user	User's home directory (e.g., ~/.bashrc, ~/.profile)



User/Shell Startup Files

➤ Shell Startup and Logout Behavior

- ✓ **Login Shell:** Reads /etc/profile and one of .bash_profile, .bash_login, or .profile (in order)
- ✓ **Non-Login Shell:** Reads only .bashrc
- ✓ **User Preference:** .bashrc is often customized as it runs for every new terminal session
- ✓ **Modern Simplification:** Many systems omit .bash_profile and .bash_login, relying on .bashrc
- ✓ **Logout Shell:** Executes .bash_logout for session cleanup



User/Shell Startup Files

➤ Environment Variables

- ✓ Set up by User Startup Files to configure the shell
- ✓ Store system and user-specific information
- ✓ Defined in .bashrc and .profile for persistence
- ✓ Are strings accessed by the shell and applications
- ✓ They store system preferences, user configurations, and influence program behavior



User/Shell Startup Files

➤ Defining Environment Variables

- ✓ Environment variables can be defined interactively in a shell session
- ✓ They are often defined in shell or User Startup Files for persistence across sessions
- ✓ Global variables are defined in `/etc/profile` or `/etc/environment`
- ✓ User-specific variables are defined in `.bashrc`, `.bash_profile`, or `.profile`



User/Shell Startup Files

command	Description
set	Displays all shell variables, including environment and shell-specific ones
env	Lists only environment variables
export	Makes a variable available to child processes of the shell



User/Shell Startup Files

HOME -> user's home directory

```
alice@thinknyxlinux:~$ cd /tmp
alice@thinknyxlinux:/tmp$ cd $HOME
alice@thinknyxlinux:~$ pwd
/home/alice
```

PATH -> list of directories where the system looks for executable programs

```
alice@thinknyxlinux:~$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
alice@thinknyxlinux:~$
alice@thinknyxlinux:~$ export PATH=$HOME/bin:$PATH
alice@thinknyxlinux:~$
alice@thinknyxlinux:~$ echo $PATH
/home/alice/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
alice@thinknyxlinux:~$
```



User/Shell Startup Files

SHELL -> default command shell for the user

```
alice@thinknyxlinux:~$ echo $SHELL  
/bin/bash
```

PS1 -> controls the appearance of your command prompt

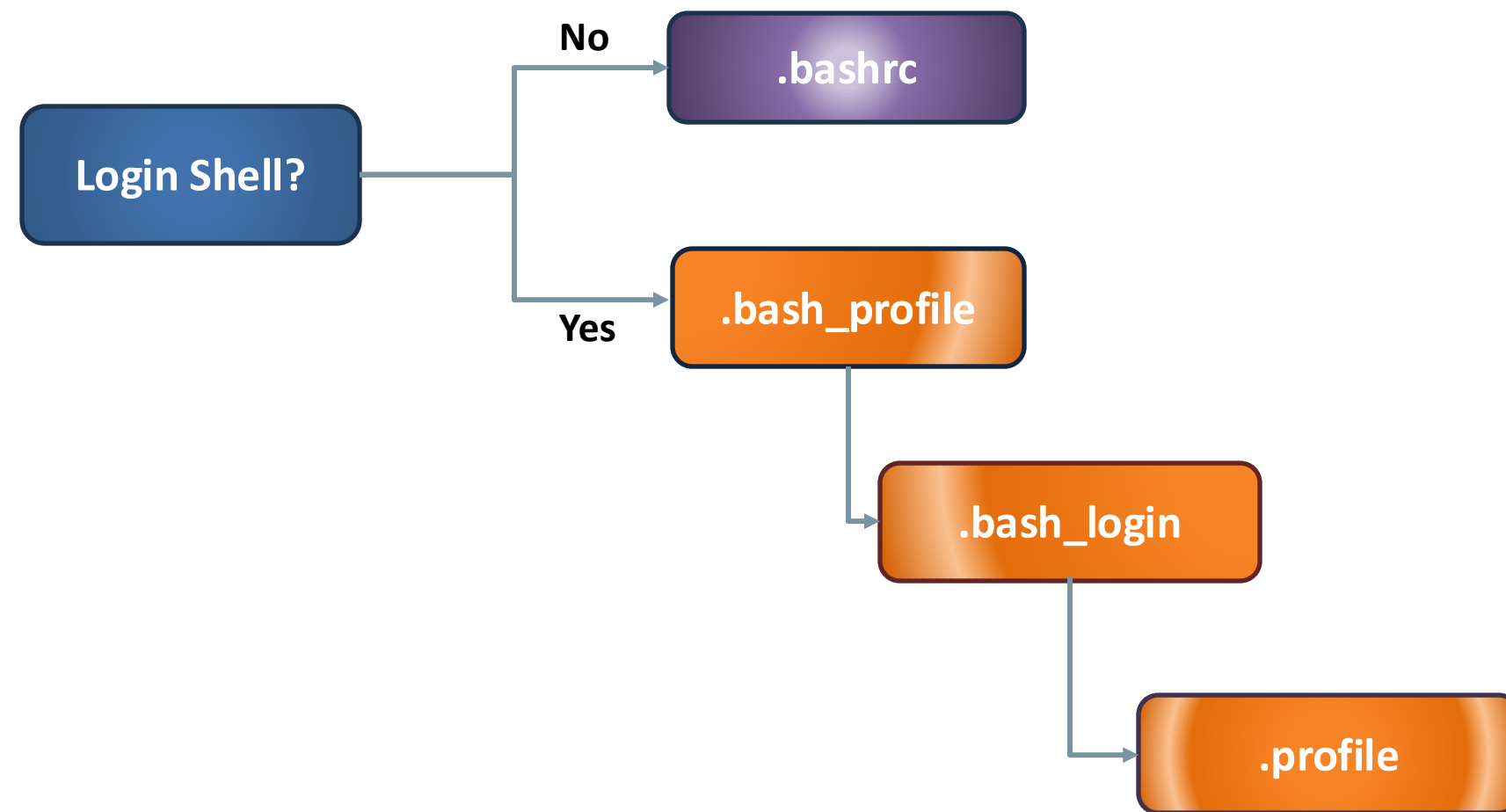
customize it to display useful information like the username, hostname, current directory

```
PS1="\d \h $ "
```

```
export PS1="\[\e[1;32m\]\d \[\e[1;34m\]\h \[\e[0m\]\$ "
```



User/Shell Startup Files



Summary



- ❖ **Hard and soft links:** Differences and uses
- ❖ **Startup files:** Configuring the environment during login
- ❖ **Environment variables:** Managing system settings and configurations





Thinknyx Technologies

Linux Course

Enhancing Strategy

Section - 10





Linux | Package Management



Package Management



- What is a package?
- Repositories and Dependencies
- Basic Package Management Tasks in Debian & Red hat-based distributions
- Downloading Software Outside Repositories
- Creating Local repository in RHEL9



Linux | Introduction to Packages



Introduction to Packages



Introduction to Packages

➤ What is Package



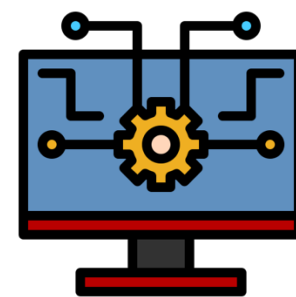
Install Software

Includes:

- ✓ Binaries
- ✓ Libraries
- ✓ Configuration files
- ✓ Documentation



Install



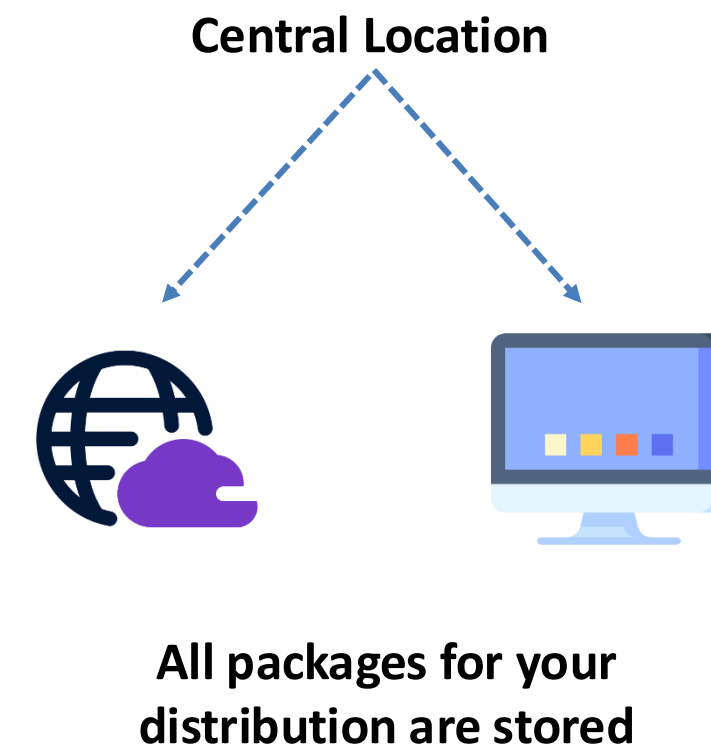
Manage Software



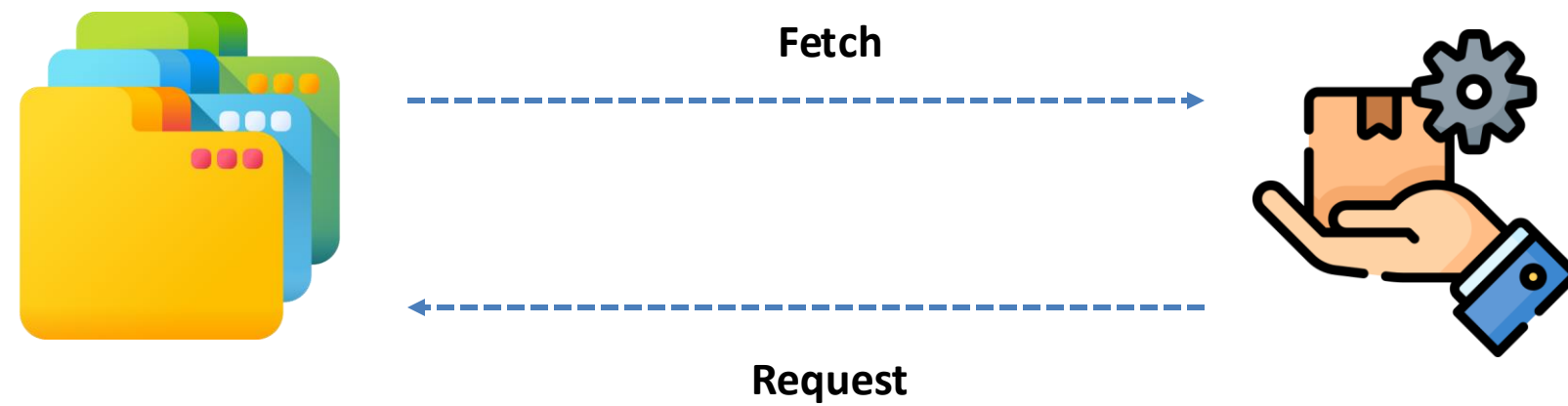
Update



Repositories



Repositories



- ✓ Online stores for software
- ✓ Contain precompiled software packages
- ✓ Ready to be downloaded
- ✓ Installed with a command



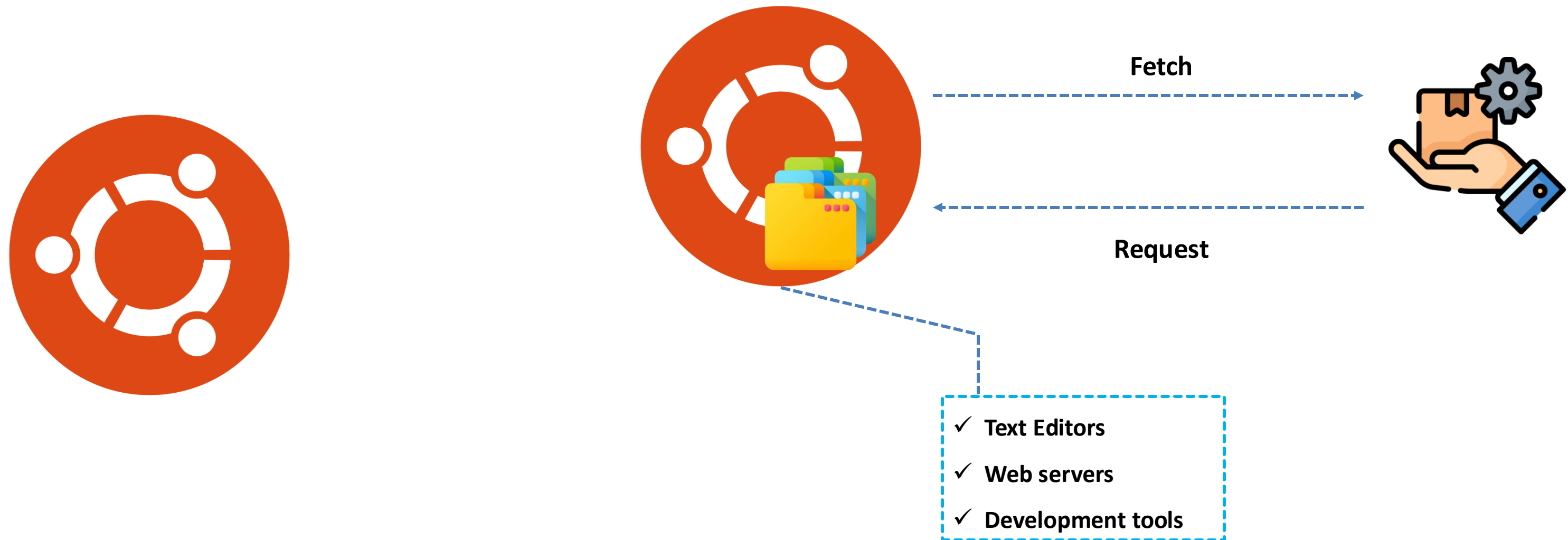
Repositories



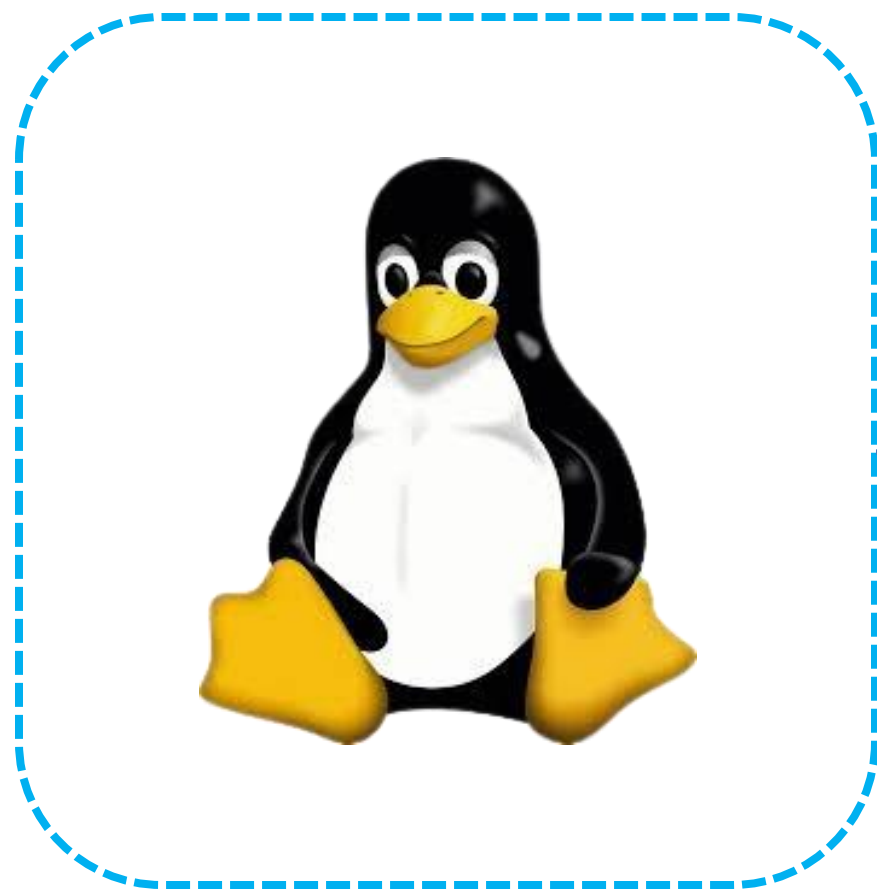
Meets different criteria like:

- ✓ **Main Repositories**
- ✓ **Security Updates**
- ✓ **Thirt-Party software Repositories**

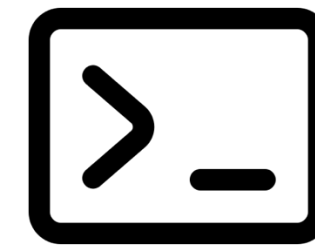
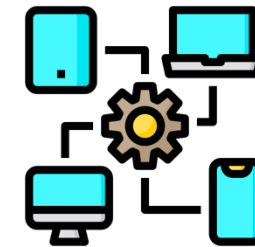
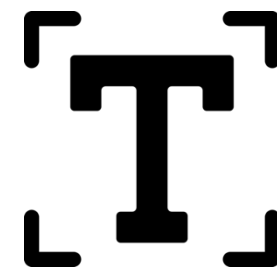
Managing Packages



Managing Packages



Everything is a file



- ✓ Simplifies system management
- ✓ Allows uniformity in how components are accessed and manipulated



Common Methods of software installation

- ✓ Precompiled binaries
- ✓ Ready-to-run executables
- ✓ Offer quick installation
- ✓ Ideal for users who don't need to modify software



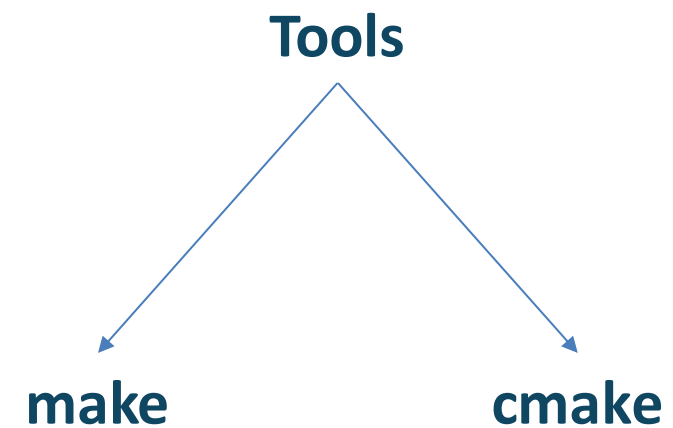
Common Methods of software *installation*

- ✓ Raw programming code
- ✓ Written before its compiled into an executable file

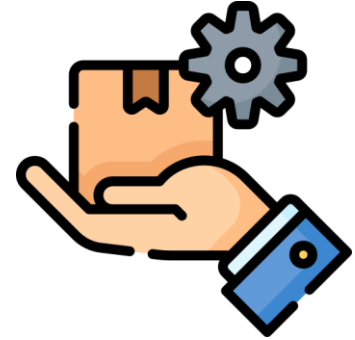


Common Methods of software *installation*

- ✓ Require manual assurance
- ✓ All libraries and tools (dependencies) are installed



Package Manager

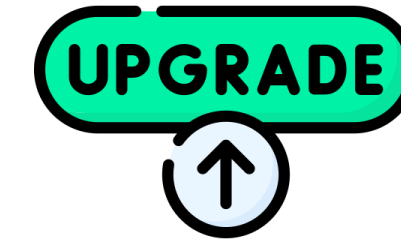


Bundled archives

- ✓ Precompiled binary files
- ✓ Necessary installation scripts
- ✓ Configuration files
- ✓ Metadata



Install

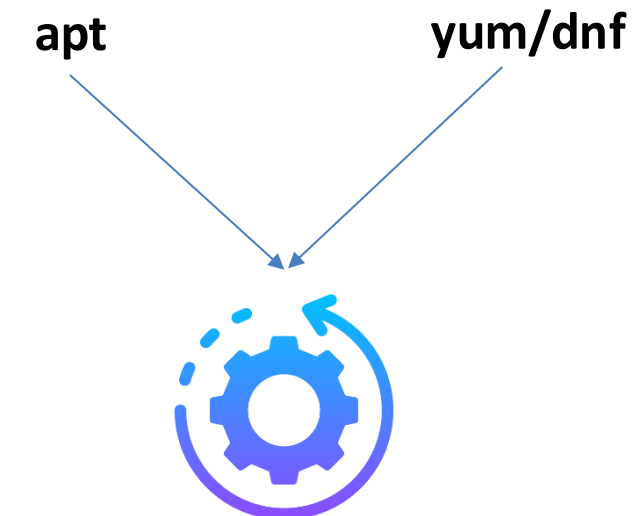


Upgrading



Removing

- ✓ Provide components needed to install and manage software efficiency
- ✓ Simplify process by automating resolution and installation



Package Terminologies

➤ Package Formats

- ✓ Comes in different format
- ✓ Depends upon linux distribution
- ✓ Defines how software is packaged, distributed and installed

.deb



.rpm



Red Hat



Package Terminologies

➤ **Package Manager**

- ✓ Helps in automate installations, maintenance and removal of package



Package Terminologies

➤ Repository

- ✓ Centralized location containing software packages
- ✓ Packages are tested and optimized for specific linux distributions

Debian Based

`“/etc/apt/source.list”`

Red Hat Based

`“/etc/yum.repos.d”`



Package Terminologies

➤ **Dependencies**

- ✓ Some packages requires other packages to function correctly
- ✓ Tools like – apt, yum and dnf automatically handle these requirements



Package Terminologies

➤ Open Source

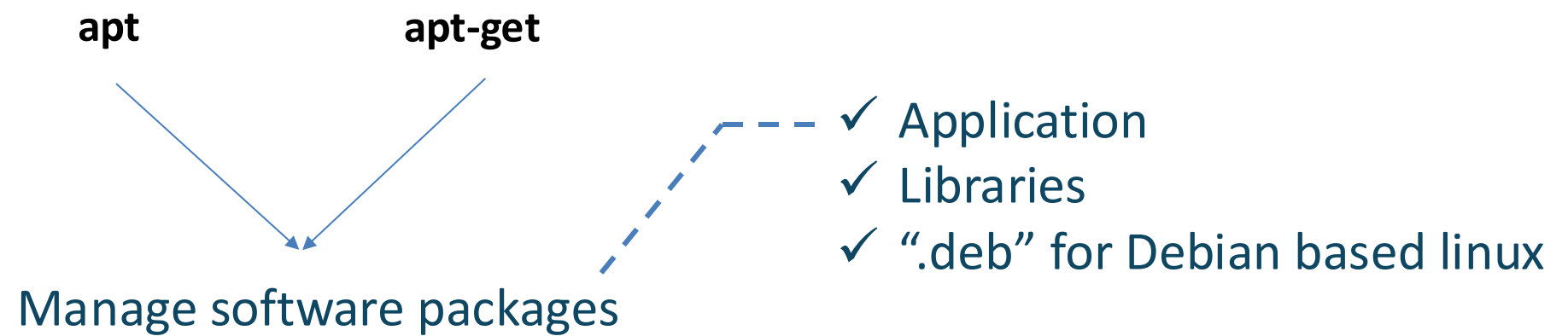
- ✓ Most distributions offer open-source software packages
- ✓ Integrate seamlessly with system
- ✓ Source code can be downloaded from project website



Package Management Tools

➤ apt

- ✓ Advance Package Tool



apt:

- ✓ User-friendly, High-level interface
- ✓ Intuitive commands
- ✓ Enhanced security features
- ✓ Predictable behavior

apt-get:

- ✓ Lower-level tool
- ✓ Interacts more closely with core linux processes



Package Management Tools

➤ yum

- ✓ Yellowdog Updater Modified
- ✓ Package manager for “.rpm” packages on Red-Hat based distribution



Package Management Tools

➤ dnf

- ✓ Dandified YUM
- ✓ Modernized replacement for yum



Package Management Tools

➤ zypper

- ✓ Command-line package manager for OpenSUSE and SUSE Linux Enterprise
- ✓ Advanced dependency resolution and repository management





Linux | apt Package Manager



apt Package Manager

A deb package file name is structured as:

name_version+release_architecture.deb

Example

vim_8.2.2434-3+deb11u1_amd64.deb

- ✓ **NAME:** Describes the contents of the package (e.g., vim)
- ✓ **VERSION:** The software's version number (8.2.2434-3)
- ✓ **RELEASE:** The release number set by the packager (deb11u1)
- ✓ **ARCHITECTURE:** The CPU architecture the package is compiled for:
 - amd_64: 64-bit systems
 - aarch64 for ARM processors



apt Package Manager

- ✓ dpkg focuses on individual .deb files
- ✓ apt acts as a user-friendly front end, streamlining tasks
 - Searching repositories
 - Resolving dependencies
 - Handling software updates
- **Advanced Package Tool (apt)**
 - ✓ Search for available software in the repositories
 - ✓ Install or upgrade packages along with their dependencies
 - ✓ Keep your system secure and up-to-date with regular updates and upgrades
 - ✓ Remove packages or free up disk space by clearing the local cache



apt Package Manager

Command	Purpose	Example
<code>sudo apt update</code>	Refreshes & updates the local package cache with the latest repository information	<code>sudo apt update</code>
<code>sudo apt upgrade</code>	Upgrades all installed packages to their latest versions	<code>sudo apt upgrade</code>
<code>sudo apt full-upgrade</code>	Installs updates, handling changes in dependencies	<code>sudo apt full-upgrade</code>
<code>sudo apt search <package-name></code>	Searches for a specific package in the repository	<code>sudo apt search vim</code>
<code>sudo apt install <package-name></code>	Installs a specified package along with its dependencies	<code>sudo apt install vim</code>



apt Package Manager

Command	Purpose	Example
<code>dpkg -l</code>	List of installed packages with details like name, version, and architecture	<code>dpkg -l</code>
<code>sudo apt remove</code>	Removes a specified package but keeps its configuration files	<code>sudo apt remove vim</code>
<code>sudo apt purge</code>	Installs updates, handling changes in dependencies	<code>sudo apt purge vim</code>
<code>sudo apt clean</code>	Clears the local repository cache, freeing up disk space	<code>sudo apt clean</code>





Linux | Package Manager for Red Hat based Distributions



Introduction

- ✓ Yum, the Yellowdog Updater Modifier
- ✓ Powerful package manager for Red Hat based systems like
 - ✓ RHEL
 - ✓ Rocky Linux
- ✓ Rpm, the Rat Hat Package manager
- ✓ Yum simplifies package management by resolving dependencies
- ✓ Rpm works at granular level for managing individual packages



RPM

- ✓ Originally developed by Red Hat
- ✓ Simplifies software distribution by packing programs into standardized formats
- ✓ Easier software management
- ✓ Admins track and remove installed files
- ✓ Ensures presence of required supporting packages
- ✓ Local rpm database maintains detailed records for installed packages



RPM

➤ Understanding the RPM Package Name

Syntax -> `name-version-release.architecture.rpm`



RPM

➤ Understanding the RPM Package Name

E.g. `vim-minimal-8.2.2637-21.el9.x86_64.rpm`

Name: Describes the contents of the package (e.g. vim-minimal)

Version: The software's version number (8.2.2637).

Release: The release number set by the packager (21.el9).

el9 indicates it's built for Enterprise Linux 9.

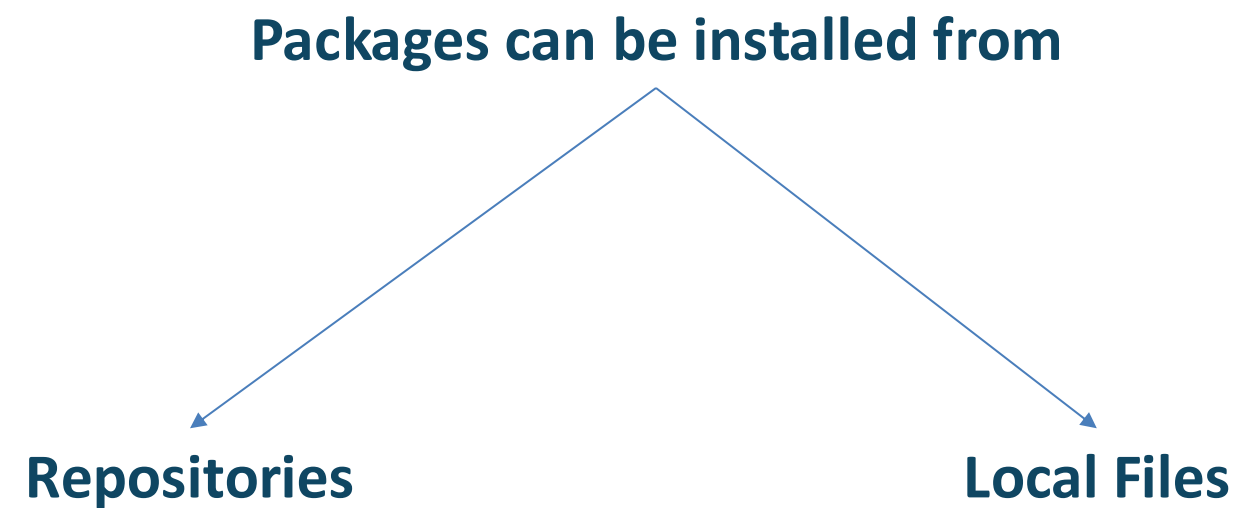
ARCHITECTURE: The CPU architecture the package is compiled for:

- ✓ x86_64: 64-bit systems.
- ✓ Other examples include aarch64 for ARM processors.



RPM

➤ Installing RPM Packages

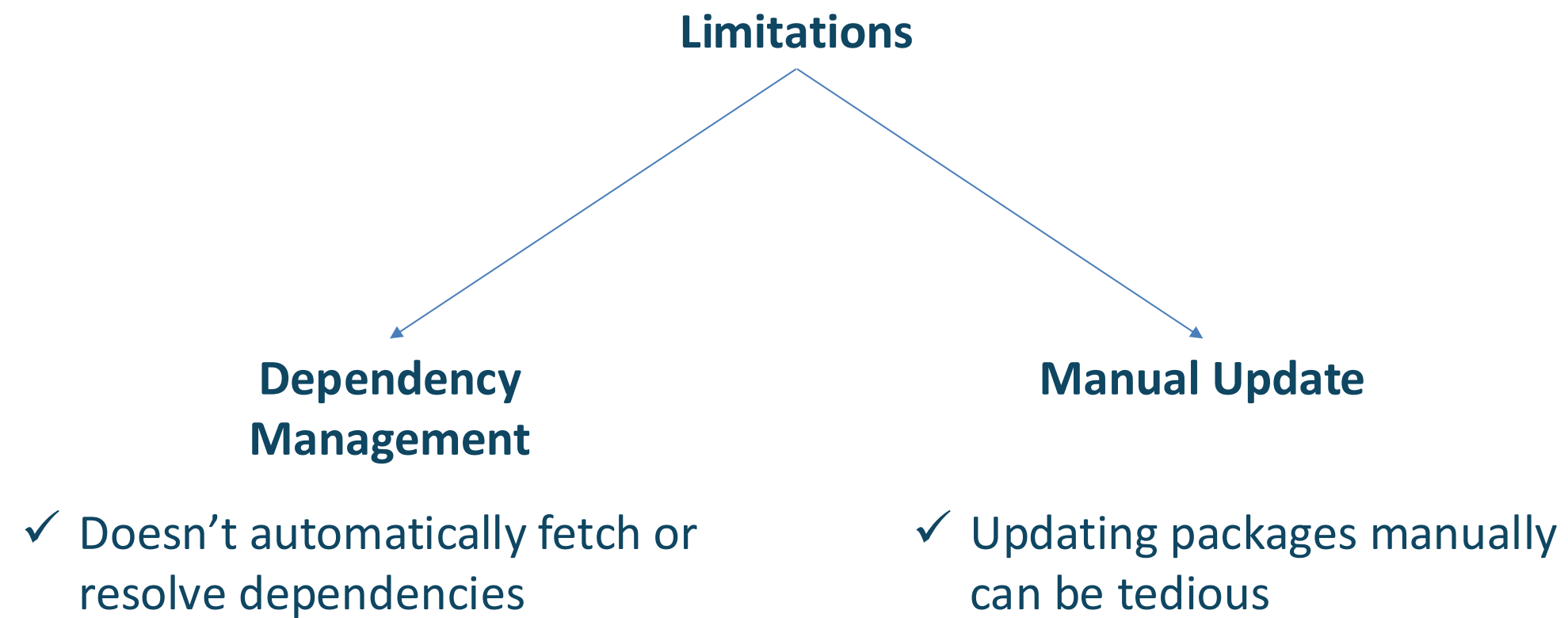


- ✓ Package with latest version is prioritized
- ✓ For multiple releases of same version, highest release number is chosen



RPM

➤ Limitations RPM



✓ To address these issues, yum and dnf were developed



RPM

➤ Essential Commands

- ✓ **rpm -q <package>**: Queries if a specific package is installed
- ✓ **rpm -qa**: Lists all installed packages
- ✓ **rpm -ivh <package.rpm>**: Installs a package, fails if the package is already installed
- ✓ **rpm -Uvh <package.rpm>**: Installs or upgrades a package from a local RPM file
- ✓ **rpm -e <package>**: Removes a specific package



Introduction to Yum

- ✓ Yellowdog Updater Modified
- ✓ Package management in Red Hat-based distributions like CentOS and Fedora
- ✓ Automates tasks like:

Resolving Dependencies

Managing repositories

Updating packages

User-friendly tool that made Linux package management accessible to everyone



YUM

➤ Common Commands

- ✓ **yum list** - Lists available and installed packages
- ✓ **yum search <keyword>** - Searches for packages by keyword
- ✓ **yum install <package>** - Installs a package
- ✓ **yum update** - Updates all packages
- ✓ **yum repolist** - Displays enabled repositories

/etc/yum.conf - Configuration file for Yum settings



DNF

- ✓ Modern package manager that replaces YUM
- ✓ In RHEL9, DNF is used for software management
- ✓ YUM still exists for compatibility with older versions
- ✓ RHEL8 and RHEL9 relies on DNF, but maintain compatibility with YUM



DNF

- ✓ Evolved to dnf
- ✓ Successor to Yum, introduced in Fedora 22 and adopted in RHEL 8+.
- ✓ Enhanced performance and better memory management.
- ✓ Improved dependency resolution and faster metadata processing.



DNF

➤ Common Commands

- ✓ **dnf install <package>**: Installs a specified package along with dependencies.
- ✓ **dnf remove <package>**: Removes a package and its unused dependencies.
- ✓ **dnf update**: Updates all installed packages to their latest versions.
- ✓ **dnf list**: Lists available or installed packages in the system.





Demonstration | Create a local repository in RHEL9

Creating Local Repository

- ✓ Essential for managing software installations

Software Installations
Updates in Environment

- ✓ Limited
- ✓ Restricted
- ✓ Unavailable

Centralized Package
Management

Ensure Consistency

Save Bandwidth

Offline Access



Summary



- ❖ The basics of package
- ❖ APT package manager in Ubuntu
- ❖ Red Hat based package manager: rpm, yum and dnf
- ❖ Creation of local repo in rhel9





Thinknyx Technologies

Linux Course

Enhancing Strategy

Section - 11





Linux | Introduction to Process Management



Process Management



- Processes and their lifecycle
- Manage processes using various commands
- Process priority
- Foreground and Background processes



Linux | Introduction to Processes



Introduction to Processes

➤ What is a Process?

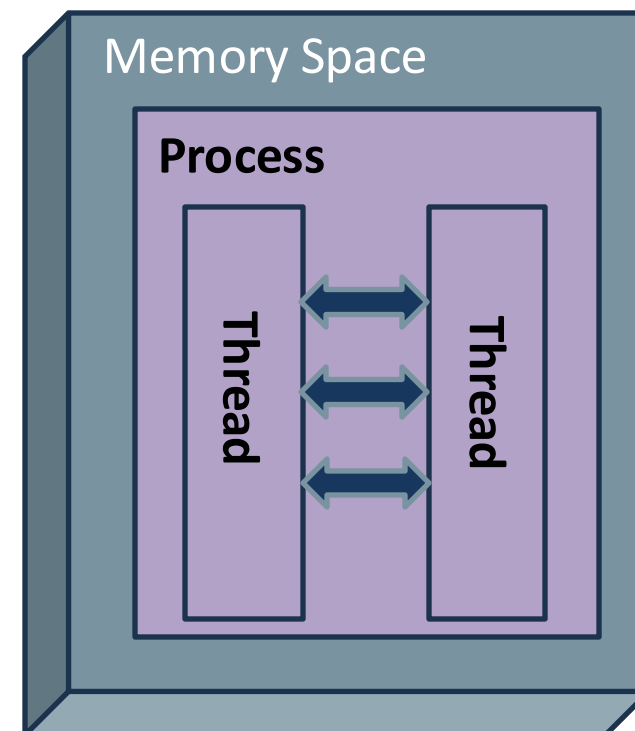
- ✓ A process is a running instance of a program
- ✓ When you open an app, like a web browser, it becomes a process on your system
- ✓ In simple terms, a process is a program that's currently running



Introduction to Processes

➤ What are Threads?

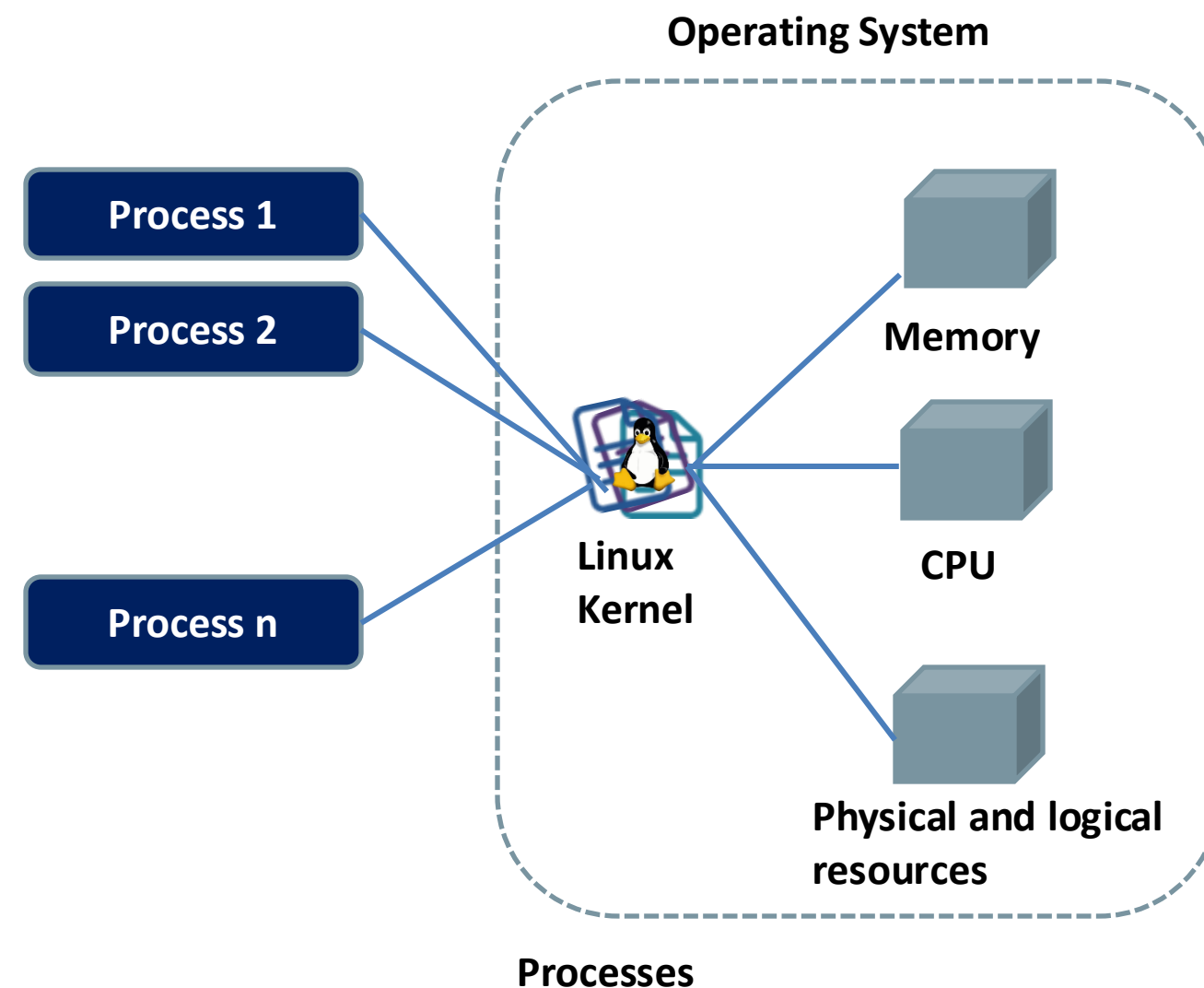
- ✓ A thread is the smallest unit of work in a process
- ✓ Each process has a main thread and can create more for multitasking
- ✓ Threads share memory, making them lightweight and efficient on multi-core systems



Introduction to Processes

➤ Processes and Resources

- ✓ Processes use memory, CPU, and devices like network cards and hard drives
- ✓ The kernel manages resource allocation to optimize performance



Introduction to Processes

➤ Process Creation in Linux

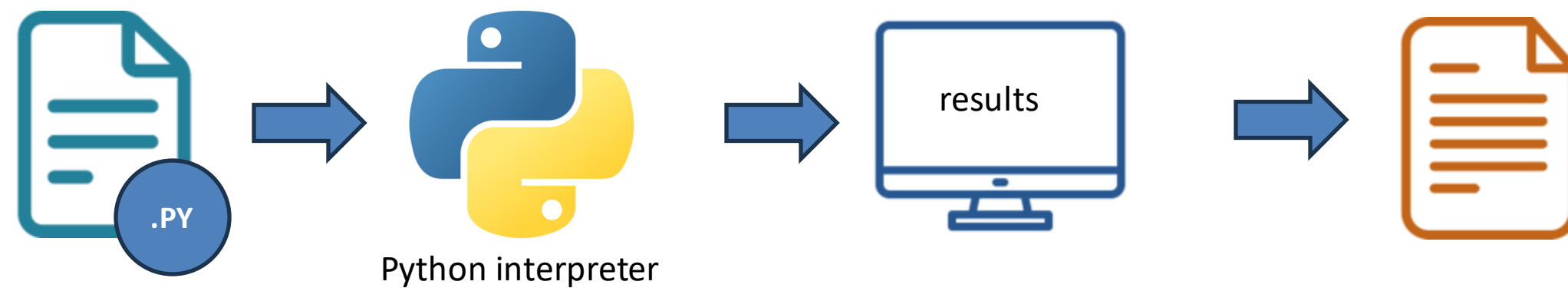
- ✓ All processes start from systemd, the first system process
- ✓ Parent processes create child processes using the fork mechanism, inheriting resources like file access and environment variables
- ✓ The child process often uses exec to replace its inherited code with a new program
- ✓ Each process has a unique PID and a PPID (Parent Process ID)
- ✓ The parent "sleeps" while the child runs and clears unused resources after the child finishes



Introduction to Processes

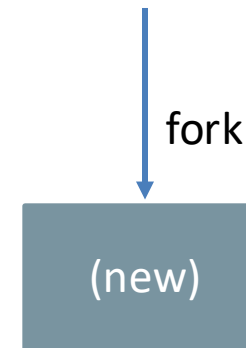
➤ Process Lifecycle in Linux

- ✓ A script runs as a process, moving through various states
- ✓ The scheduler allocates CPU time based on priority and time needed



Introduction to Processes

➤ Created (New)

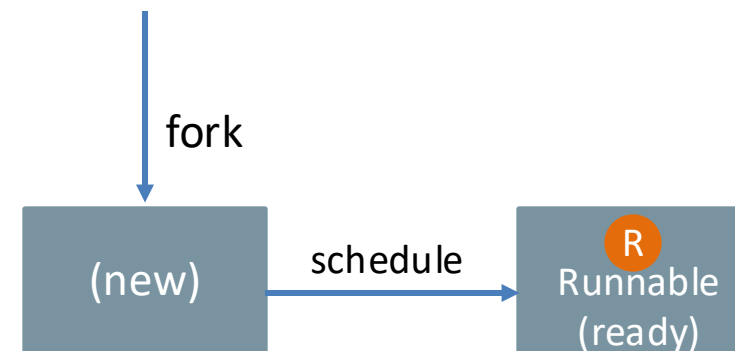


- ✓ The OS uses fork to create a new process for the Python interpreter
- ✓ The process exists but hasn't started executing yet
- ✓ It waits to be admitted into the scheduler's queue, with no specific state identifier until then



Introduction to Processes

➤ Ready (R)

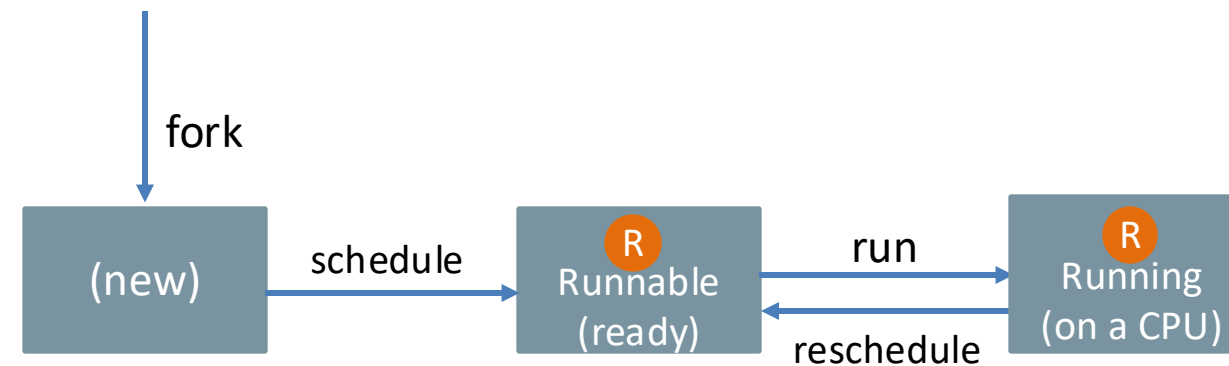


- ✓ After creation, the process enters the Ready state (R)
- ✓ The scheduler adds the process to its queue, waiting for its turn to use the CPU
- ✓ The scheduler evaluates the queue based on priority and fairness to decide the next process to run



Introduction to Processes

➤ Running (R)

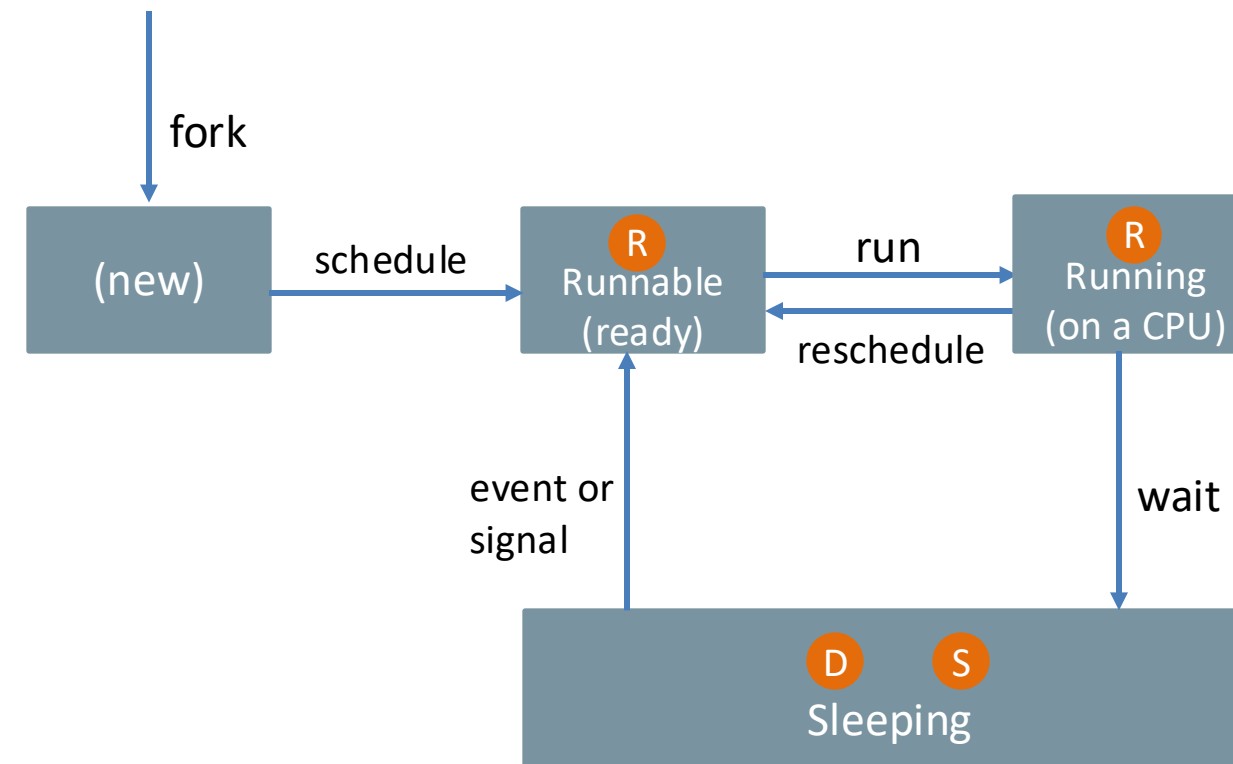


- ✓ Once assigned the CPU, the process enters the Running state (R)
- ✓ The Python script executes tasks like opening files, reading, and processing data
- ✓ The scheduler periodically switches processes to ensure fairness, so the script may not stay in this state continuously



Introduction to Processes

➤ Sleeping (S or D)

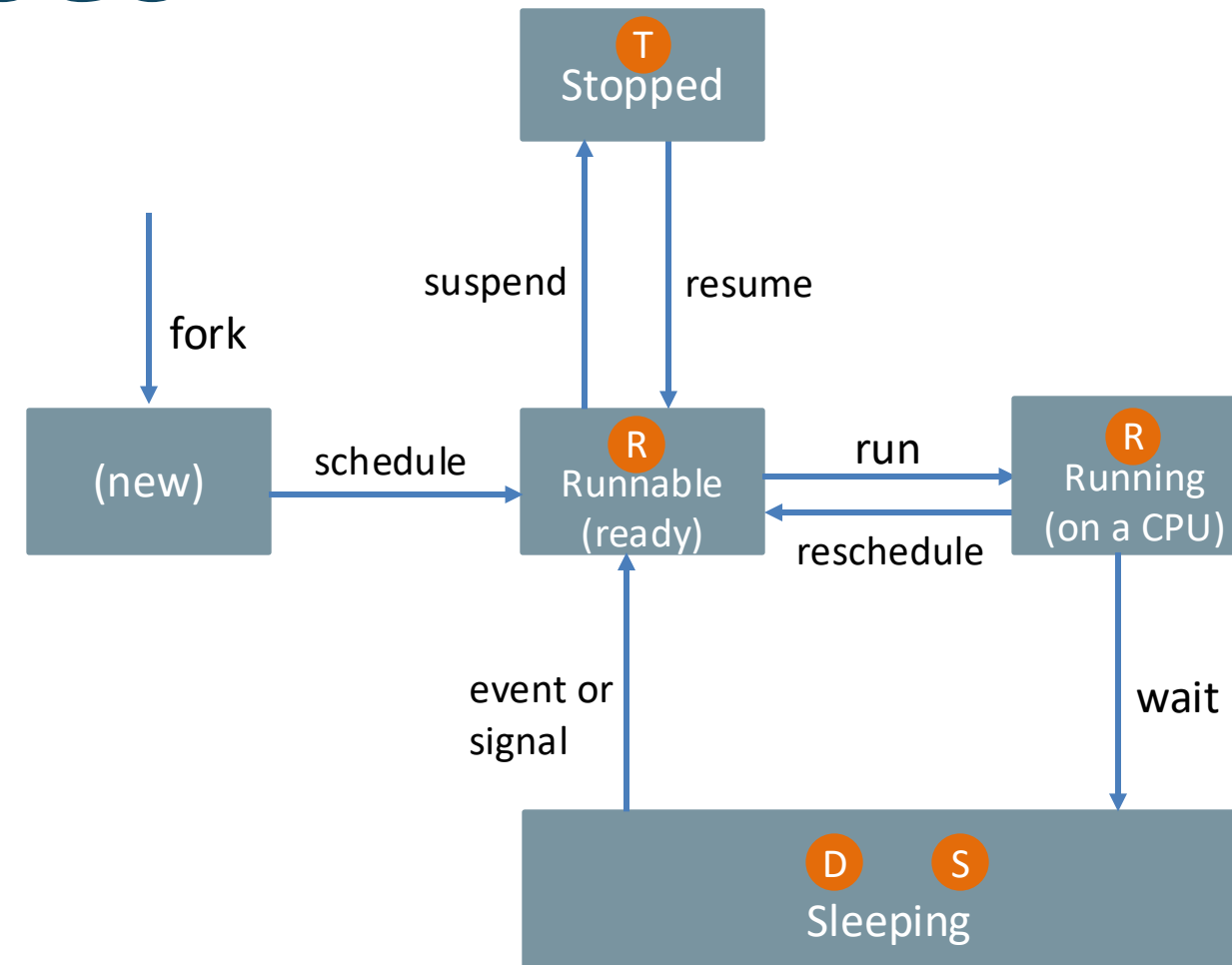


- ✓ The process enters Sleeping state when it pauses, waiting for resources like file data or database input
- ✓ Interruptible Sleep (S) allows the process to wake up when the resource is ready or a signal is received
- ✓ Uninterruptible Sleep (D) occurs when the process waits for a critical resource and can't be interrupted
- ✓ During sleep, the scheduler focuses on other processes that are ready to run



Introduction to Processes

➤ Stopped (T)

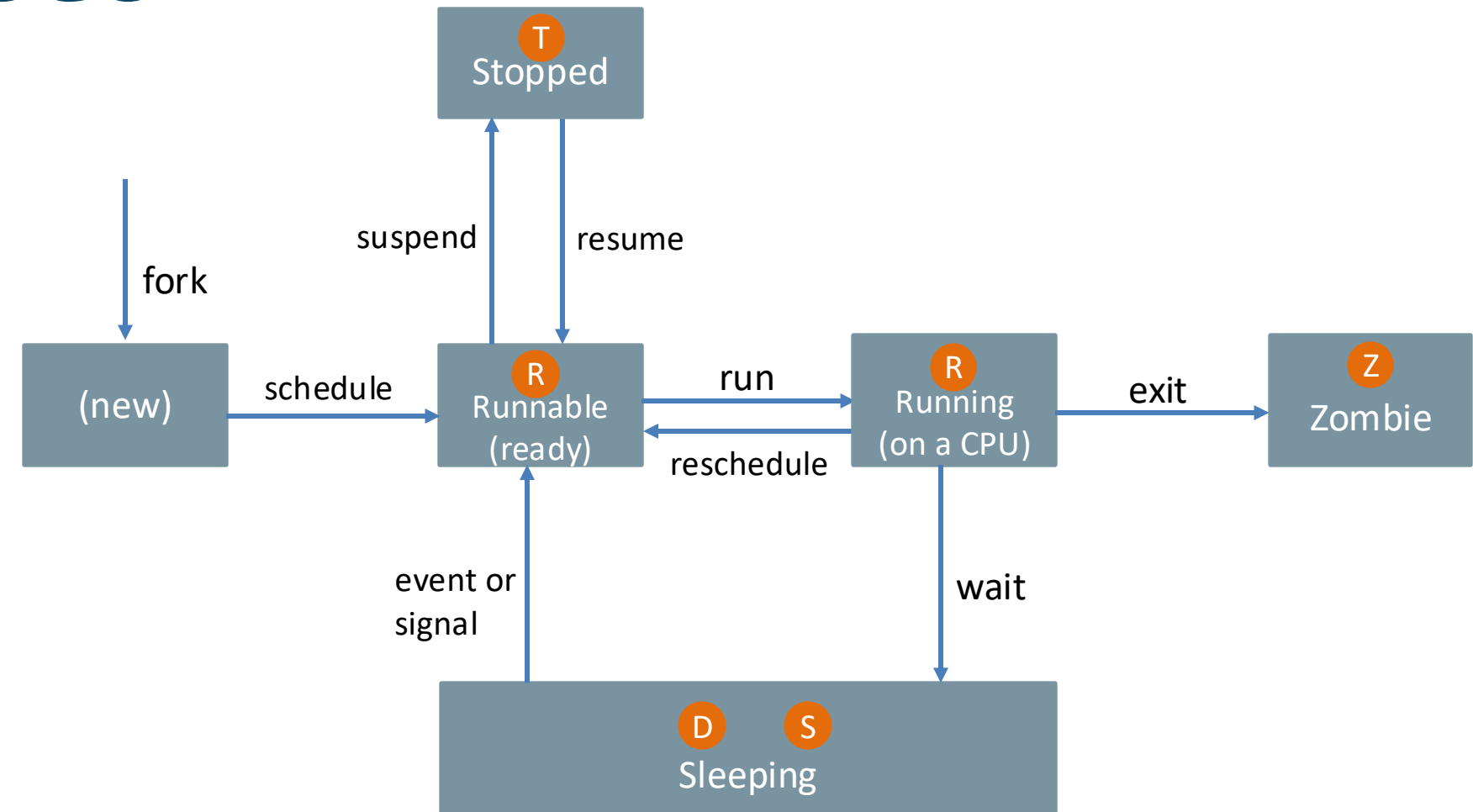


- ✓ When manually paused (e.g., pressing Ctrl+Z), the process enters the Stopped state (T)
- ✓ The process halts execution, releases the CPU, but stays in memory
- ✓ It can be resumed later using the fg (foreground) command
- ✓ While stopped, the scheduler does not allocate CPU time to the process



Introduction to Processes

➤ Zombie (Z)

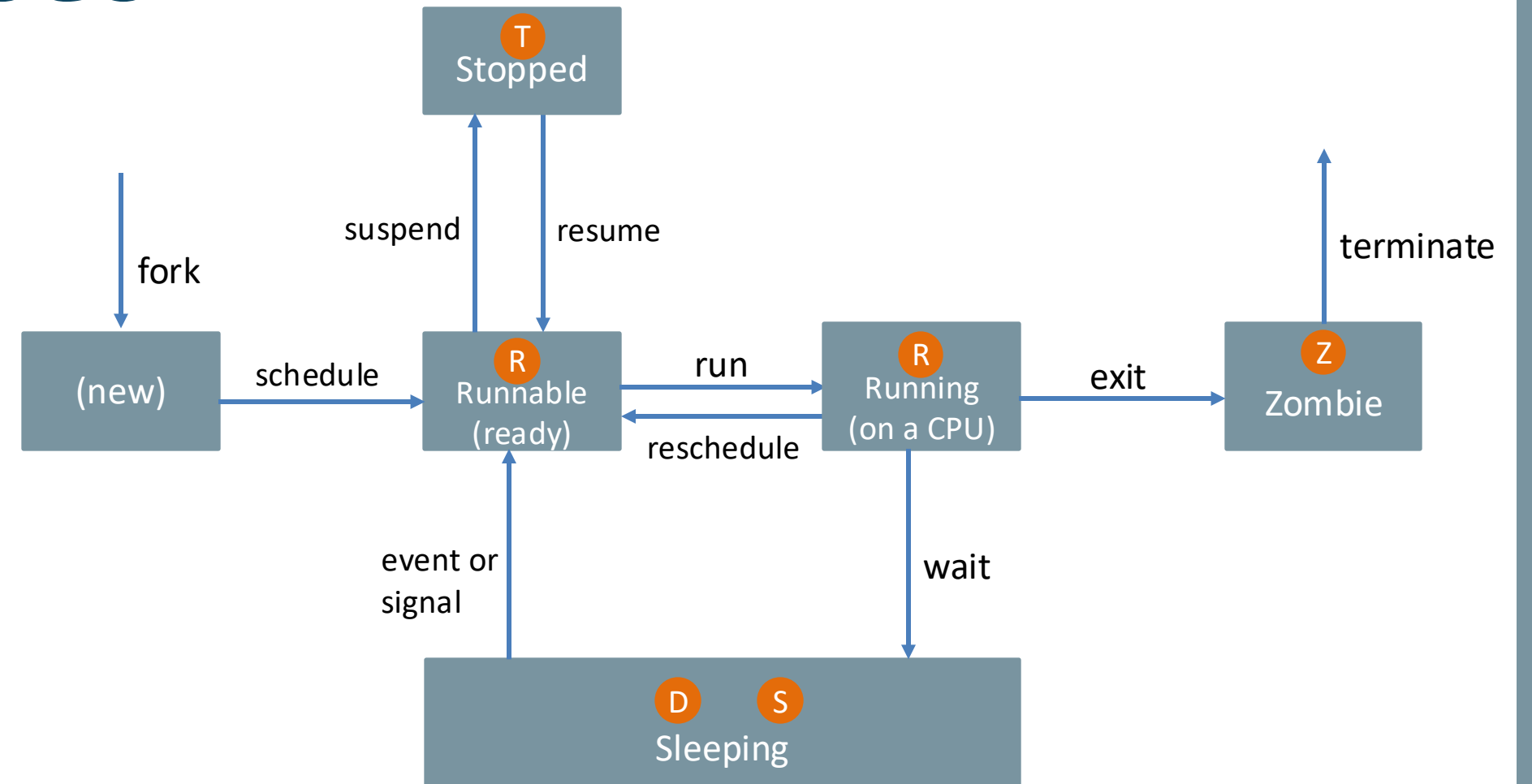


- ✓ After execution, the process enters the Zombie state (Z)
- ✓ The process has completed its work, but the parent process hasn't acknowledged its exit status
- ✓ This state is transitional and doesn't consume system resources
- ✓ The scheduler is not involved, as the process is no longer active



Introduction to Processes

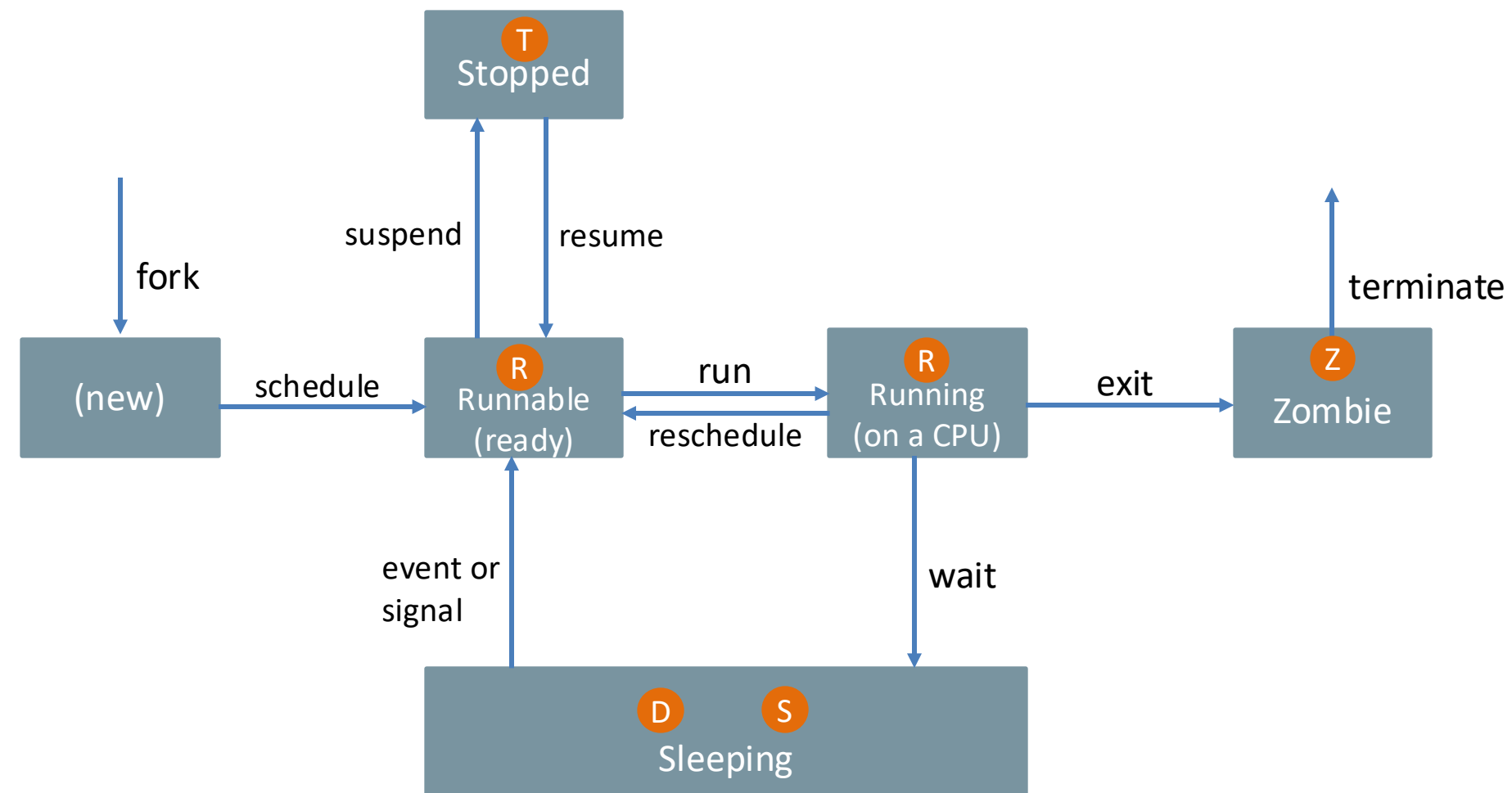
➤ Terminated (Dead)



- ✓ After the parent process acknowledges termination, the process enters the Terminated state
- ✓ All resources (memory, file handles) are freed, and the process is removed from the system
- ✓ The process no longer exists in the process table
- ✓ The scheduler's role is finished, marking the end of the script's lifecycle



Introduction to Processes





Linux | Process Management



Process Management

➤ Fundamental Commands Related to Processes

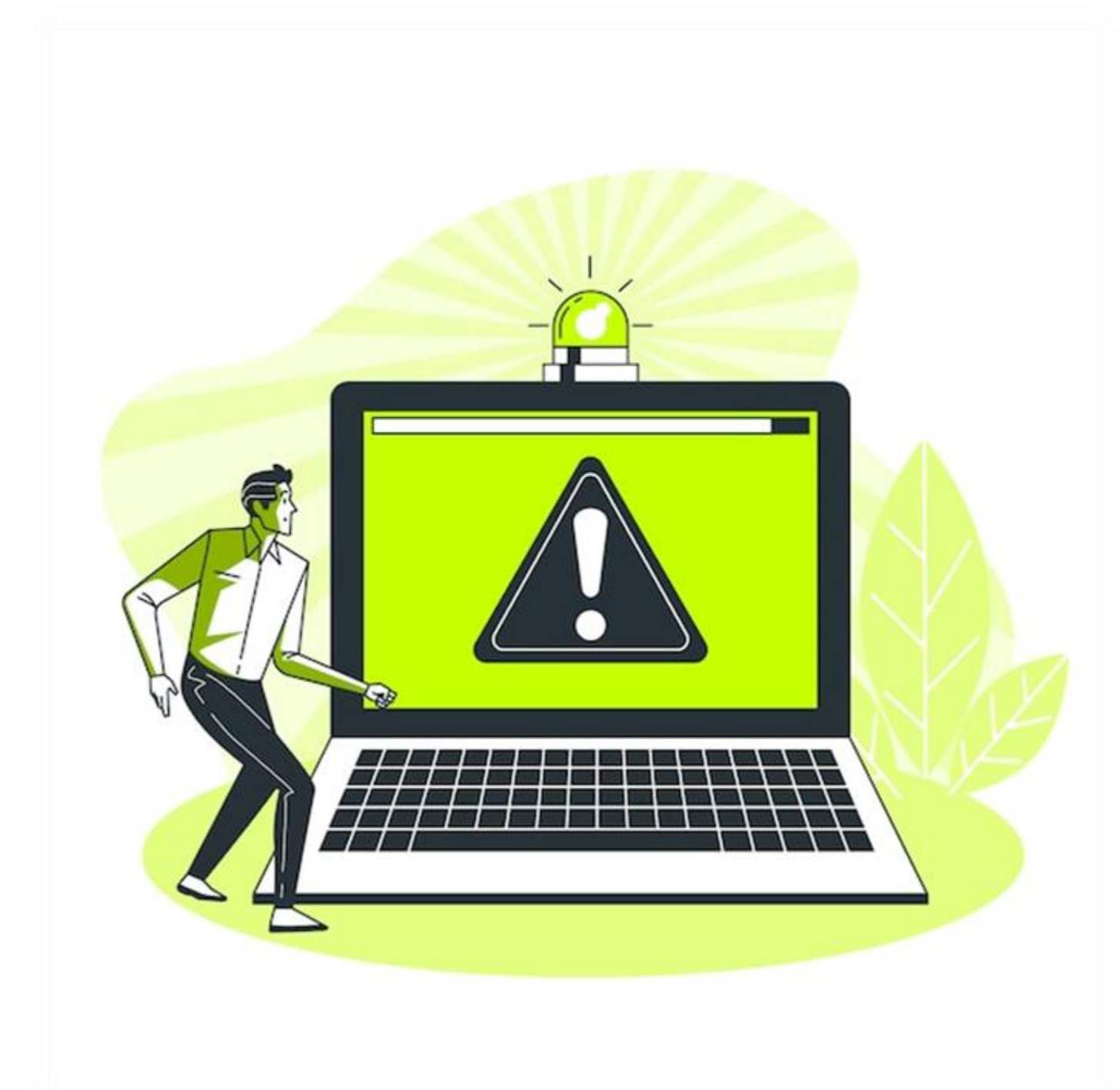
Command	Description	Syntax	Usage
pidof	Finds the process ID of a running program	pidof [options] <program_name>	pidof sshd
ps	Displays information about a selection of the active processes	ps [options]	ps aux
pstree	Shows running processes as a tree	pstree [options]	pstree
psgrep	Searches for processes by name and returns their process IDs	pgrep [options] <pattern>	pgrep -u user1



a	all users
u	user, CPU, and memory usage
x	not tied to a specific terminal



Process Management



Process Management

- ✓ Terminate the Process
- ✓ Free Up Resources
- ✓ Restore Server Performance
- ✓ Maintain User Experience



Process Management

- ✓ Sends signals to manage processes, such as TERM, KILL, or STOP
- ✓ **Default signal:** TERM (graceful termination)
- ✓ **KILL:** Forcefully stops a process
- ✓ **STOP:** Pauses the process



Process Management

- ✓ Only owners or root users can kill processes
- ✓ The term "kill" can be misleading
- ✓ The command sends various signals, not limited to termination



Process

Management

1) SIGHUP	2) SIGINT	3) SIGQUIT	4) SIGILL	5) SIGTRAP
6) SIGABRT	7) SIGBUS	8) SIGFPE	9) SIGKILL	10) SIGUSR1
11) SIGSEGV	12) SIGUSR2	13) SIGPIPE	14) SIGALRM	15) SIGTERM
16) SIGSTKFLT	17) SIGCHLD	18) SIGCONT	19) SIGSTOP	20) SIGTSTP
21) SIGTTIN	22) SIGTTOU	23) SIGURG	24) SIGXCPU	25) SIGXFSZ
26) SIGVTALRM	27) SIGPROF	28) SIGWINCH	29) SIGIO	30) SIGPWR
31) SIGSYS	34) SIGRTMIN	35) SIGRTMIN+1	36) SIGRTMIN+2	37) SIGRTMIN+3
38) SIGRTMIN+4	39) SIGRTMIN+5	40) SIGRTMIN+6	41) SIGRTMIN+7	42) SIGRTMIN+8
43) SIGRTMIN+9	44) SIGRTMIN+10	45) SIGRTMIN+11	46) SIGRTMIN+12	47) SIGRTMIN+13
48) SIGRTMIN+14	49) SIGRTMIN+15	50) SIGRTMAX-14	51) SIGRTMAX-13	52) SIGRTMAX-12
53) SIGRTMAX-11	54) SIGRTMAX-10	55) SIGRTMAX-9	56) SIGRTMAX-8	57) SIGRTMAX-7
58) SIGRTMAX-6	59) SIGRTMAX-5	60) SIGRTMAX-4	61) SIGRTMAX-3	62) SIGRTMAX-2
63) SIGRTMAX-1	64) SIGRTMAX			



Process

Management

`kill [options] <pid>`



Signal number (e.g., 9 for SIGKILL)

Full signal name (e.g., SIGKILL)

Signal name without the SIG prefix (e.g., KILL)

```
kill -9 <pid>
kill -SIGKILL <pid>
kill -KILL <pid>
```



Process Management

PID Value	Effect
PID > 0	Signal targets the process with the specified PID
PID = 0	Signal targets all processes in the current process group
PID = -1	Signal targets all user-owned processes; as root, it excludes init and itself
PID < -1	Signal targets all processes in the group with a GID matching the absolute value of the PID

- ✓ Use signal names, not numbers, for consistency and cross-platform portability



Process Management

- ✓ Unsure of the process ID or prefer to target processes by name

killall

- ✓ Sends signals to processes based on names, more flexible than killall
- ✓ Uses pgrep criteria (name, user, pattern) to target processes



Process Management

For example

```
kill -u ubuntu vi
```

Default signal (SIGTERM) to all matching processes

```
kill -9 -u ubuntu vi
```

Additional options -> terminal (-t) or session (-s)



Process Management

- ✓ Avoid killing system processes to prevent instability or crashes
- ✓ **SIGTERM:** Gracefully terminates processes
- ✓ **SIGKILL:** Forcefully terminates without cleanup, causing potential issues
- ✓ Use signals carefully to avoid unintended consequences



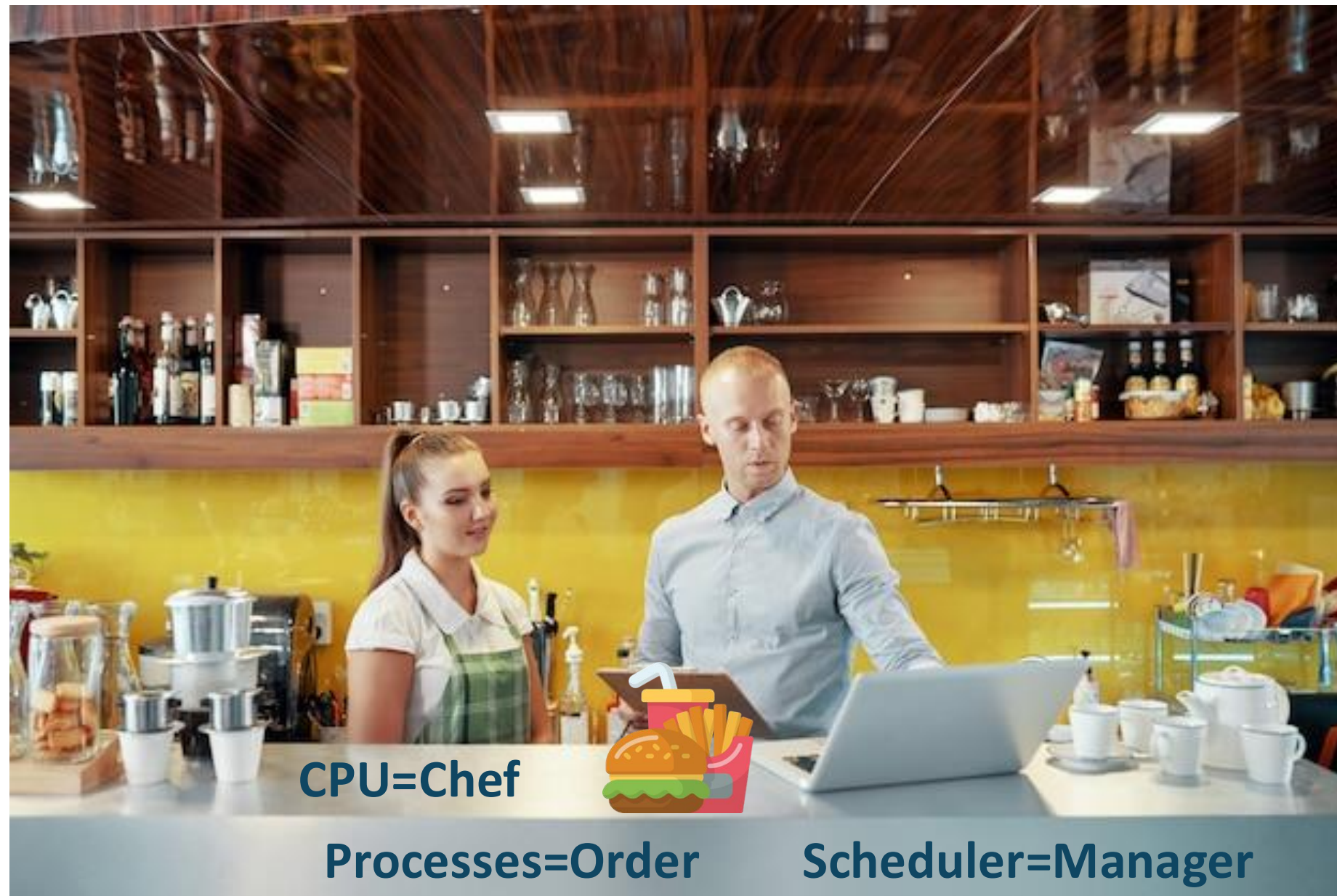


Linux | Process Priorities



Process Priorities

➤ Nice and Renice



- ✓ Lower nice value = Higher priority
- ✓ Higher nice value = Lower priority



Process Priorities

➤ **Adjusting Process Priority**

❑ **nice Command**

- ✓ Used when starting a process
- ✓ Sets the initial priority

❑ **renice Command**

- ✓ Used for running processes
- ✓ Dynamically adjusts priority



Process Priorities

➤ **Process Priority in Multi-Core Systems**

- ✓ **Parallel Execution:** Processes run on multiple cores
- ✓ **Priority Matters:** Higher priority processes are favored
- ✓ **Optimized Performance:** Tasks balanced across cores



Process Priorities

➤ Understanding Nice Values

- ✓ **Range:** -20 (highest priority) to 19 (lowest priority)
- ✓ **High Priority (-20):** Receives more CPU time
- ✓ **Low Priority (19):** CPU favors higher-priority processes first
- ✓ **Default:** 0

Ensures fair CPU resource allocation

```
nice -n -10
```

```
renice -n -5 1234
```



Process Priorities

➤ Monitoring System Performance: top vs. ps

❑ ps Command

- ✓ Provides a static snapshot of processes
- ✓ Requires manual reruns for updates
- ✓ Useful for specific point-in-time process data

❑ top Command

- ✓ **Offers real-time, dynamic updates (default: every 2 seconds[Interval updates can be customized])**
- ✓ Displays processes consuming the most CPU and memory
- ✓ Allows for continuous monitoring without rerunning commands



Process Priorities

➤ Why Use top?

- ✓ Efficient for ongoing performance tracking
- ✓ Easier to monitor system health dynamically



Process Priorities

➤ Understanding Load Average

- ✓ Shows processes waiting for CPU time
- ✓ Load of 1 fully uses a single-core CPU; >1 causes contention
- ✓ Multi-core ideal load matches core count (e.g., 4 cores = load of 4)
- ✓ High load suggests CPU overuse or system issues



Process Priorities

- ✓ High kernel time (sy) with low user time (us) suggests system-level issues
- ✓ Low idle time with high us or sy indicates heavy CPU usage
- ✓ High wa values suggest memory or disk bottlenecks, with the CPU waiting on storage
- ✓ High hi or si indicates hardware or software interruption issues
- ✓ High st values suggest the host system is overloaded, affecting the VM's performance





Linux | Foreground and Background Processes



Foreground and Background Processes

- ✓ **Foreground:** Directly interacted with in the terminal, blocking other tasks until finished or stopped
- ✓ **Background:** Run independently, allowing terminal use for other tasks while still being monitored or controlled



Foreground and Background Processes

Command	Description	Syntax	Usage
jobs	Lists all background jobs and their statuses	jobs	jobs
fg	Brings a background process to the foreground	fg %<job_id>	fg %1
bg	Resumes a paused process in the background	bg %<job_id>	bg %1

- ✓ The kernel also runs background processes (kernel threads) for essential system tasks



Summary



- ❖ **Concept of processes and their lifecycle**
- ❖ **Process states:** ready, sleep and stop
- ❖ **Managing Processes:** pidof, ps and kill
- ❖ **Process Priorities:** nice and renice
- ❖ **Real-Time Monitoring:** top
- ❖ **Foreground and Background Processes:** fg, bg and jobs





Thinknyx Technologies

Linux Course

Enhancing Strategy

Section - 12





Linux | Service Management



Service Management



- Service management concepts
- Commands for managing services in Linux
- systemd and systemctl
- Install Apache on Ubuntu
- Manage Apache service using systemctl



Linux | Introduction to Service Management



Service Management

- ✓ Web servers, databases, and logging systems run smoothly without manual intervention. How?



Service Management

➤ Services

- ✓ Are background processes vital for system operation
- ✓ Handle tasks like networking, scheduling, and more
- ✓ Manage critical tasks like web hosting and database access
- ✓ Service management controls how services start, stop, and interact



Service Management

➤ SysVinit VS Systemd

❑ SysVinit

- ✓ Linux used the SysVinit system for managing services
- ✓ Launched essential services during system boot
- ✓ SysVinit faced challenges like managing dependencies between services



- ✓ SysVinit lacked an efficient way to handle dependencies
- ✓ delays or even failures during startup

➤ Systemd

- ✓ Modern replacement for SysVinit
- ✓ Introducing features like parallel startup, dependency resolution, and robust logging
- ✓ Handles entire boot process, ensures system resources allocated efficiently
- ✓ Tracks all processes



Service Management

➤ **Daemons**

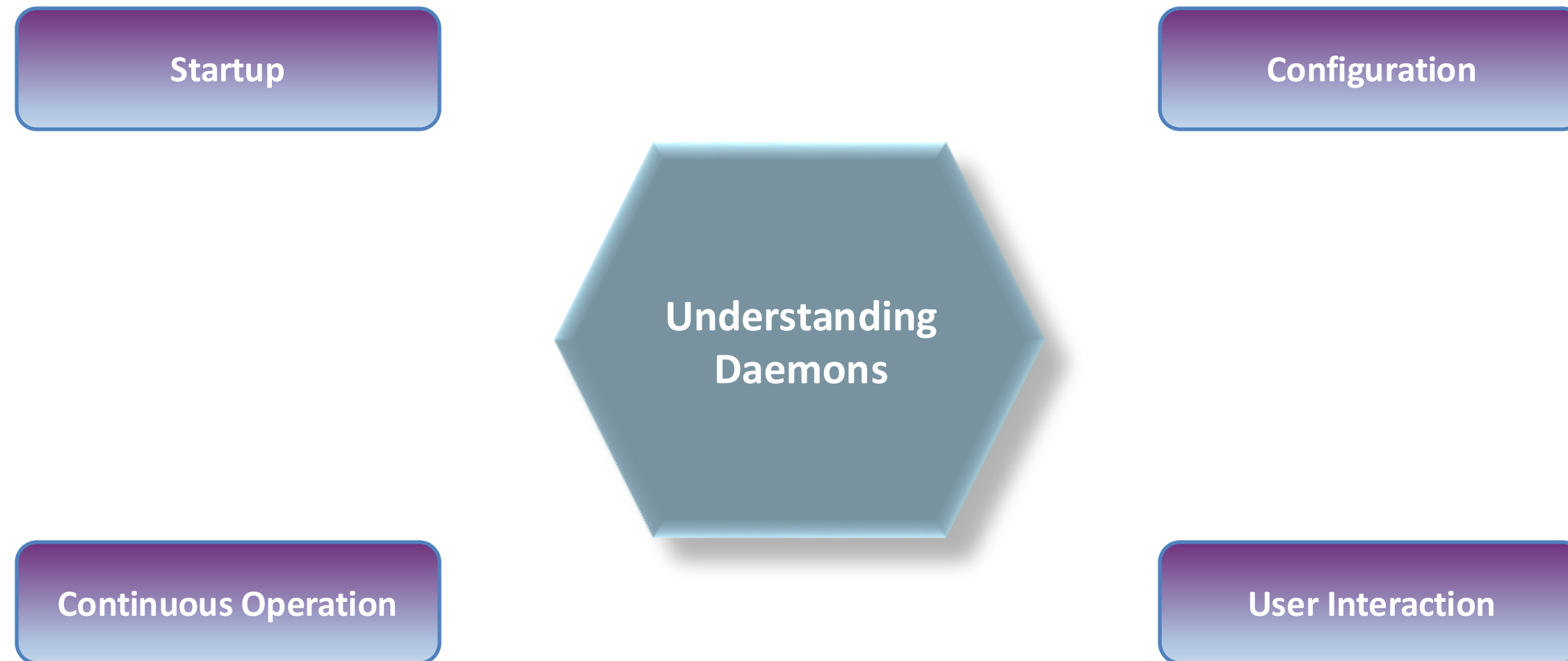
- ✓ Daemons are a subset of services managed by tools like systemd
- ✓ Processes designed to run continuously in the background
- ✓ Systemd interacts with these daemons to control their lifecycle

For Example

- ✓ Systemd initializes the sshd daemon, which handles secure remote logins
- ✓ Systemd can restart a daemon if it fails
- ✓ Services remain available without manual intervention

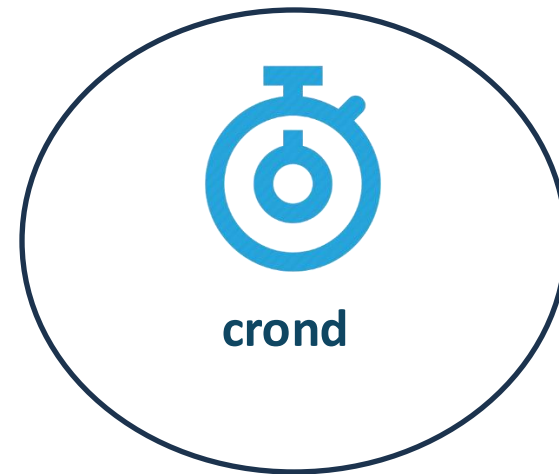


Service Management



Service Management

➤ Examples of Daemons





Linux | Role of systemd



Role of systemd

- ✓ Systemd is responsible for initializing the system and managing services
- ✓ Services are launched in the correct order and handles dependencies between services
- ✓ Responsible for controlling the lifecycle of services



Role of *systemd*

❑ Boot the System

- ✓ Kernel loads and initializes systemd
- ✓ It initializes the system by launching essential services
- ✓ It optimizes the boot process using parallelization

❑ Control Services

- ✓ Start, stop, restart, enable, or disable services
- ✓ Handle dependencies between services
- ✓ Use timers to schedule tasks



Role of *systemd*

❑ Resource Management

- ✓ Integrates with cgroups (control groups) to monitor and control resource

❑ Logging and Troubleshooting

- ✓ Systemd includes journalctl, a centralized logging utility
- ✓ Logs are invaluable for troubleshooting and auditing



Role of *systemd*

❑ Power and Session Management

- ✓ Handles power management tasks like shutting down, rebooting, and hibernating the system

❑ Advanced Networking and System States

- ✓ Supports managing network configurations using systemd-networkd
- ✓ Logs are invaluable for troubleshooting and auditing



Role of *systemd*

systemd introduces units

- ✓ *Services*
- ✓ *Sockets*
- ✓ *Devices*
- ✓ *Timers*



Role of *systemd*

→ Important systemd units

Service Units (.service)

Socket Units (.socket)

Path Units (.path)





Linux | Introduction to systemctl



Introduction to *systemctl*

Command	Description	Usage
<code>systemctl start <service></code>	Starts a service immediately	<code>systemctl start apache2</code>
<code>systemctl stop <service></code>	Stops a running service	<code>systemctl stop apache2</code>
<code>systemctl restart <service></code>	Stops and then starts a service. Useful after config changes	<code>systemctl restart apache2</code>
<code>systemctl reload <service></code>	Reloads a service without stopping it	<code>systemctl reload apache2</code>
<code>systemctl enable <service></code>	Configures the service to start automatically on boot	<code>systemctl enable apache2</code>
<code>systemctl disable <service></code>	Prevents the service from starting on boot	<code>systemctl disable apache2</code>



Introduction to *systemd*

Command	Description	Usage
<code>systemctl status <service></code>	Displays the current status of a service (e.g., active or inactive)	<code>systemctl status apache2</code>
<code>systemctl is-active <service></code>	Checks if the service is currently running	<code>systemctl is-active apache2</code>
<code>systemctl is-enabled <service></code>	Checks if the service is set to start on boot	<code>systemctl is-enabled apache2</code>
<code>systemctl list-units</code>	Lists all the systemd units currently loaded	<code>systemctl list-units</code>
<code>systemctl list-units --type service --all</code>	Lists systemd units & filters by type, as services	<code>systemctl list-units --type=service</code>





Demonstration | Managing Services

User Management

File	Description
UNIT	Service unit name, e.g., cron.service
LOAD	Loaded into memory; typically shows loaded if successful
ACTIVE	Common states include active or inactive
SUB	Detailed state, varies by service type and behavior, e.g., running, exited, waiting (active), or dead, failed (inactive)
Description	Brief purpose of the service



User Management

Active State	Description
active (exited)	Service started successfully and completed its task (one-shot)
active (waiting)	Service running but waiting for an event or condition to proceed



Summary



- ❖ Services and daemons
- ❖ Role of systemd in service management
- ❖ Common systemctl commands
- ❖ Installed and managed Apache2 using systemctl





Thinknyx Technologies

Linux Course

Enhancing Strategy

Section - 13





Linux | Disk Partitioning



Disk Partitioning



- Types of filesystems
- Fundamentals of disk partitioning
- Demonstration of creating and managing disk partitions



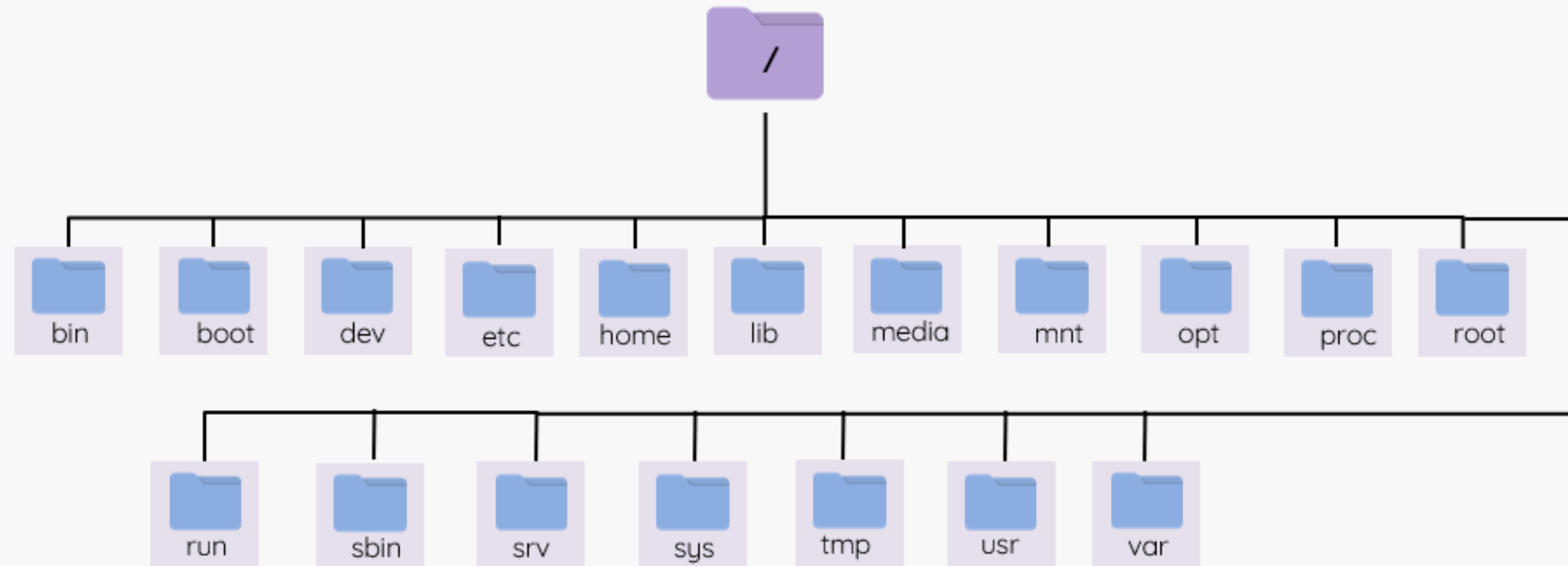
Linux | Filesystem Types



Filesystem Types

➤ What is a Filesystem?

- ✓ Defines how data is stored, accessed, and organized
- ✓ Manages file naming, directory structure, and metadata



Filesystem Types

➤ Filesystem Types

- ✓ Specific implementations of file storage structures
- ✓ Each offers unique features, optimizations, and limitations
- ✓ Suitable for different scenarios



Filesystem Types

➤ Journaling

- ✓ Used by ext4, XFS to ensure data consistency
- ✓ Logs changes before applying them
- ✓ Enables recovery from crashes, preventing data corruption
- ✓ Replays journal on restart to apply or roll back changes



Filesystem Types

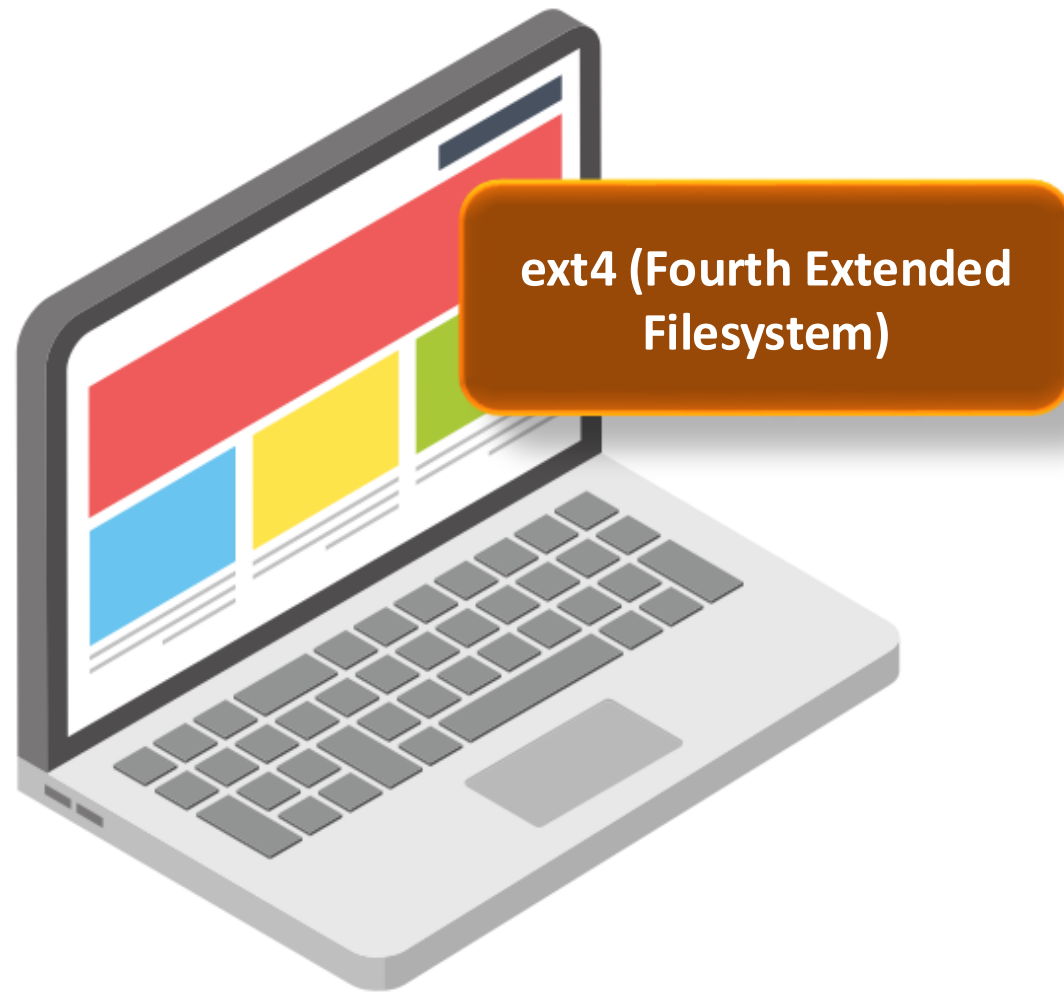
➤ Inodes

- ✓ Data structures storing information about files and directories
- ✓ Contain metadata: size, permissions, timestamps, and pointers to data blocks
- ✓ Help manage and access files efficiently by separating metadata from content
- ✓ Enables recovery from crashes, preventing data corruption



Filesystem Types

➤ Popular Filesystem Types



- ✓ Default for many Linux distributions
- ✓ Uses journaling for crash recovery
- ✓ Stores metadata in inodes (permissions, timestamps, data pointers)
- ✓ Reliable and performs well for general-use scenarios



Filesystem Types

➤ Popular Filesystem Types

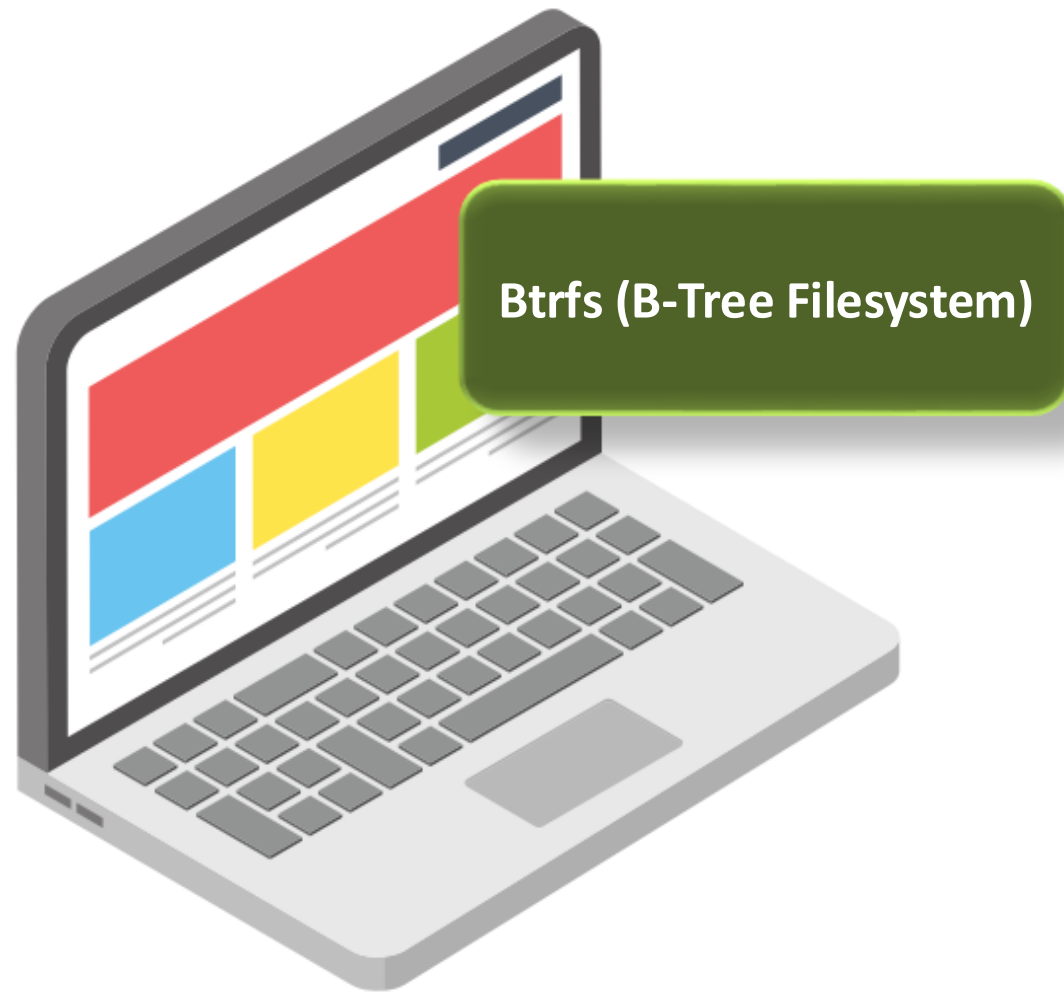


- ✓ High-performance filesystem with journaling for data consistency
- ✓ Uses B-trees for better scalability and faster file lookups
- ✓ Ideal for large filesystems like databases and high-capacity servers



Filesystem Types

➤ Popular Filesystem Types

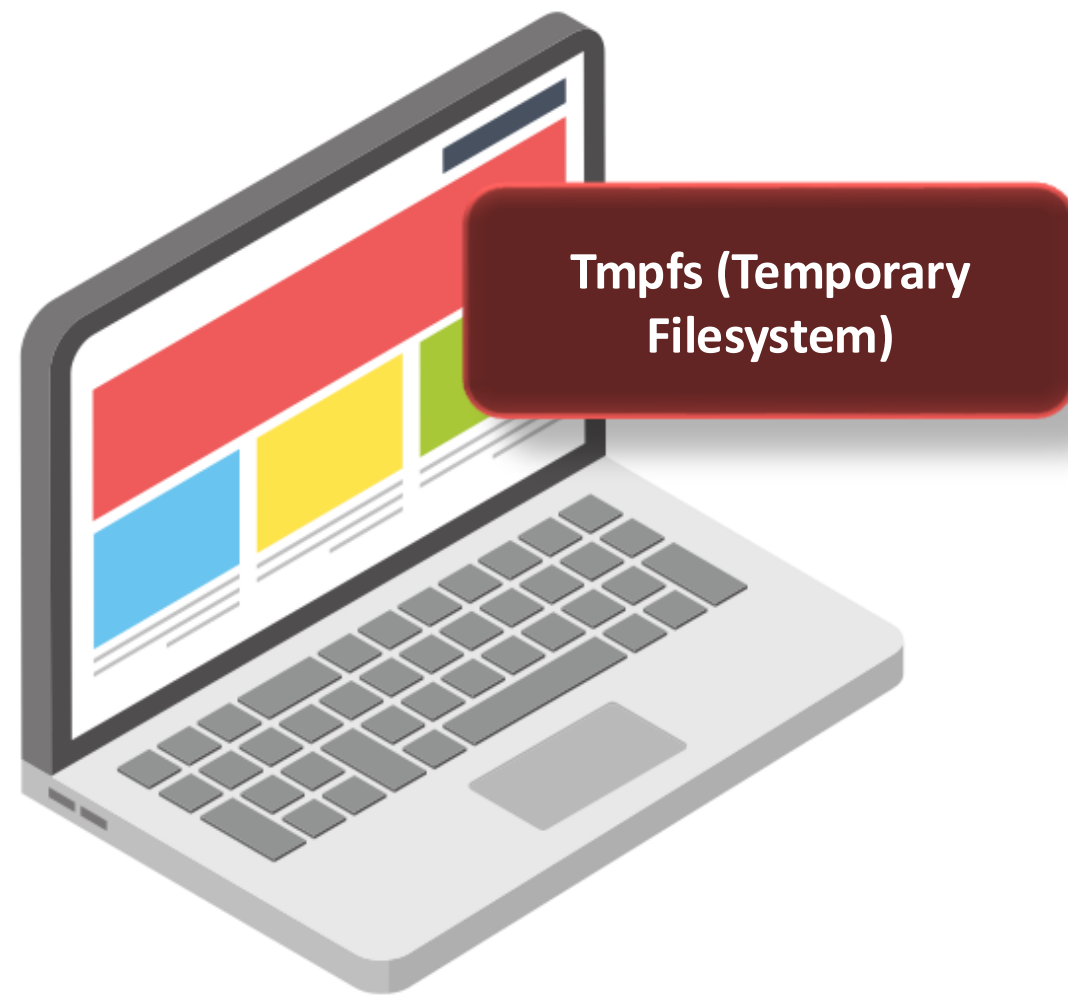


- ✓ Modern filesystem with advanced features:
 - Snapshots
 - Data checksumming
 - Copy-on-write (COW) for efficient storage
- ✓ Supports dynamic resizing and data integrity in complex storage setups



Filesystem Types

➤ Popular Filesystem Types

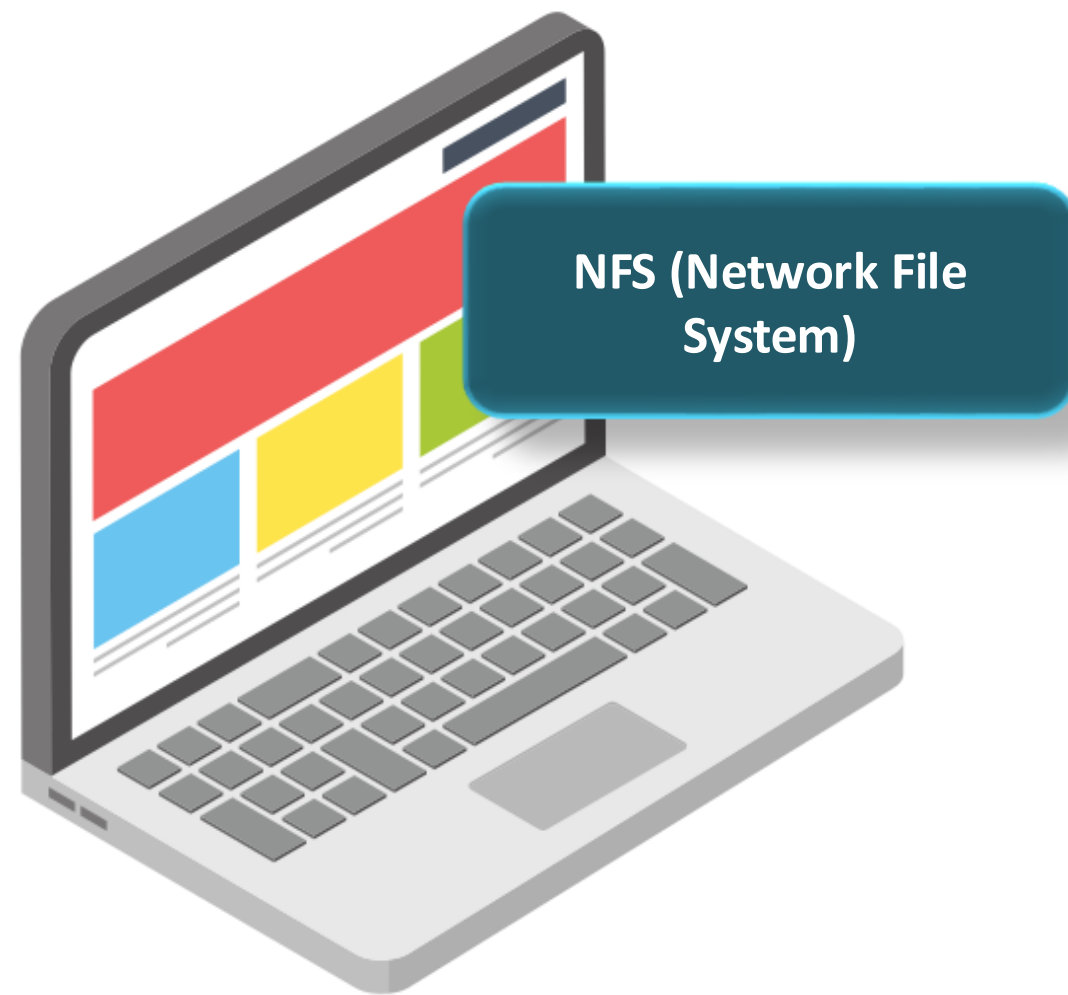


- ✓ Stored in RAM for ultra-fast access
- ✓ Used for temporary data that disappears after a reboot
- ✓ Commonly used for system directories like `/tmp` and `/var/run`



Filesystem Types

➤ Popular Filesystem Types



- ✓ Allows file sharing over a network
- ✓ Used in distributed environments where multiple systems need shared access
- ✓ Ideal for remote storage solutions



Filesystem Types

➤ **Choosing the Right Filesystem**

- ✓ Essential for optimizing performance and ensuring data integrity
- ✓ Helps meet specific use cases based on system requirements
- ✓ Understanding filesystems aids in effective disk partitioning
- ✓ Ensures the system is customized for its workload





Linux | Introduction to Disk Partitioning



Introduction to Disk Partitioning

Root Disk (Boot Disk): Default disk for OS, not ideal for storing application data

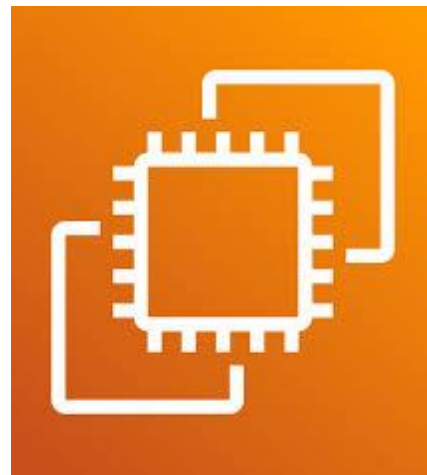
➤ **Impact of Storing Application Data on Root Disk:**

- ✓ Consumes significant disk space and I/O resources
- ✓ Can slow down OS and critical system processes
- ✓ Increases the risk of accidental modification or exposure of sensitive system files



Introduction to Disk Partitioning

➤ Setup



Amazon EC2



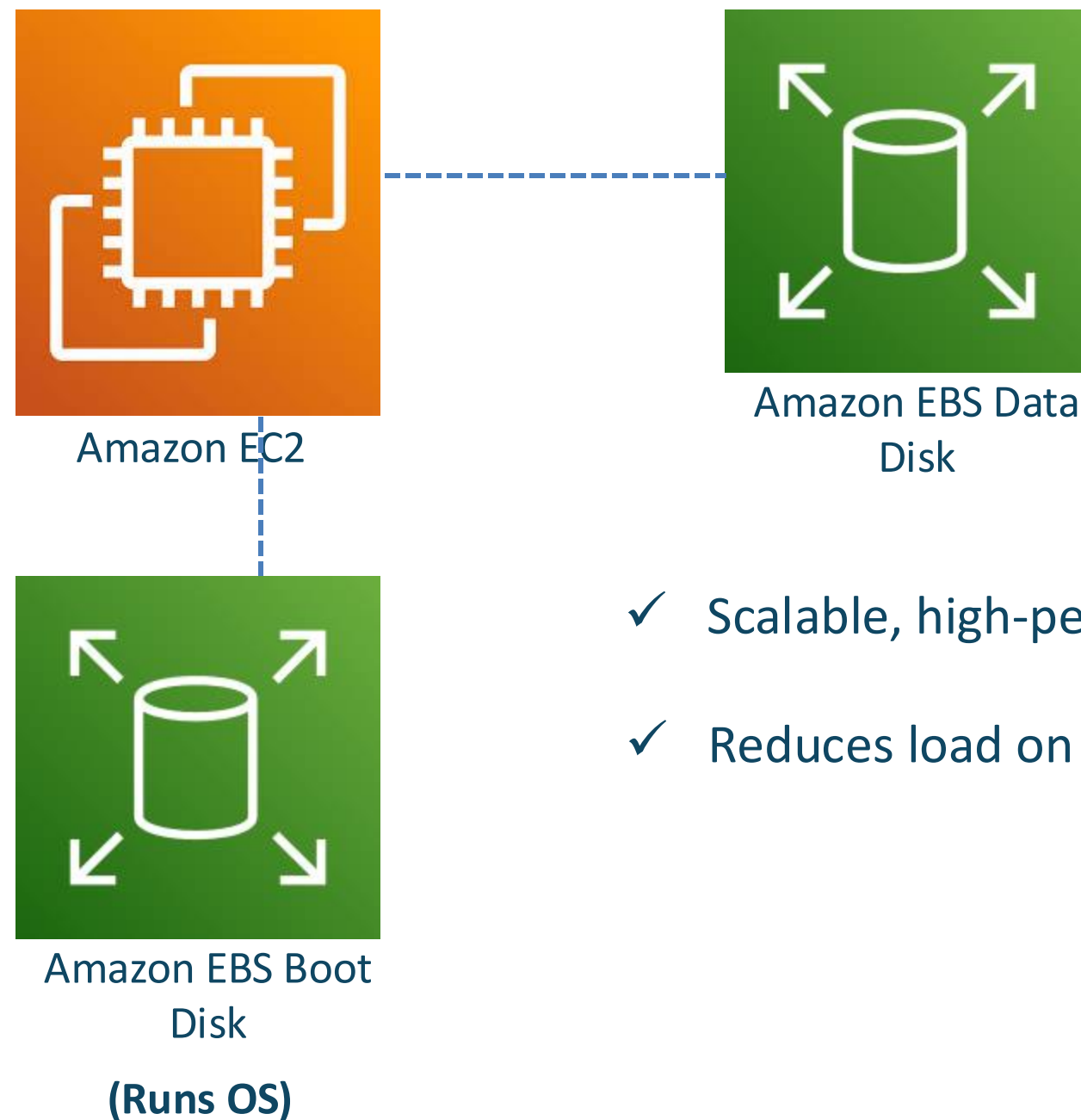
Amazon EBS Boot
Disk

(Runs OS)



Introduction to Disk Partitioning

➤ Optimizing Storage with AWS EBS



- ✓ Scalable, high-performance storage separate from the root disk
- ✓ Reduces load on the OS, enhancing performance and reliability



Introduction to Disk Partitioning

```
root@thinknyxlinux:~# ls -lrt /dev/xv*  
brw-rw---- 1 root disk 202,  0 Jan 29 09:18 /dev/xvda  
brw-rw---- 1 root disk 202,  1 Jan 29 09:18 /dev/xvda1  
brw-rw---- 1 root disk 202, 14 Jan 29 09:18 /dev/xvda14  
brw-rw---- 1 root disk 259,  0 Jan 29 09:18 /dev/xvda16  
brw-rw---- 1 root disk 202, 15 Jan 29 09:18 /dev/xvda15  
brw-rw---- 1 root disk 202, 112 Jan 29 09:29 /dev/xvdh
```



Introduction to Disk Partitioning

➤ Partitioning

- ✓ Divides a disk into smaller, logical sections
- ✓ Enhances organization and manageability

➤ Types of partitions

Partition

→ Primary (4)

→ Extended



Introduction to Disk Partitioning

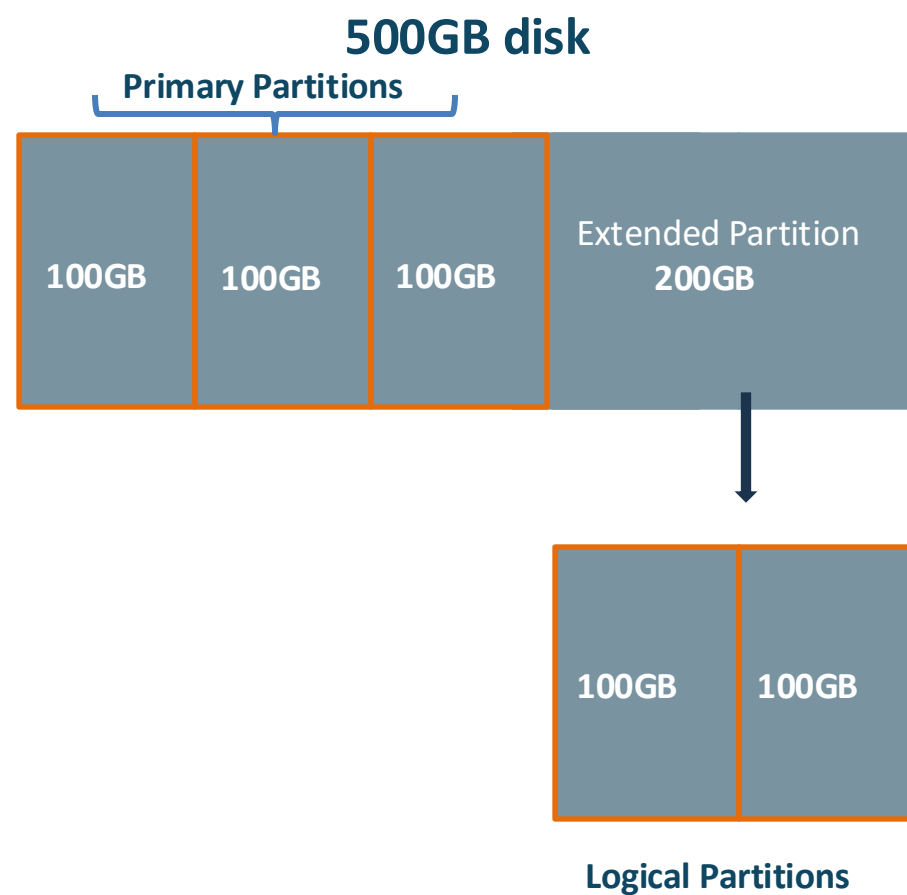
➤ Partitioning

500GB disk



Introduction to Disk Partitioning

➤ Partitioning



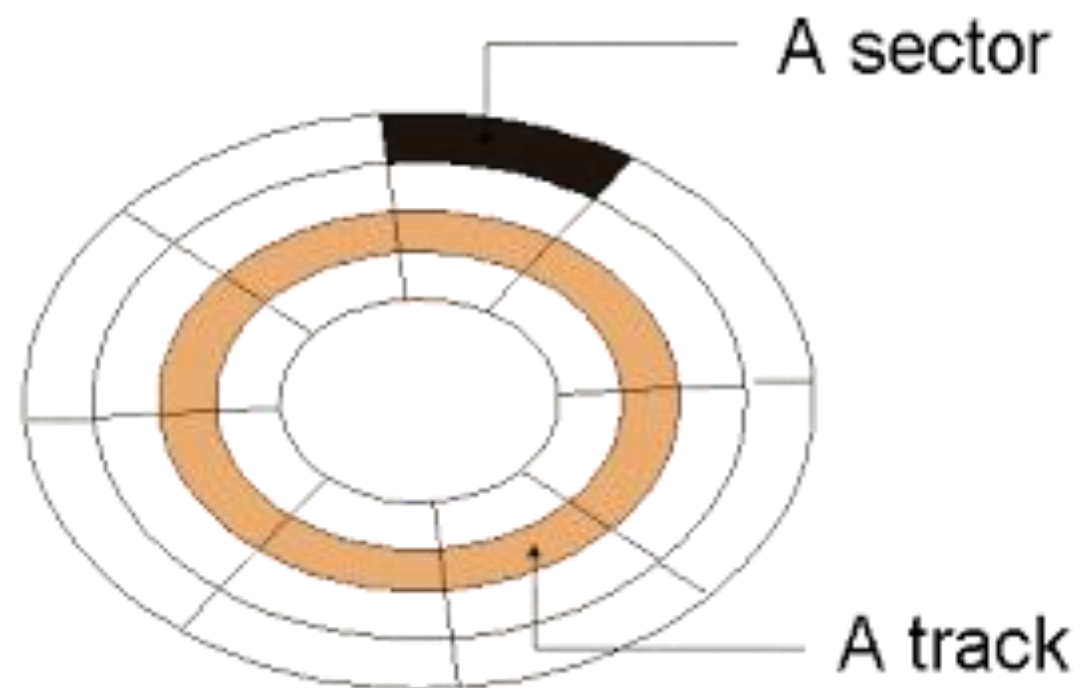
- ✓ GPT (GUID Partition Table)- No Limit
- ✓ MBR (Master Boot Record) - 15



Introduction to Disk Partitioning

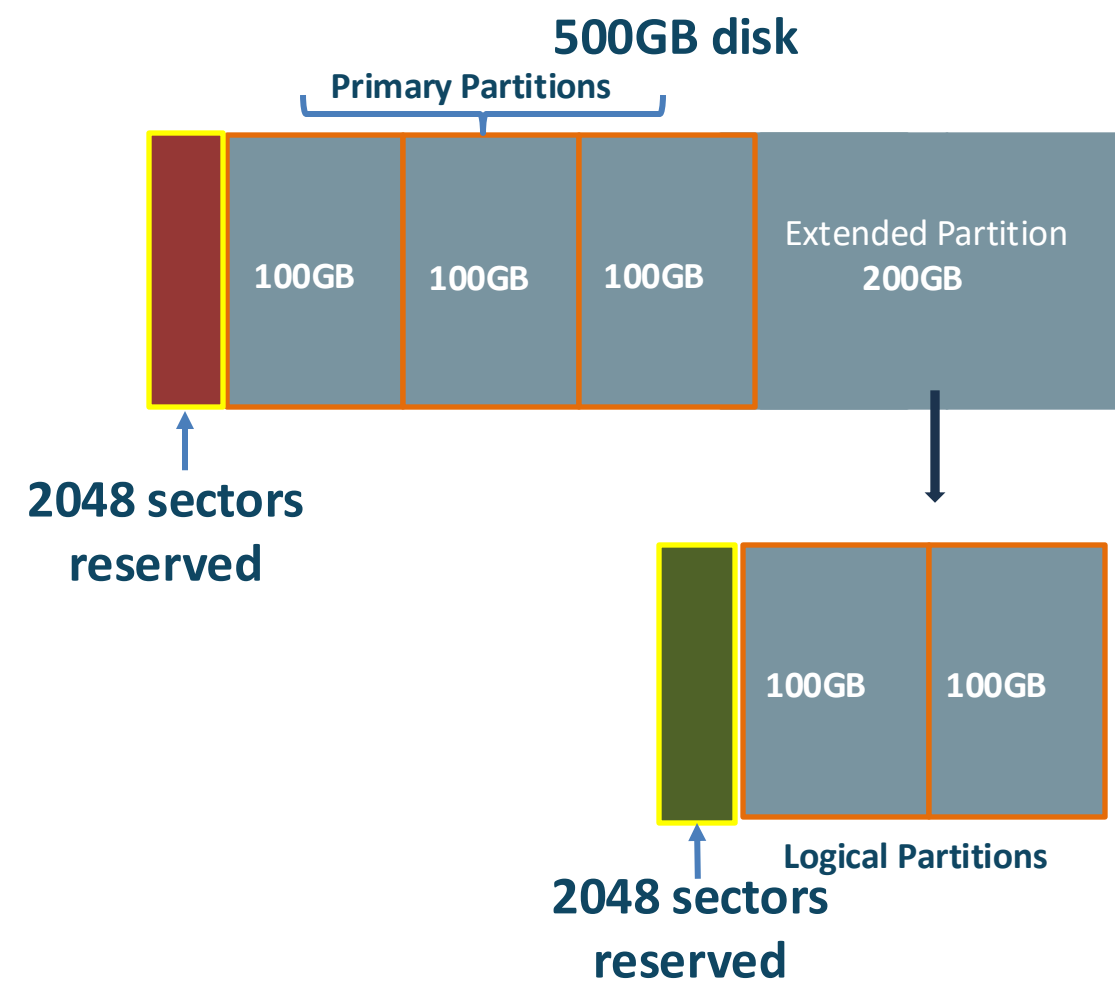
➤ Understanding Sectors

- ✓ Smallest unit of data storage on a disk (typically 512 bytes)
- ✓ Disk platters are divided into tracks, which are further divided into sectors



Introduction to Disk Partitioning

➤ Partitioning



Introduction to Disk Partitioning

➤ **Creating a Filesystem**

- ✓ Partitioning alone isn't enough; a filesystem is required
- ✓ Filesystems like ext4 and XFS enable reading and writing
- ✓ Formatting prepares the partition to store files and directories



Introduction to Disk Partitioning

➤ **Creating a Mount Point (Directory)**

- ✓ Create a directory as the mount point
- ✓ This directory serves as the access point for partition data



Introduction to Disk Partitioning

➤ Mounting the Filesystem

- ✓ Temporary Mounting: Uses the mount command; lost after a reboot
- ✓ Permanent Mounting: Updates /etc/fstab to ensure automatic mounting at boot



Introduction to Disk Partitioning

➤ **Unmounting and Decommissioning**

☐ **To securely decommission a disk**

- ✓ Delete the data
- ✓ Remove the partition table
- ✓ Reformat the disk



Introduction to Disk Partitioning

Key Steps Summary

Create an EBS Volume

Partition the Volume

Create a Filesystem

Create a Mount Point

Mount the Filesystem

Unmount and Decommission



Introduction to Disk Partitioning

➤ **Disk Management Commands**

Command	Description	Syntax
lsblk	Lists all block devices, including disks and partitions	lsblk
blkid	Displays UUIDs of disks and partitions	blkid
fdisk -l	Lists all disks and partitions (MBR partitioning)	fdisk -l
df -h	Displays disk space usage for mounted filesystems	df -h
fdisk /dev/sdX	Opens fdisk to create or manage partitions	fdisk /dev/sdX



Introduction to Disk Partitioning

➤ **Disk Management Commands**

Command	Description	Syntax
<code>mkfs.<filesystem> /dev/sdX1</code>	Creates filesystem on a partition	<code>mkfs.xfs /dev/sdX1</code>
<code>mount -a</code>	Mounts all filesystems listed in <code>/etc/fstab</code>	<code>mount -a</code>
<code>umount /mnt</code>	Unmounts the partition mounted at <code>/mnt</code>	<code>umount /mnt</code>





Demonstration | Disk Partitioning

Summary



- ❖ **Types of filesystems and their use cases**
- ❖ **Concepts and commands related to disk partitioning**
- ❖ **Demonstrated partitioning an EBS volume and creating a filesystem on it**





Thinknyx Technologies

Linux Course

Enhancing Strategy

Section - 14





Linux | Logical Volume Manager (LVM)





- LVM overview, key components, advantages, and essential commands
- Practical demonstration of the LVM lifecycle on AWS EC2



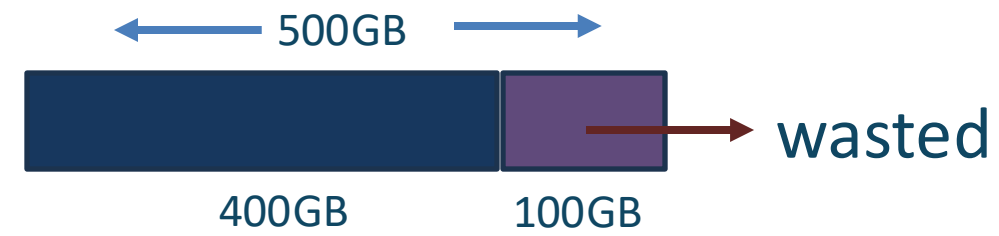
Linux | Introduction to LVM



LVM

➤ Traditional Disk Management & Its Limitations

- ✓ Involves attaching, formatting, partitioning, and creating filesystems
- ✓ Works well in static environments but has limitations in dynamic settings
- ✓ Unused disk space remains inaccessible (e.g., 500GB disk with 400GB used)



- ✓ Expanding the filesystem often requires reformatting, risking data loss and downtime



LVM

➤ Introduction to LVM

- ✓ LVM overcomes traditional disk management limitations
- ✓ Provides flexible and dynamic storage management
- ✓ Decouples physical storage from logical storage
- ✓ Allows resizing filesystems on the fly without downtime



LVM

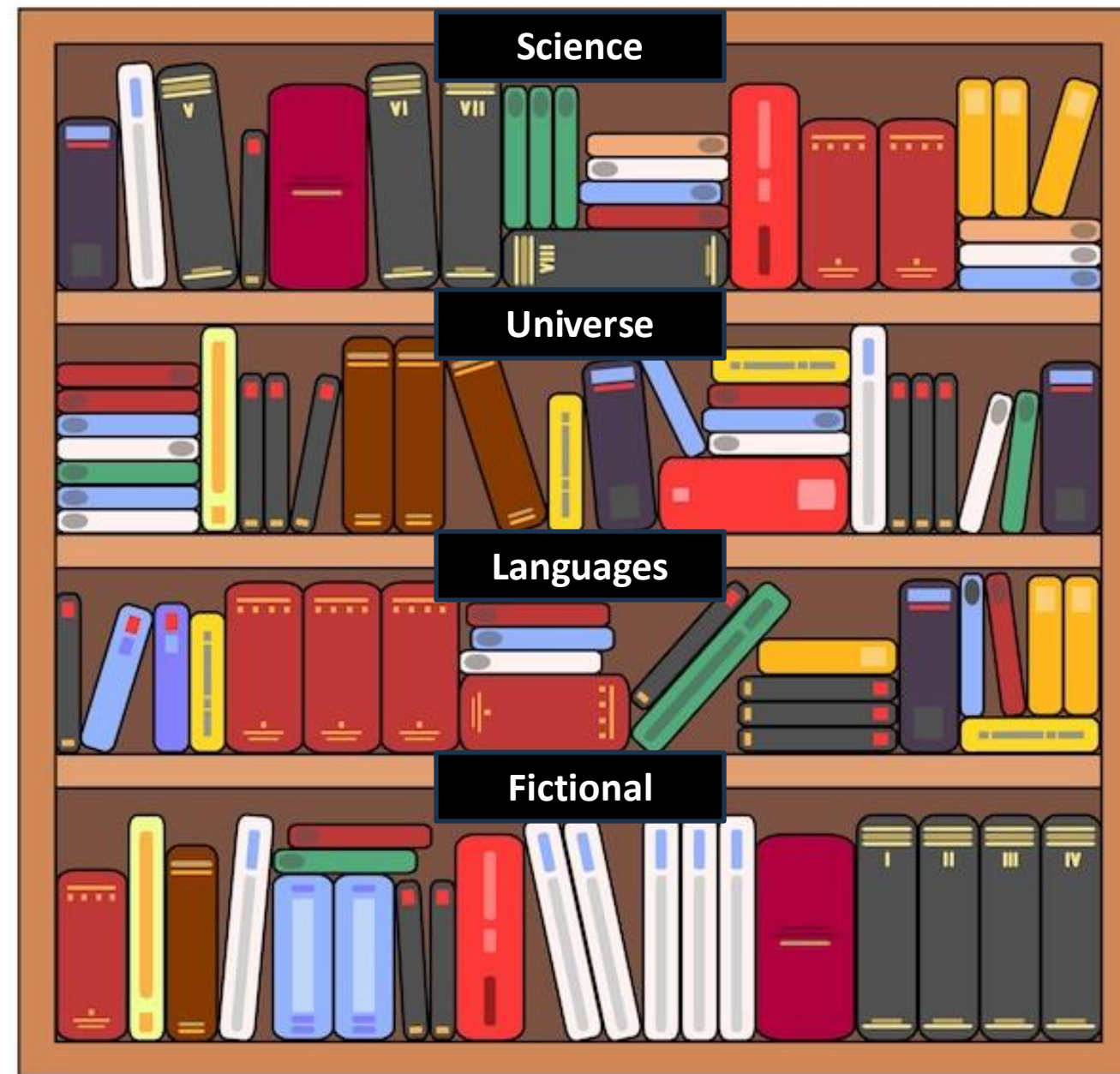


LVM

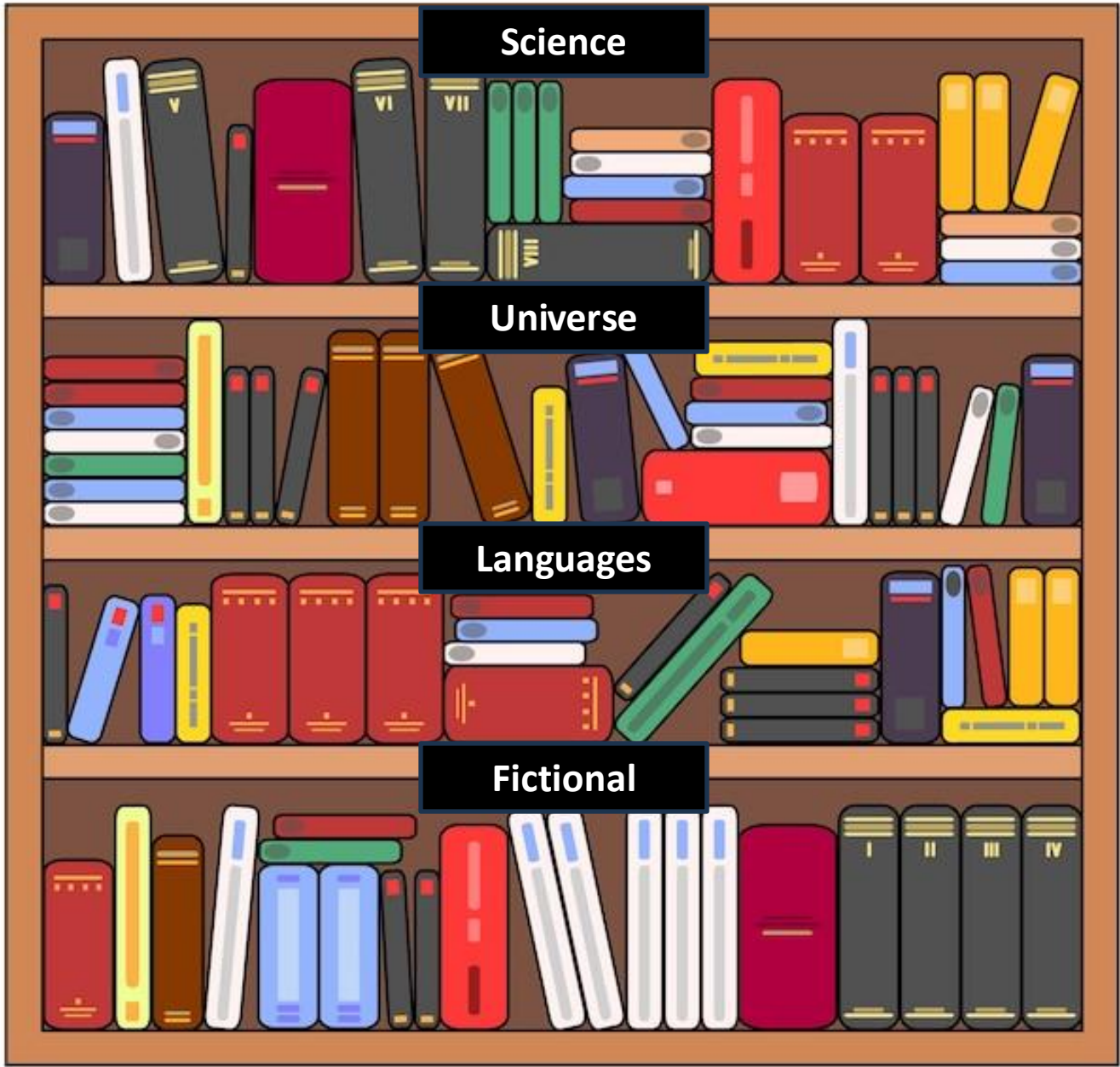
➤ Physical Shelves



➤ Section Labels



➤ Catalog Entries



Catalog		
1.	6.	11.
2.	7.	12.
3.	8.	13.
4.	9.	
5.	10.	

1.	6.	11.
2.	7.	12.
3.	8.	13.
4.	9.	
5.	10.	

1.	6.	11.
2.	7.	12.
3.	8.	13.
4.	9.	
5.	10.	

1.	7.	13.
2.	8.	14.
3.	9.	15.
4.	10.	
5.	11.	
6.	12.	

Side view



LVM

➤ Key Components of LVM

❑ Physical Volumes (PVs)

- ✓ Raw storage devices or disk partitions
- ✓ Initialized as PVs instead of direct disk usage
- ✓ Can be pooled with other PVs for better storage management
- ✓ Similar to individual shelves in a library

Physical Volume (PV)

Physical
Volume

Physical
Volume

Physical
Volume

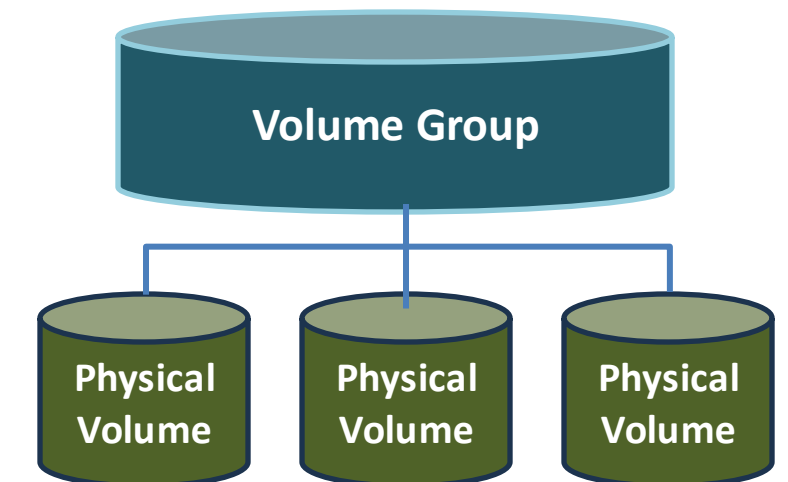
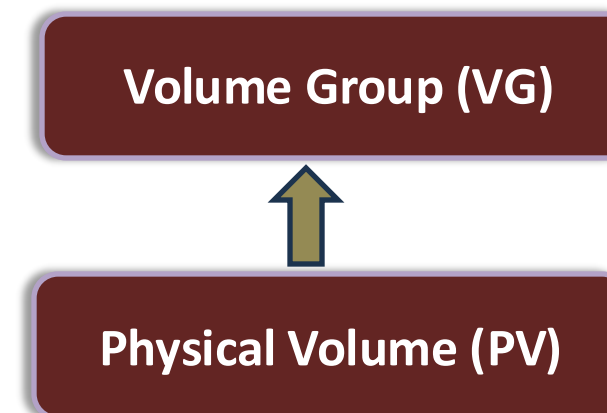


LVM

➤ Key Components of LVM

❑ Volume Groups (VGs)

- ✓ Collections of multiple Physical Volumes (PVs)
- ✓ Form a single, unified storage resource
- ✓ Similar to library sections organizing multiple shelves
- ✓ Enable efficient storage allocation based on needs (e.g., application data, databases)

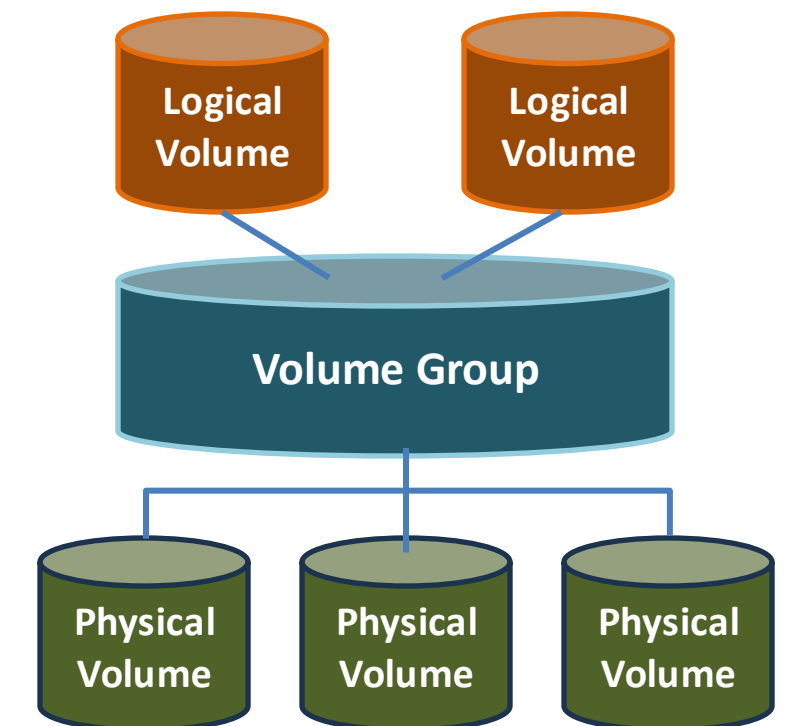
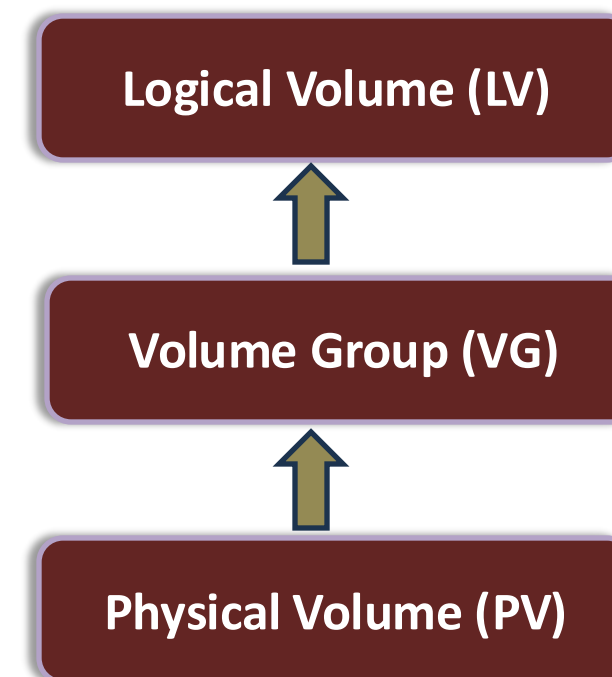


LVM

➤ Key Components of LVM

❑ Logical Volumes (LVs)

- ✓ Resizable virtual partitions within a Volume Group (VG)
- ✓ Similar to books cataloged in a library section
- ✓ Divide storage into manageable units
- ✓ Function like traditional partitions but can be expanded or reduced dynamically

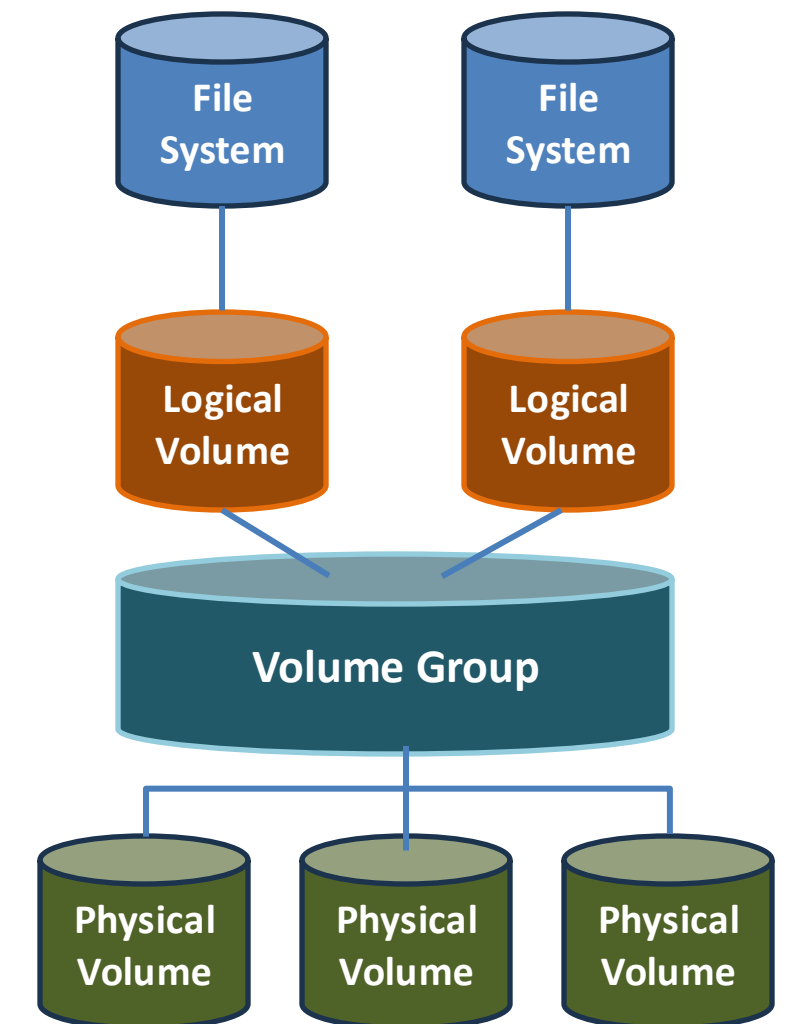
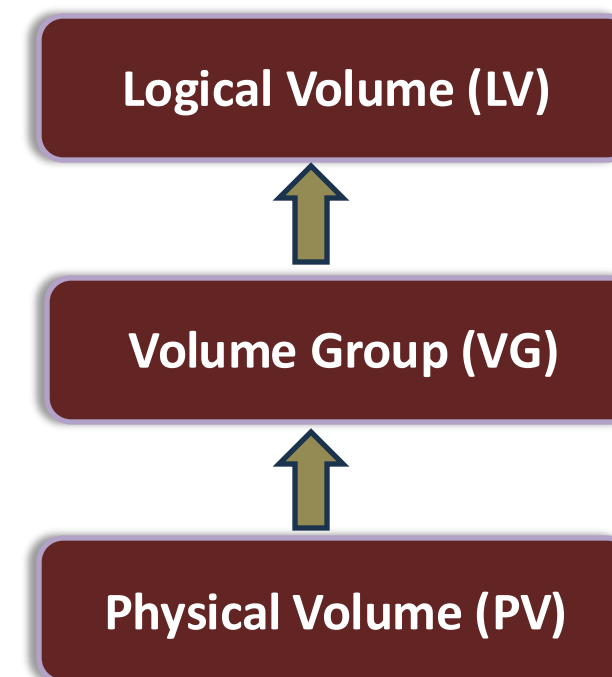


LVM

➤ Key Components of LVM

❑ Key Advantage of LVM

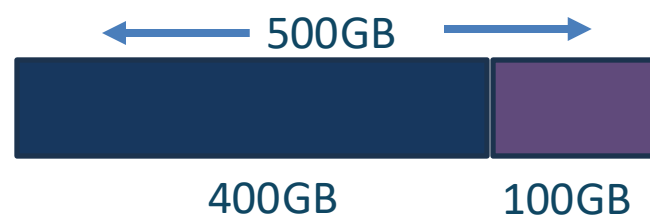
- ✓ Filesystem resides on top of the Logical Volume (LV)
- ✓ Allows on-the-fly resizing without downtime
- ✓ Supports expanding or shrinking storage as needed
- ✓ Ideal for production environments requiring uninterrupted access



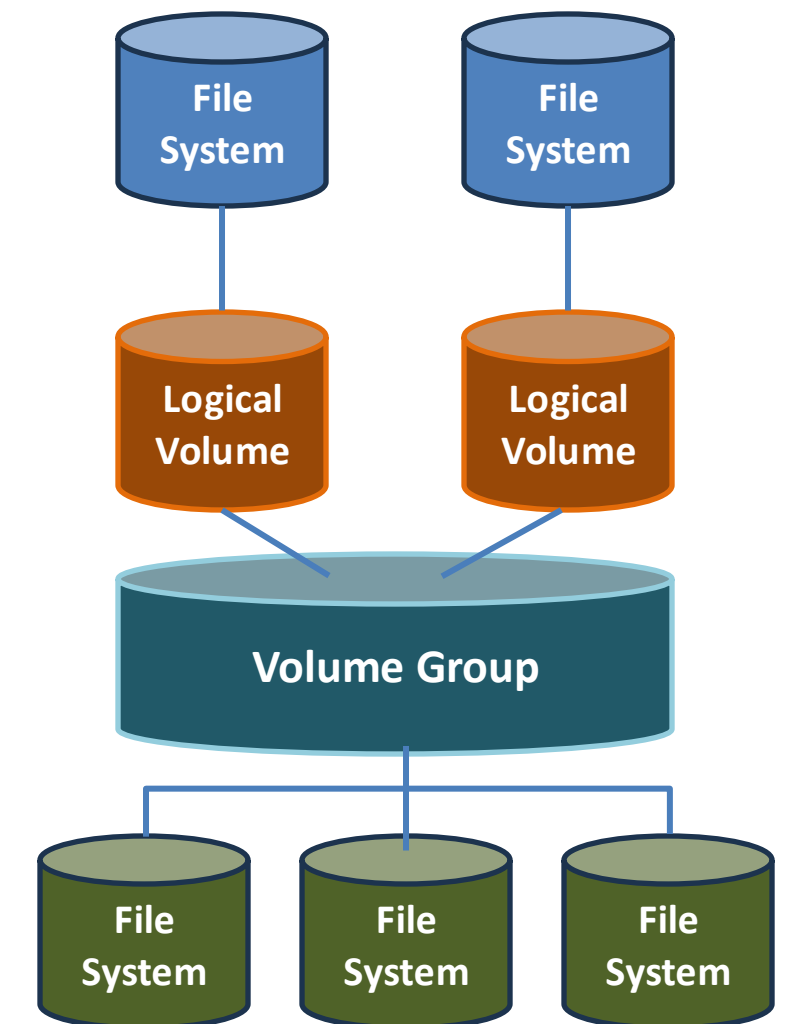
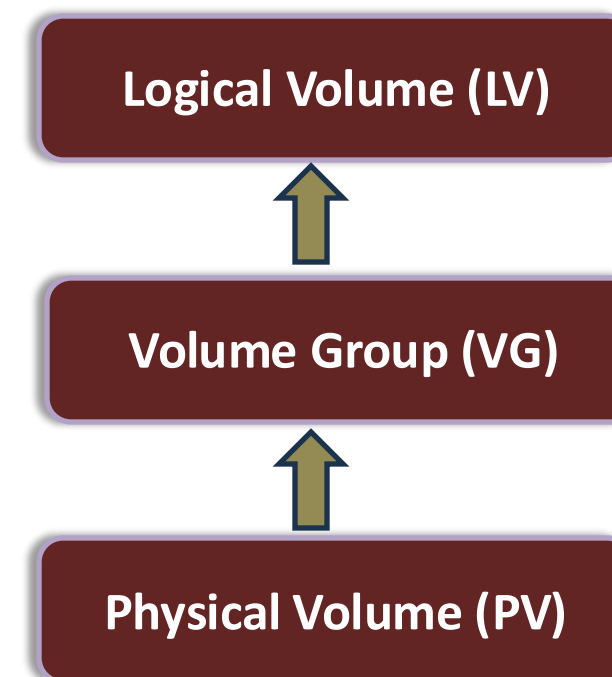
LVM

➤ Implementing LVM for Dynamic Storage Management

- ✓ Involves attaching, formatting, partitioning, and creating filesystems
- ✓ Works well in static environments but has limitations in dynamic settings
- ✓ Unused disk space remains inaccessible (e.g., 500GB disk with 400GB used)



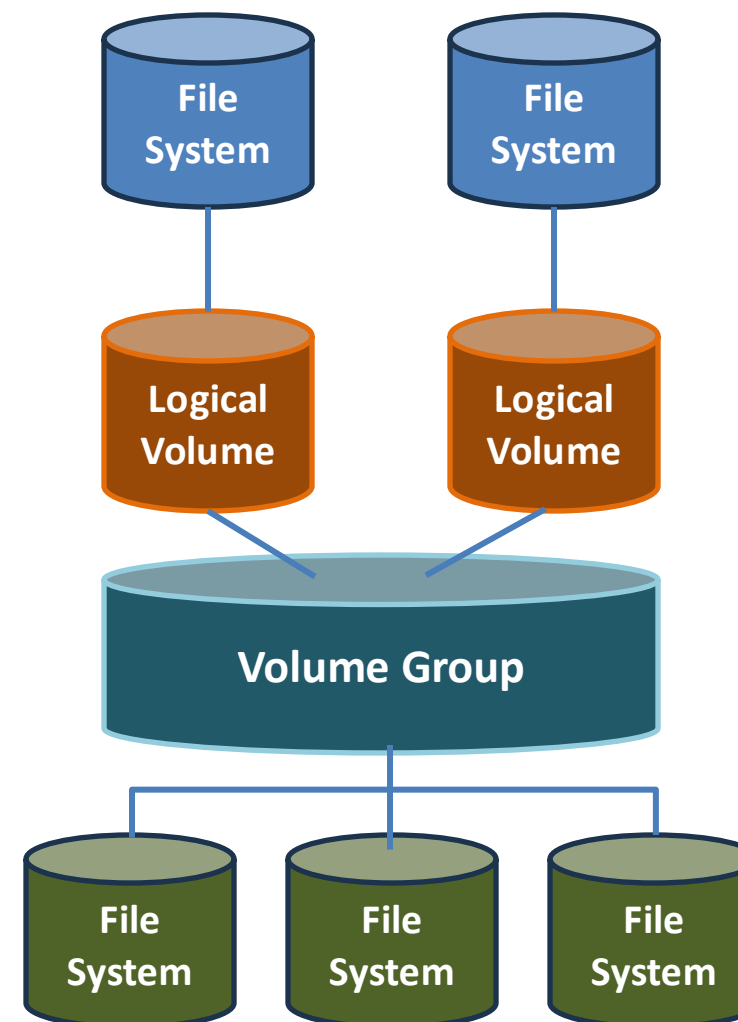
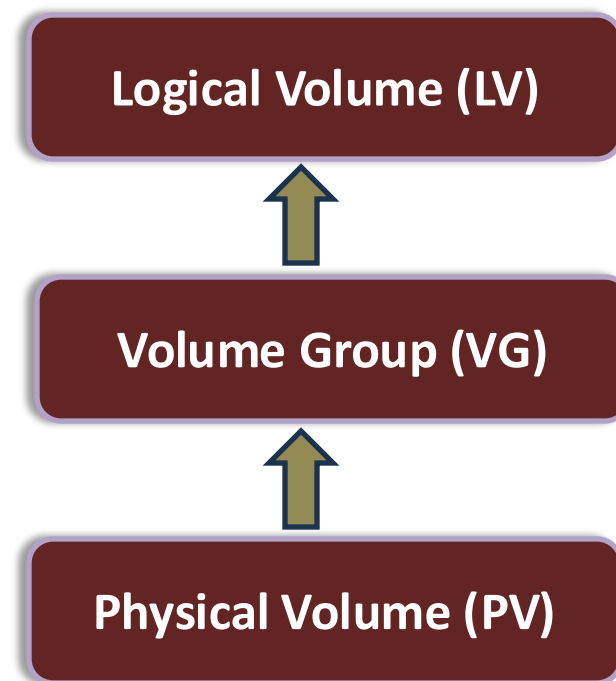
- ✓ Expanding the filesystem often requires reformatting, risking data loss and downtime



LVM

➤ Key Components of LVM

500GB



LVM

➤ LVM offers several advantages

Flexible Capacity

- ✓ Combines multiple devices into one logical volume
- ✓ Allows filesystems to span multiple devices

Resizable Storage Volumes

- ✓ Resize Logical Volumes easily without reformatting
- ✓ Adjust storage on demand

Online Data Relocation

- ✓ Move data between devices within a Volume Group
- ✓ Perform maintenance or upgrades without downtime



LVM

➤ LVM offers several advantages

Snapshot Support

- ✓ Create read-only or read-write copies of a Logical Volume
- ✓ Useful for backups and testing
- ✓ Capture a consistent state without disrupting operations

Improved Disk Utilization

- ✓ Reduces wasted space by pooling storage resources
- ✓ Logical Volumes can be resized as needed for better efficiency



LVM

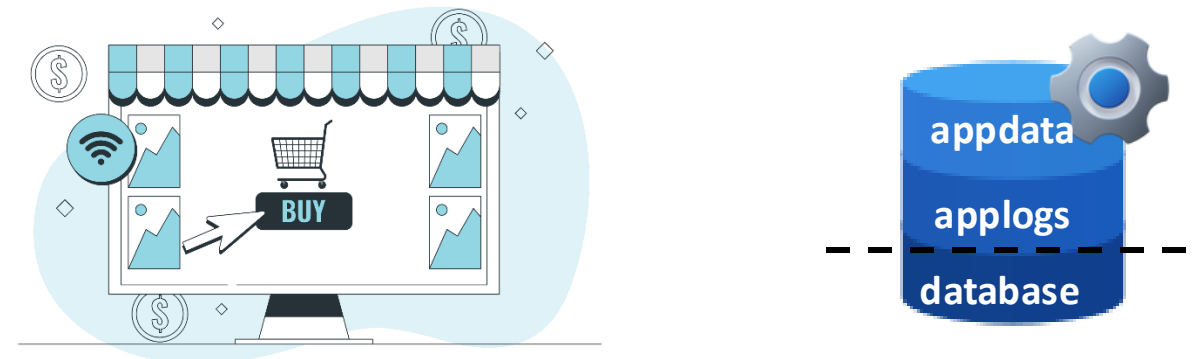
➤ LVM2 Enhancements

- ✓ Improved metadata storage and recovery formats
- ✓ Increased flexibility and reliability in storage management



LVM

➤ Real-world scenario



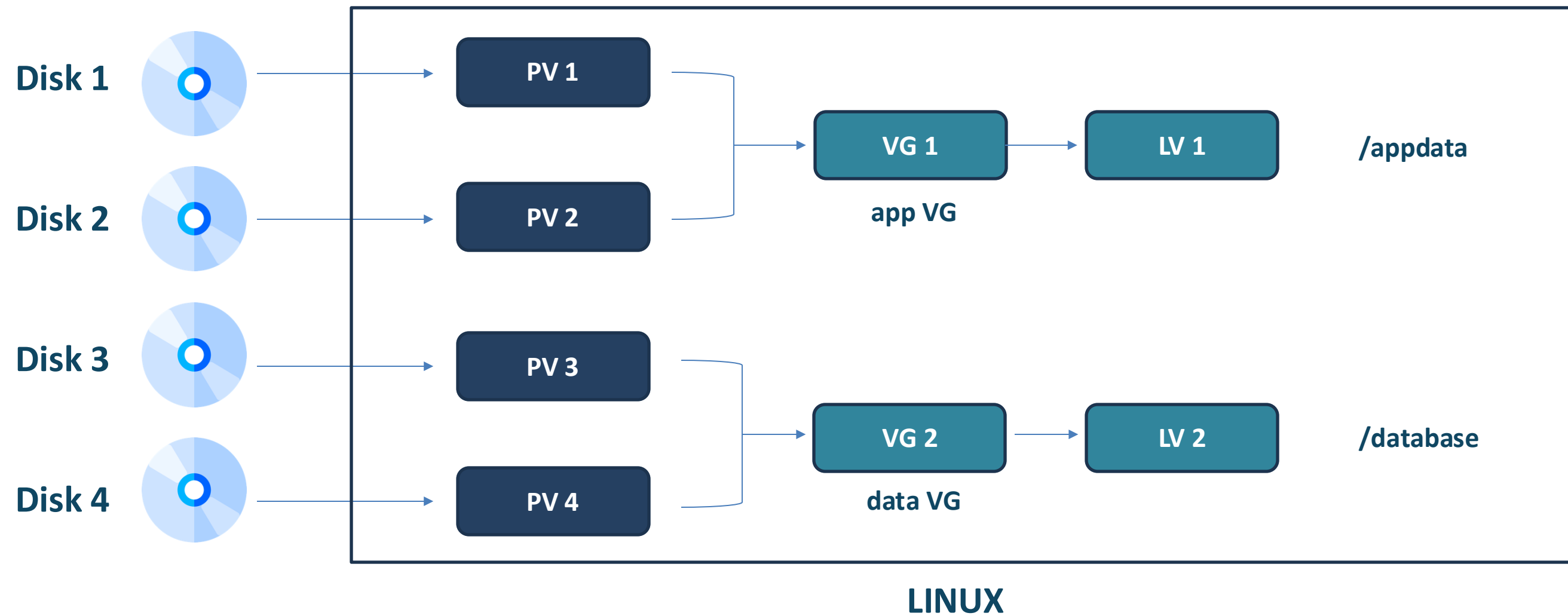
➤ Limitations of Traditional Partitioning

- ✓ Fixed storage allocation can lead to imbalances
- ✓ Expanding partitions may require reformatting or migration
- ✓ Risk of downtime and potential data loss



LVM

➤ LVM for Scalable Storage



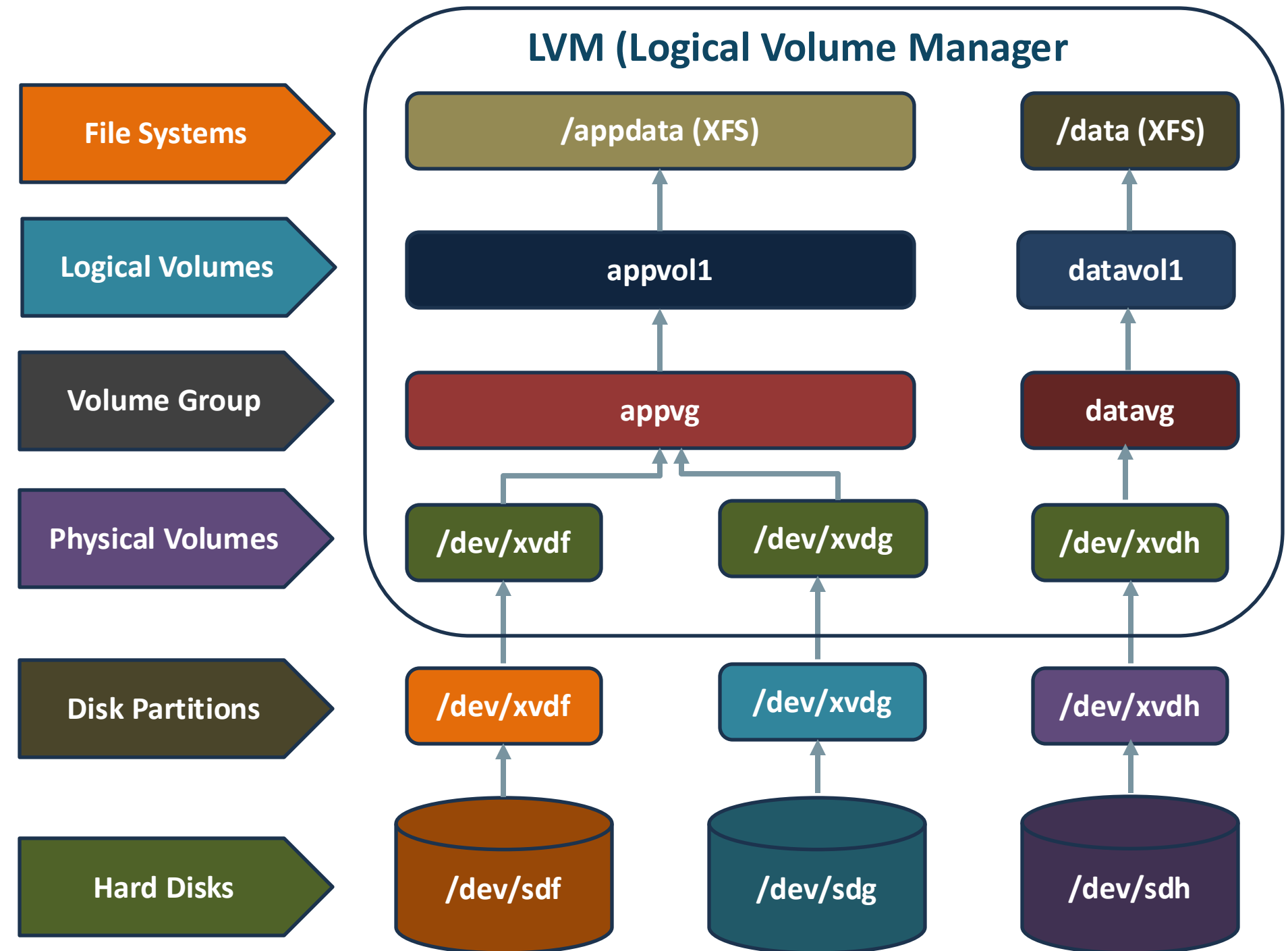
LVM

➤ Implementing LVM for Dynamic Storage Management

- ✓ Attaching Multiple Disks to an EC2 Instance
- ✓ Initializing Physical Volumes (PVs)
- ✓ Creating a Volume Group (VG)
- ✓ Creating Logical Volumes (LVs)
- ✓ Formatting LVs with a Filesystem
- ✓ Creating Directories as Mount Points
- ✓ Mounting Filesystems
- ✓ Decommissioning and Cleanup



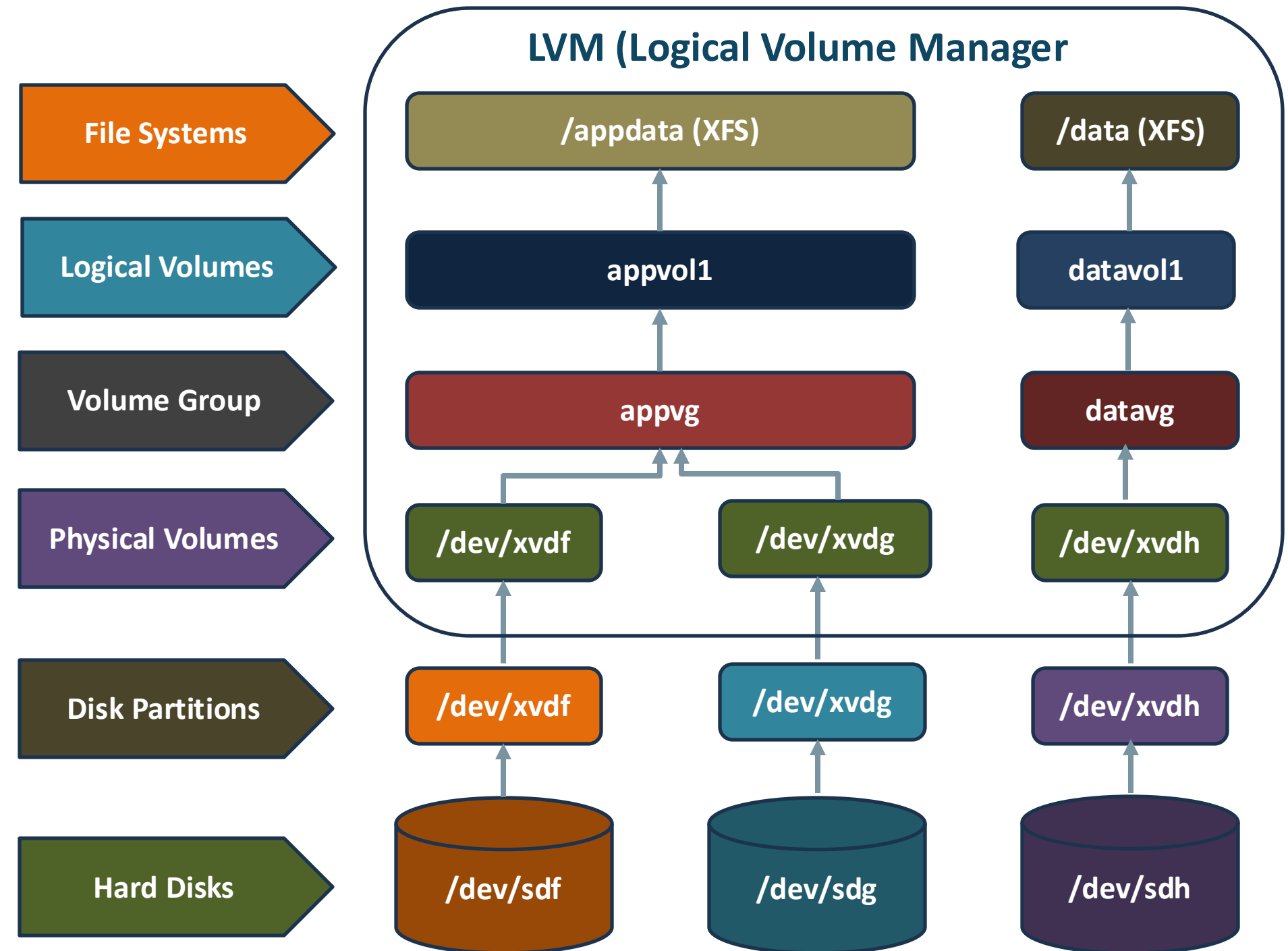
mkdirt



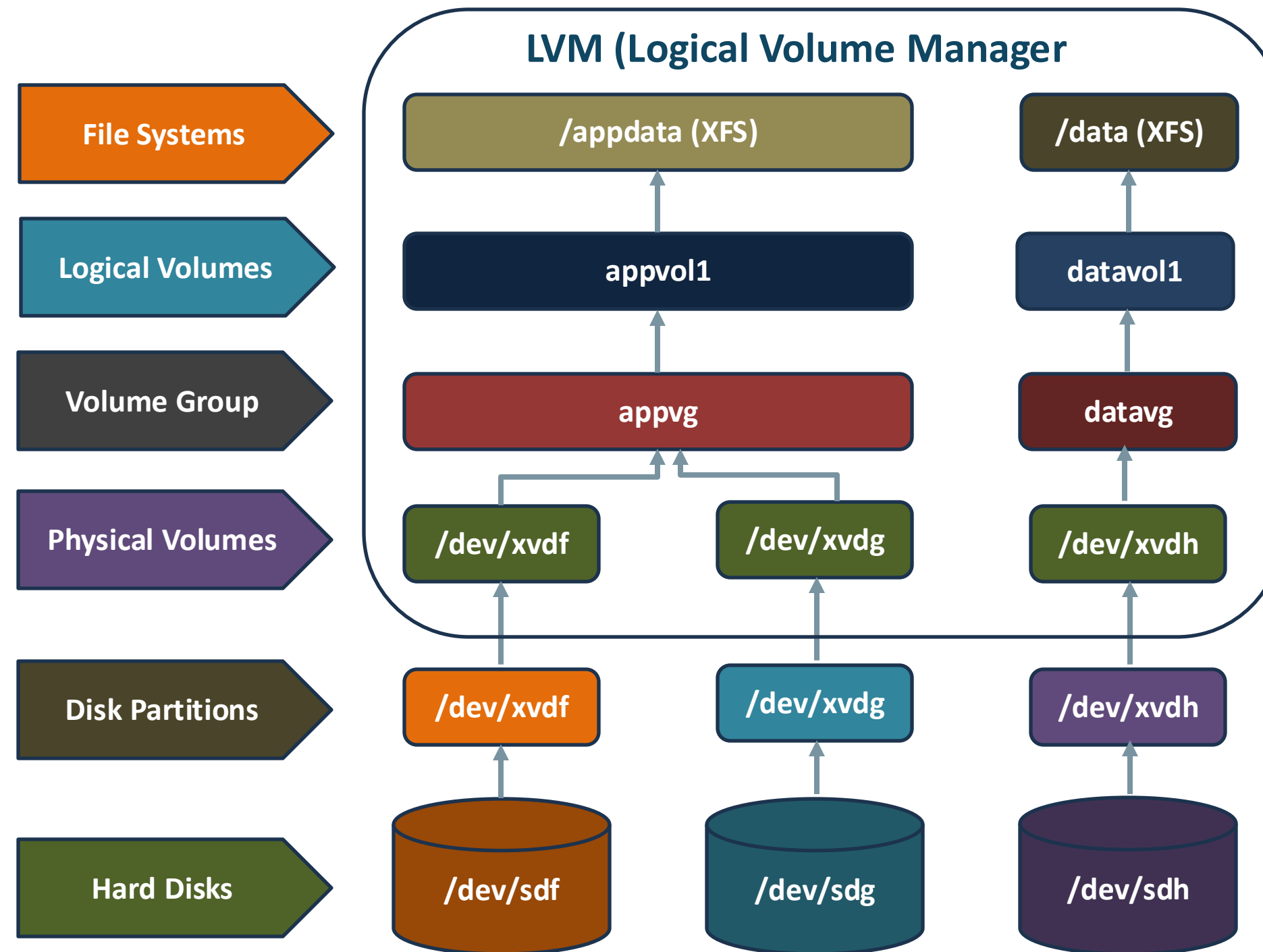
LVM

➤ Implementing LVM for Dynamic Storage Management

- ✓ Attaching Multiple Disks to an EC2 Instance
- ✓ Initializing Physical Volumes (PVs)
- ✓ Creating a Volume Group (VG)
- ✓ Creating Logical Volumes (LVs)
- ✓ Formatting LVs with a Filesystem
- ✓ Creating Directories as Mount Points
- ✓ Mounting Filesystems
- ✓ Decommissioning and Cleanup



LVM



LVM

➤ LVM Management Commands

☐ Physical Volume (PV) Management

Command	Description	Syntax
pvcreate	Initializes a disk as a physical volume	System is unusable
pvdisplay	Displays information about a physical volume	pvdisplay /dev/xvdf
pvs	Displays a summary of physical volumes	pvs
pvscan	Scans all disks for physical volumes	pvscan
pvremove	Removes a physical volume	pvremove /dev/xvdf



LVM

➤ LVM Management Commands

❑ Volume Group (VG) Management

Command	Description	Syntax
vgcreate	Creates a volume group from one or more physical volumes	vgcreate datavg /dev/xvdf
vgextend	Adds a new physical volume to an existing volume group	vgextend appvg /dev/xvdg
vgdisplay	Displays detailed information about a volume group	vgdisplay datavg
vgs	Displays a summary of volume groups	vgs
vgscan	Scans all disks for volume groups and detects new ones	vgscan
vgreduce	Removes a physical volume from a volume group	vgreduce datavg /dev/xvdf
vgremove	Removes a volume group	vgremove datavg



LVM

➤ LVM Management Commands

❑ Logical Volume (LV) Management

Command	Description	Syntax
lvmdiskscan	Scans for block devices that can be used as physical volumes	lvmdiskscan
lvcreate	Creates a logical volume within a volume group	lvcreate -l 100%FREE -n datavol1 datavg
lvextend	Extends the size of a logical volume	lvextend -L +6G /dev/appvg/appvol1
lvreduce	Reduces the size of a logical volume (use with caution)	lvreduce -L -5G
lvdisplay	Displays detailed information about a logical volume	lvdisplay /dev/datavg/datavol1
lvs	Displays a summary of logical volumes	lvs
lvremove	Removes a logical volume	lvremove /dev/datavg/datavol1



LVM

➤ LVM Management Commands

❑ Filesystem Management

Command	Description	Syntax
xfs_growfs	Expands a mounted XFS filesystem after resizing a logical volume	xfs_growfs /appdata
resize2fs	Expands a mounted ext2/ext3/ext4 filesystem after resizing a logical volume	resize2fs /dev/appvg/appvol1





Demonstration | LVM

Summary



- ❖ **LVM overview:** components, advantages, and commands
- ❖ **Demonstrated LVM lifecycle on AWS EC2**
- ❖ **Created, extended, and resized volumes**
- ❖ **Decommissioned EBS volumes**





Thinknyx Technologies

Linux Course

Enhancing Strategy

Section - 16





Linux | Introduction to SSH



SSH



- SSH is and its key components such as ssh, sshd, and scp
- SSH configuration file (sshd_config)
- Brief of DHCP (Dynamic Host Configuration Protocol)
- Demonstration to set up passwordless authentication



Linux | SSH Client/Server



SSH Client/Server

➤ SSH

- ✓ Secure Shell
- ✓ Protocol used for securely connecting to remote systems over a network
- ✓ Secure, encrypted channel between a local machine and a remote machine

➤ Telnet and FTP transmitted data

- Including passwords



➤ SSH

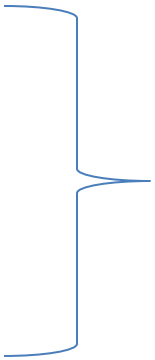
- ✓ Uses encryption to ensure that the communication between the client and server is secure



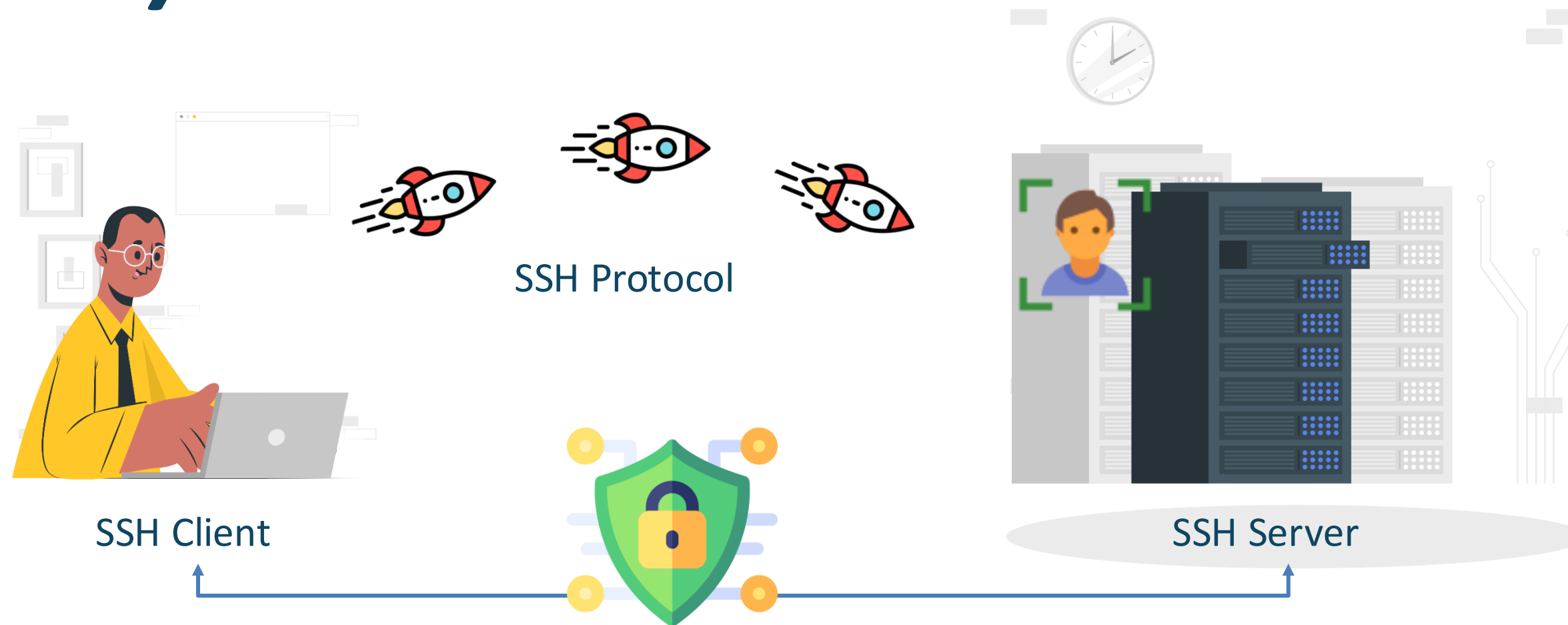
SSH Client/Server



Cloud Server
Personal Machine
Production System



SSH Client/Server



- ✓ The SSH client connects to the remote machine via port 22
- ✓ The remote server's SSH daemon (sshd) listens on this port and handles incoming connections
- ✓ Once authenticated (via password or key-based authentication), the server grants access to the client



SSH Client/Server

➤ Connect to Remote Host

```
ssh -i "thinknyx.pem" ubuntu@13.233.117.155
```

➤ Private Key and Public Key Authentication

- ✓ A pair of cryptographic keys
 - A private key and a public key
- ✓ Private key is kept on the client machine
- ✓ Public key is stored on the server in the `authorized_keys` file
- ✓ Public key on the server matches the private key
- ✓ Connection is authenticated without needing a password



SSH Client/Server

➤ **sshd (SSH Daemon)**

✓ OpenSSH server process responsible for handling incoming connections via the SSH protocol

- User authentication
- Encryption
- Terminal sessions
- File transfers
- Tunneling

➤ **sshd (SSH Daemon)**

- ✓ `sudo systemctl stop ssh` # To Stop the SSH service
- ✓ `sudo systemctl start ssh` # To Start the SSH service
- ✓ `sudo systemctl restart ssh` # Restart the SSH service after configuration changes
- ✓ `sudo systemctl reload ssh` # Without stopping just Reload the SSH service after configuration changes



SSH Client/Server

➤ Configuration Files for SSH

- ✓ sshd_config (the server-side configuration file)
- ✓ authorized_keys (where public keys are stored)

❑ /etc/ssh/sshd_config

- ✓ Primary configuration file for the SSH daemon



```
Include /etc/ssh/sshd_config.d/*.conf

#Port 22
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
#SyslogFacility AUTH
#LogLevel INFO

# Authentication:

#LoginGraceTime 2m
#PermitRootLogin prohibit-password
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10
```

SSH Client/Server

➤ Configuration Files for SSH

- ✓ sshd_config (the server-side configuration file)
- ✓ authorized_keys (where public keys are stored)

❑ /etc/ssh/sshd_config

- ✓ Primary configuration file for the SSH daemon

❑ ~/.ssh/authorized_keys

- ✓ File holds the public keys of clients that are authorized to access the server





Demonstration | Password Authentication



Demonstration | Key-Based Authentication

SSH Client/Server

➤ Introduction to SCP (Secure Copy)

```
scp [options] <path_of_file/filename>remoteuser@ip_address_remote_server:~/path_of_destination_directoty/
```

❑ Some of the common options:

- ✓ -r for Recursive copy of entire directories
- ✓ -i for identity file



SSH Client/Server

➤ Steps followed for key based authentication:

- ✓ Modify SSH settings in the sshd_config file on the VM, to enable key based authentication
- ✓ Create a new Ubuntu VM (ubuntu_2) with the same configuration as the first VM
- ✓ Generate SSH Key Pair(public and private keys) Locally
- ✓ Transfer the Private Key to the Remote Server ubuntu_1 using scp command
- ✓ Set the minimum permissions on the private key file on the remote server using chmod
- ✓ Transfer the Public Key to both the Remote Servers, ubuntu_1 and ubuntu_2
- ✓ Now to test SSH Connections:
 - SSH into ubuntu_1 using the private key
 - SSH into ubuntu_2, without a key and password, to ensure key-based authentication works





Linux | DHCP



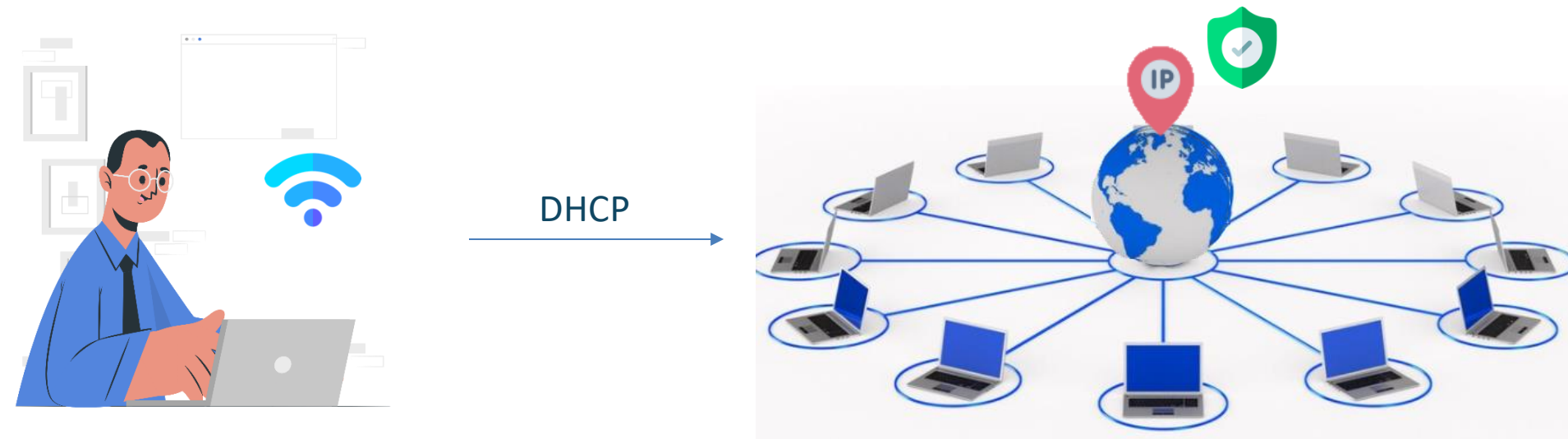
DHCP

➤ DHCP (Dynamic Host Configuration Protocol)

- ✓ Network protocol automatically assign IP addresses to devices on network
- ✓ Allows devices to receive IP address dynamically from DHCP server

❑ Role of DHCP

- ✓ Eliminates the need to assign IP addresses to each device manually
- ✓ Each device gets unique IP address



DHCP

➤ Checking IP Configuration

✓ ip a

- IP address (inet)
- MAC address
- Configuration information

✓ Interface

- enx0
- wlan0

```
ubuntu@ip-172-31-4-2:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enx0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc fq_codel state UP group default qlen 1000
    link/ether 0a:08:1b:b7:c7:19 brd ff:ff:ff:ff:ff:ff
    inet 172.31.4.2/20 metric 100 brd 172.31.15.255 scope global dynamic enx0
        valid_lft 3319sec preferred_lft 3319sec
    inet6 fe80::808:1bff:feb7:c719/64 scope link
        valid_lft forever preferred_lft forever
```



DHCP

➤ Important Components of DHCP

- ✓ **DHCP Server** – Assigns IP addresses and network configurations to clients
- ✓ **DHCP Client** – Device that requests an IP address from the DHCP server
- ✓ **IP Address Pool** – Range of available IP addresses that DHCP server can allocate to clients
- ✓ **Lease Time** – Duration for which an IP address is assigned to client before it needs renewal



DHCP

➤ Systemd-networkd

- ✓ System service used to manage network configurations on Linux systems
- ✓ Handles network interfaces, automatically assign IP addresses via DHCP

➤ Netplan

- ✓ Default tool used for network configuration on Ubuntu 24.04 and later
- ✓ Define network settings using YAML configuration files
- ✓ Works alongside systemd-networkd
- ✓ Network setting - **/etc/netplan/**



DHCP

➤ **Why use dhclient?**

- ✓ Manually request, release, or renew IP addresses
- ✓ Useful in troubleshooting, demonstrations, configuring network interfaces manually



Summary



- ❖ SSH (Secure Shell)
- ❖ SSH client-server model and key-based authentication
- ❖ sshd daemon
- ❖ Key configuration files - sshd_config and authorized_keys
- ❖ Demonstration - setting up password and key-based authentication
- ❖ DHCP





Thinknyx Technologies

Linux Course

Enhancing Strategy

Section - 17





Linux | Introduction to System Management



System Management



- System Management and its key concepts
 - Scheduler
 - Logging Mechanisms
 - Alerting Systems
 - Memory Management



Linux | Scheduler



Scheduler

➤ Scheduler



➤ Popular Tools

Cron

Anacron

At

Batch



Scheduler

➤ Cron

- ✓ Time-based job scheduler
- ✓ Task known as 'cron jobs'
- ✓ Crontab
 - List of commands
 - Execution time



Scheduler

➤ Understanding Cron's Special Strings

Special Strings	Equivalent to	Execution Result
@reboot	-	Executed once at system startup
@yearly or @annually	0 0 1 1 *	Once a year at midnight on January 1st
@monthly	0 0 1 * *	Once a month at midnight on the first day of the month
@weekly	0 0 * * 0	Once a week at midnight on Sunday
@daily or @midnight	0 0 * * *	Once a day at midnight
@hourly	0 * * * *	Once an hour at the beginning of the hour



Scheduler

➤ For Example:

```
30 2 * * * ~/backup_script.sh
```

Understanding this structure allows you to schedule tasks with precision, ensuring they run exactly when needed

30 Every 30th Minute
2 AM
* Every Applicable Value



Scheduler

- ✓ Manage and monitor scheduled jobs

crontab -l

[To list current cronjob]

crontab -e

[To edit cronjob]

crontab -r

[To remove crontab]



Scheduler

➤ **Anacron**

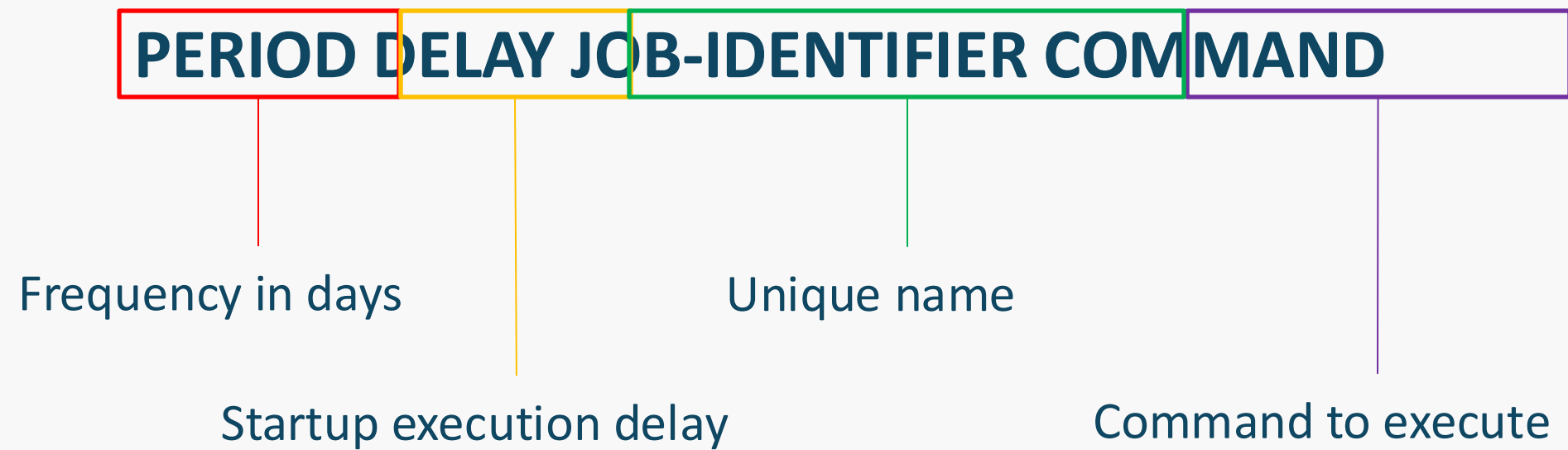
- ✓ Systems that may not run 24/7
- ✓ Periodically running tasks
- ✓ System is turned off for some time



Scheduler

➤ Syntax

✓ /etc/anacrontab



✓ **NOTE: /etc/anacrontab is not found by default in ubuntu 24.04 LTS**



Scheduler

➤ Installation

- ✓ `sudo apt install anacron`
- ✓ `cat /etc/anacrontab`

```
ubuntu@ip-172-31-1-192:/etc$ cat /etc/anacrontab
# /etc/anacrontab: configuration file for anacron

# See anacron(8) and anacrontab(5) for details.

SHELL=/bin/sh
HOME=/root
LOGNAME=root

# These replace cron's entries
1      5      cron.daily      run-parts --report /etc/cron.daily
7      10     cron.weekly    run-parts --report /etc/cron.weekly
@monthly 15     cron.monthly  run-parts --report /etc/cron.monthly
ubuntu@ip-172-31-1-192:/etc$ |
```



Scheduler

✓ /etc/anacrontab

```
1 5 log-cleanup find /root/scripts/logs -type f -name "*.log" -mtime +7 -exec rm {} \;
```

- ✓ Anacron doesn't provide direct commands
- ✓ Manage by editing the /etc/anacrontab file



Scheduler

➤ Listing & Deleting Jobs

✓ `sudo nano /etc/anacrontab`

➤ Use Case

✓ Periodic maintenance tasks

- System Backups

- System Updates



Scheduler

➤ **AT**

- ✓ Schedule a one-time task to run at a specific time

➤ **Syntax**

at TIME [Time – specifies when the command should run]

➤ **Related Commands**

Commands	Execution Result
atq	List Scheduled Jobs
atrm [job_number] or at -r [job_number]	Remove Scheduled Jobs



Scheduler

➤ Batch

- ✓ Schedules tasks when the system load is low

➤ Syntax - BATCH

➤ For Example - cleanup_script1.sh located in /root/scripts/cleanup_script1.sh

```
root@ip-172-31-1-192:~# batch
warning: commands will be executed using /bin/sh
at Tue Feb  4 08:29:00 2025
at> bash /root/scripts/cleanup_script1.sh
at> <EOT>
job 8 at Tue Feb  4 08:29:00 2025
```



Scheduler

➤ Listing & Deleting Scheduled Batch Jobs

```
root@ip-172-31-1-192:~# at -l  
8      Tue Feb  4 08:29:00 2025 b root  
root@ip-172-31-1-192:~#
```

atrm [job_id] ---- Deleting the scheduled batch jobs

➤ Use Case

- ✓ Data Backup
- ✓ System Updates
- ✓ Resource-Intensive Calculations





Demonstration | Scheduler



Linux | Logging



Logging

syslogd (rsyslog)

logger

login logs

journalctl

log rotation



Logging

- **syslogd / rsyslog used for System Logging**
 - ✓ Higher performance
 - ✓ More filtering options
 - ✓ Support for various protocols (TCP, UDP, TLS)
 - ✓ Ability to write logs to databases, files, and remote servers



Logging

➤ logger – Manually Logging Messages

`/var/log/syslog`

➤ For Example:

`logger "Custom log message from thinknyx user"`

☐ Some of the common usecases are:

- ✓ Log a message to syslog
- ✓ Log messages with a specific tag
- ✓ Input from command line and log to the syslog
- ✓ Assigning priorities to custom log messages



Logging

➤ Login Logs – User Authentication Tracking

- ✓ Enhanced Security: Ensures that only authorized users access the Linux environment
- ✓ Comprehensive Activity Records: Logs detailed user actions such as login attempts and sudo usage
- ✓ Early Threat Detection: Helps administrators quickly spot unauthorized access and potential breaches
- ✓ Regulatory Compliance: Supports adherence to organizational policies and industry standards



Logging

➤ Key Authentication Log Files:

`/var/log/auth.log`

☐ Command

`sudo less /var/log/auth.log`

☐ Reviewing Recent Logins

- ✓ The last command reads from the `/var/log/wtmp` file
- ✓ Such as username, terminal used, IP address or hostname of the remote host, and login/logout times

☐ Reviewing Recent Logins

- ✓ The lastb command reads from the `/var/log/btmp` file



Logging

➤ Real-Time Monitoring:

`sudo tail -f /var/log/auth.log`

```
ubuntu@ip-172-31-15-110:/var/log$ sudo tail -f /var/log/auth.log
2025-02-05T04:05:01.984744+00:00 ip-172-31-15-110 CRON[3357]: pam_unix(cron:session): session closed for user root
2025-02-05T04:09:34.368877+00:00 ip-172-31-15-110 sshd[3362]: Accepted publickey for ubuntu from 49.36.189.246 port 5
0289 ssh2: RSA SHA256:RhqZ2Zrnhyi49ZOLmj0xEHbMOxYEUeRInSCHZmyrhzw
2025-02-05T04:09:34.374777+00:00 ip-172-31-15-110 sshd[3362]: pam_unix(sshd:session): session opened for user ubuntu(
uid=1000) by ubuntu(uid=0)
2025-02-05T04:09:34.393364+00:00 ip-172-31-15-110 systemd-logind[596]: New session 88 of user ubuntu.
2025-02-05T04:09:34.414083+00:00 ip-172-31-15-110 (systemd): pam_unix(systemd-user:session): session opened for user
ubuntu(uid=1000) by ubuntu(uid=0)
2025-02-05T04:09:37.610696+00:00 ip-172-31-15-110 sudo:    ubuntu : TTY=pts/0 ; PWD=/home/ubuntu ; USER=root ; COMMAND
=/usr/bin/lastb
2025-02-05T04:09:37.613232+00:00 ip-172-31-15-110 sudo: pam_unix(sudo:session): session opened for user root(uid=0) b
y ubuntu(uid=1000)
2025-02-05T04:09:37.616350+00:00 ip-172-31-15-110 sudo: pam_unix(sudo:session): session closed for user root
2025-02-05T04:13:24.771153+00:00 ip-172-31-15-110 sudo:    ubuntu : TTY=pts/0 ; PWD=/var/log ; USER=root ; COMMAND=/us
```



Logging

➤ Real-Time Monitoring:

- ✓ **Timestamp:** Indicates the date and time when the event occurred
- ✓ **Hostname:** The name of the host or server where the event was logged
- ✓ **Service or Program Name:** Identifies the service or application that generated the log entry
- ✓ **Process ID (PID):** The unique identifier of the process that logged the event
- ✓ **Message Content:** Provides detailed information about the event
 - ✓ **Module:** Refers to the Pluggable Authentication Module (PAM) used
 - ✓ **Service Context:** Specifies the service and the type of PAM interaction (session)
 - ✓ **Description:** Indicates that a session for the user root has ended



Logging

➤ journalctl – Systemd Journal Logs

- ✓ journalctl is used to view logs managed by systemd in Ubuntu 24.04 LTS

☐ Basic Usage

- View All Logs
 - journalctl
- View Logs in Real-Time
 - journalctl -f
- View Boot Logs
 - journalctl -b



Logging

➤ journalctl – Systemd Journal Logs

○ journalctl -xe

- ✓ The journalctl -xe command in Linux is used to view detailed **system logs** and **troubleshoot errors**

□ The option

- **-x** → Provides extra details and explanations for log messages (if available)
- **-e** → Jumps to the end of the log, showing the most recent entries first

□ This command

- ✓ Helps diagnose system errors and failures.
- ✓ Provides detailed explanations of logs.
- ✓ Useful for troubleshooting service failures and security alerts





Linux | Alerting



Alerting

- ✓ Alerting is essential for monitoring **system health, security, and performance**
 - ✓ Such as **high CPU usage, low disk space, or service failures**
 - ✓ System logs or specific conditions like **resource thresholds**
-
- **Linux provides tools for alerting**
 - cron jobs – Schedule checks and trigger alerts
 - mail or sendmail – Send email notifications for critical events





Linux | Memory Management



Memory Management

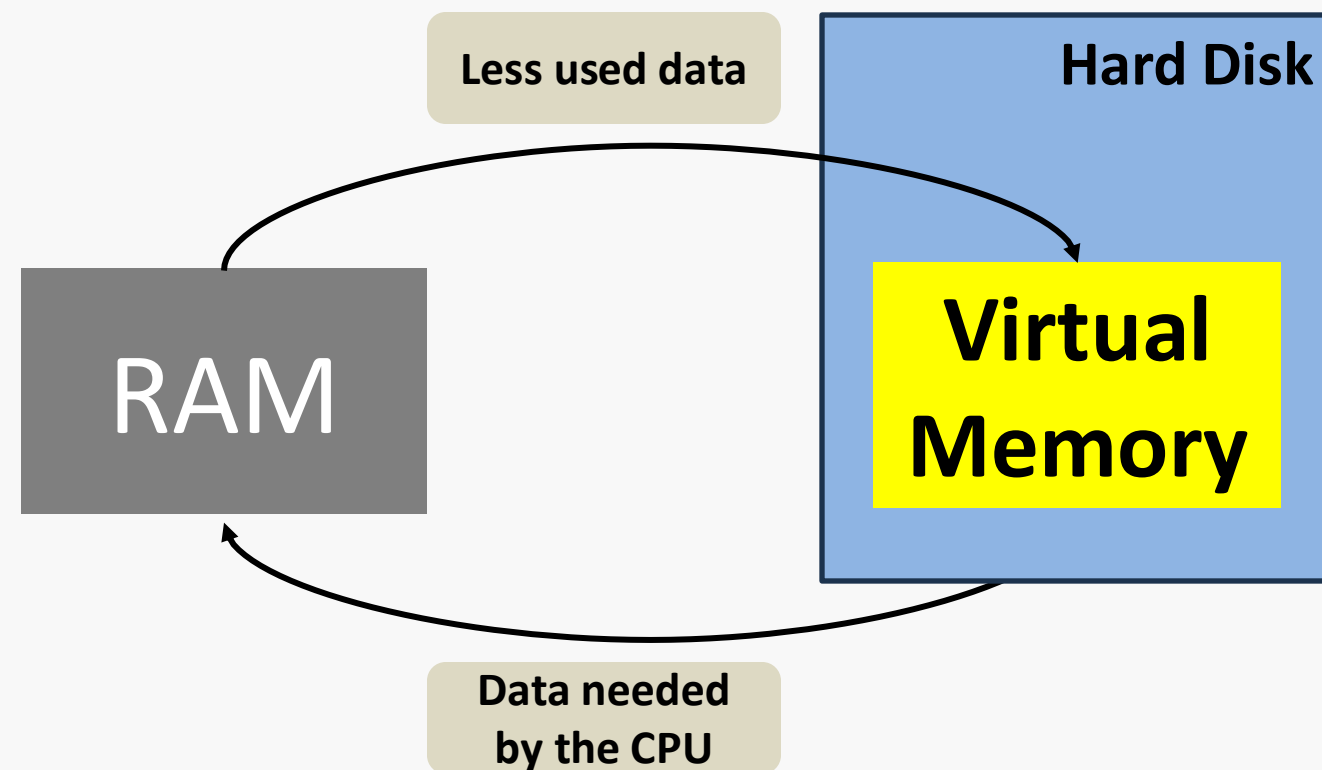
- **Memory management is a fundamental aspect of the Linux operating system, ensuring efficient utilization of system resources and maintaining optimal performance**



Memory Management

➤ Virtual Memory

- ✓ Virtual memory is a technique that allows a computer to use its hard disk to extend its RAM



Memory Management

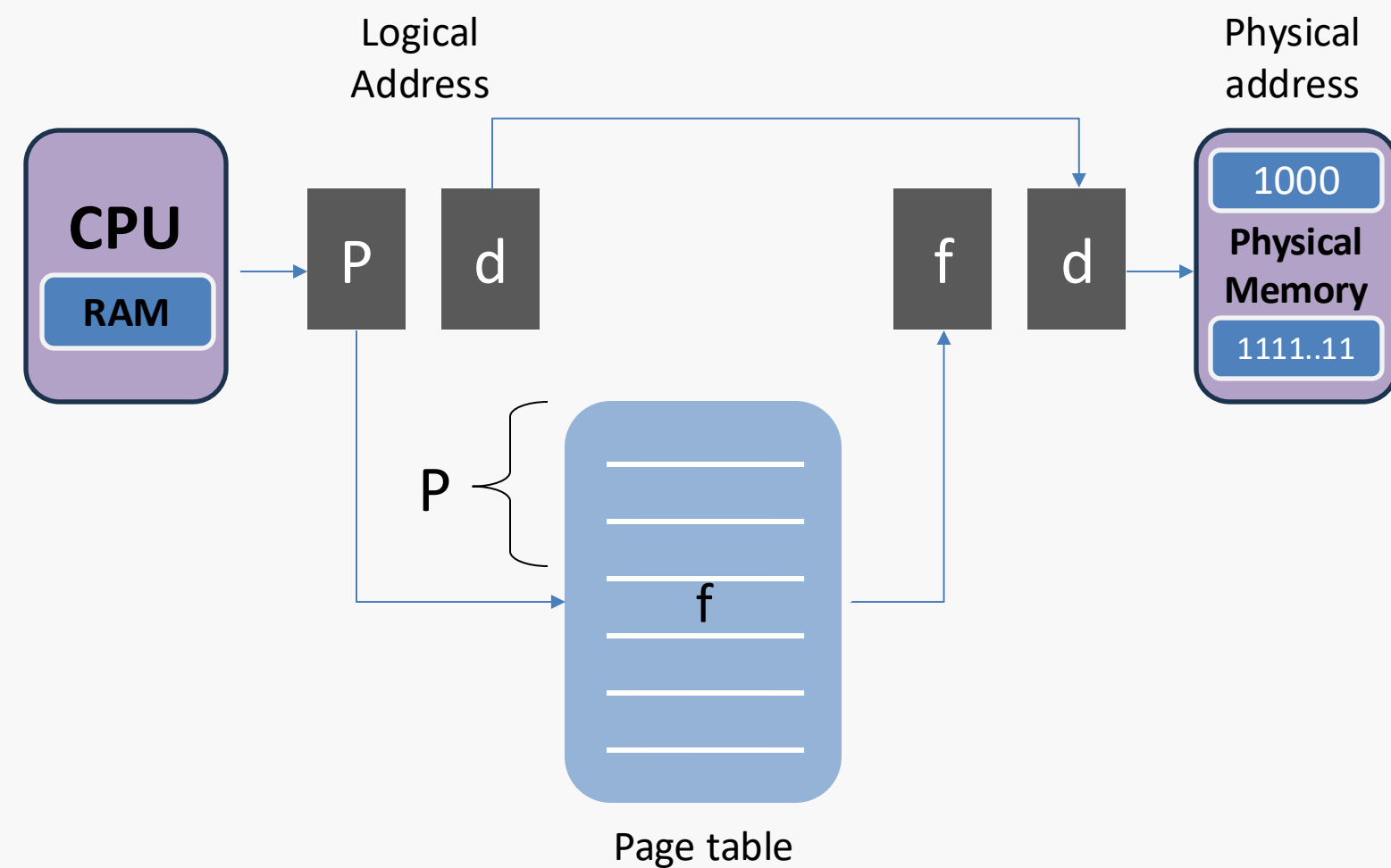
- **Linux provides:**
 - ✓ **Process with its own virtual address space**
 - ✓ **Large, contiguous memory area**
 - ✓ **The kernel to manage mapping between virtual addresses and physical memory**
 - ✓ **Efficient memory utilization and process isolation**



Memory Management

➤ Paging

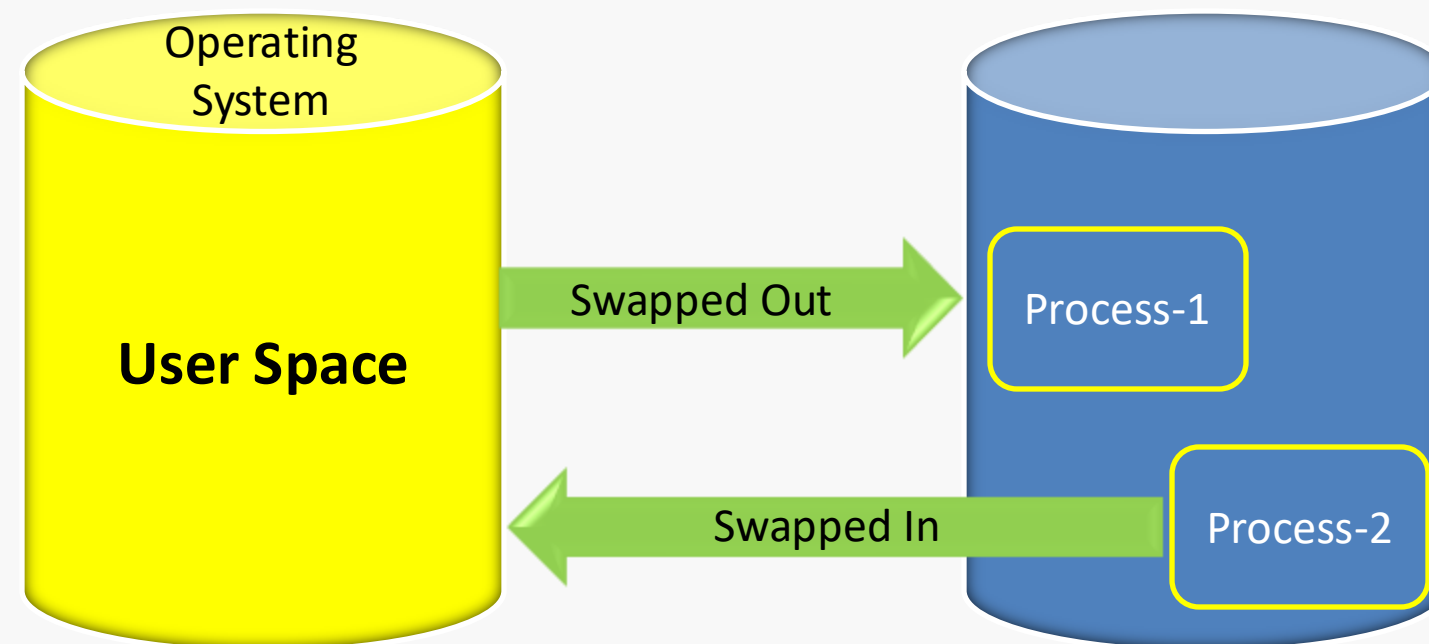
- ✓ Memory is divided into fixed-size blocks called pages



Memory Management

➤ Swap space

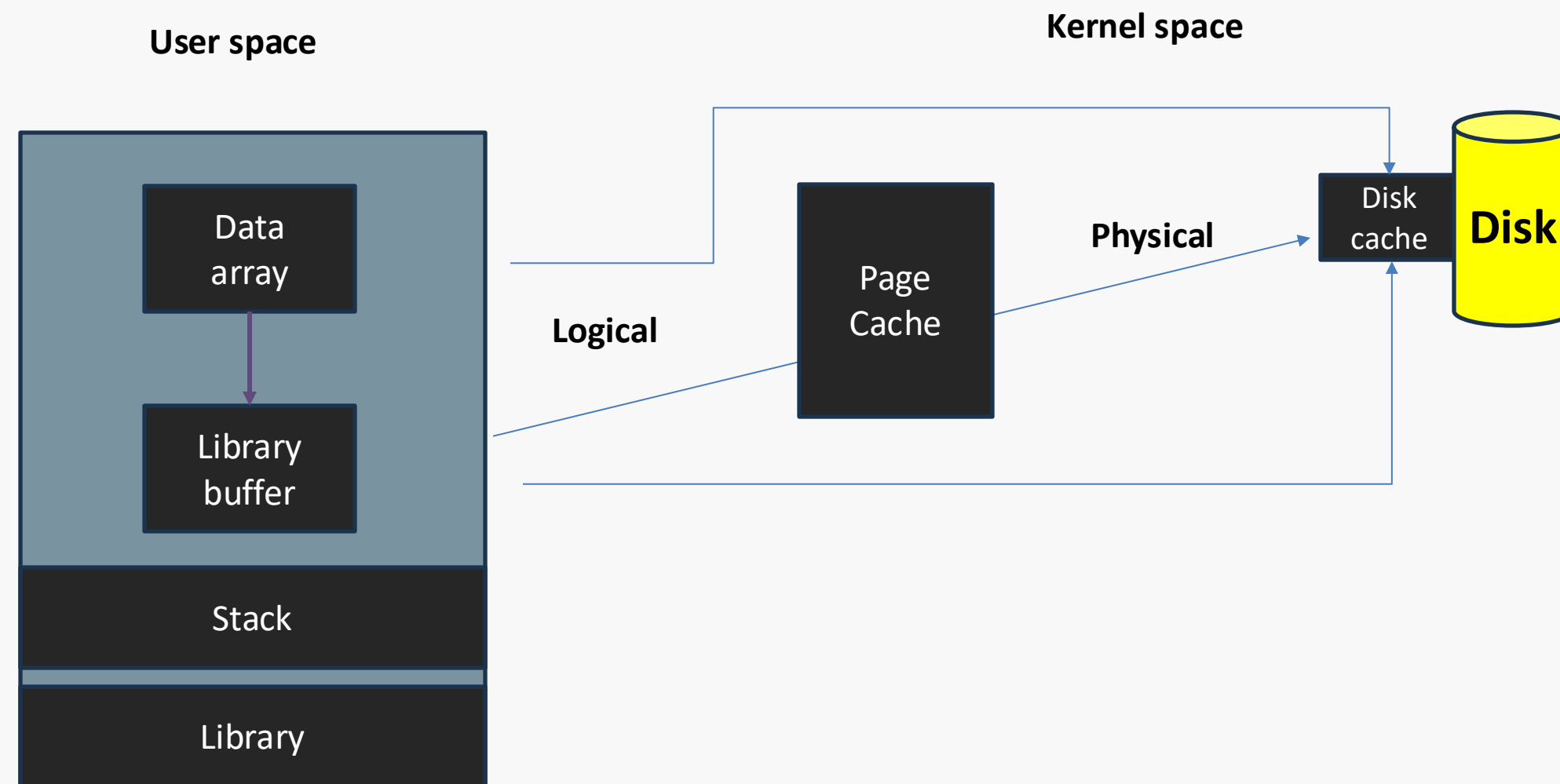
- ✓ Refers to an area on storage device that Linux OS uses as an extension of system's physical RAM



Memory Management

➤ Page Cache

- ✓ Linux uses a page cache to store frequently accessed disk data in memory



Memory Management

➤ Memory Zones

- ✓ Kernel organizes physical memory into zones to optimize allocation
 - ✓ ZONE_DMA
 - ✓ ZONE_NORMAL
 - ✓ ZONE_HIGHMEM



Memory Management

➤ Slab Allocator

- ✓ Maintains caches of commonly used objects, allowing for quick allocation and deallocation
- ✓ Reduces fragmentation and improves allocation efficiency



Memory Management

➤ Overcommit Handling

- ✓ Linux allows processes to allocate more memory than physically available
- ✓ Kernel uses overcommit, to balance the need for memory allocation with the risk of running out of memory



Memory Management

- **NUMA (Non-Uniform Memory Access)**
 - ✓ Supports Multi processor system
 - ✓ In Linux kernel optimizes memory allocation to minimize latency
 - ✓ Improves performance by allocating memory on the same node
 - ✓ Reduces the need for cross-node memory access



Summary



- ❖ Schedulers in linux like cron, at, anacron and batch
- ❖ Logging in linux
- ❖ /var/log/syslog file
- ❖ Logger command
- ❖ Memory management





Thinknyx Technologies

Linux Course

Enhancing Strategy

Section - 18





Linux | Introduction to Monitoring



Monitoring



➤ Monitoring

- Tracking
- Troubleshooting
- Optimal Performance



Linux | Need of Monitoring in Linux



Need of Monitoring in Linux

➤ **Monitoring**

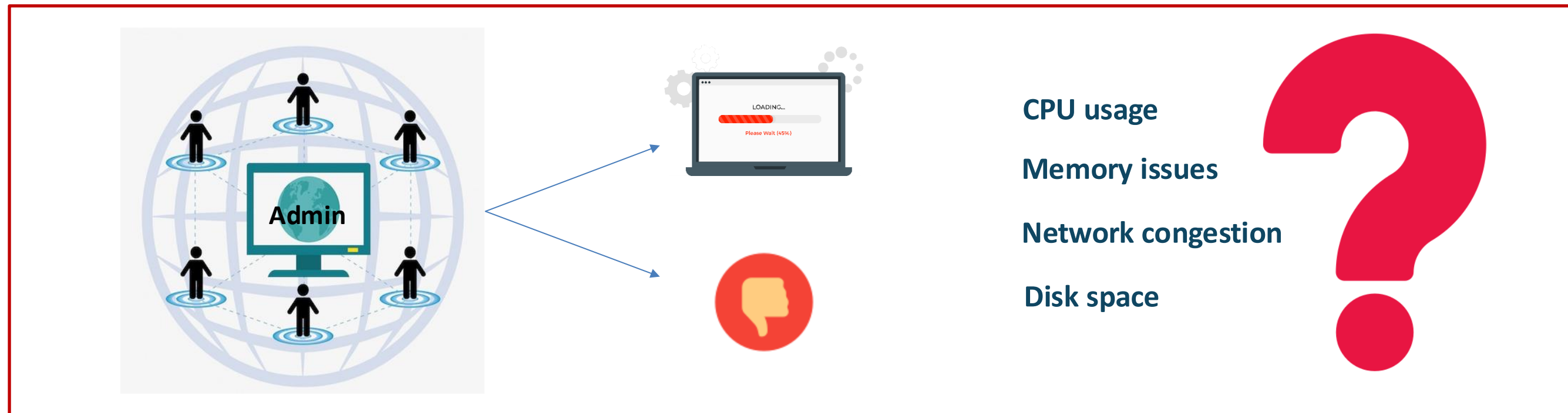
- ✓ Maintaining health of linux system

➤ **Importance of Monitoring**



Need of Monitoring in Linux

➤ Why Monitoring is Crucial?

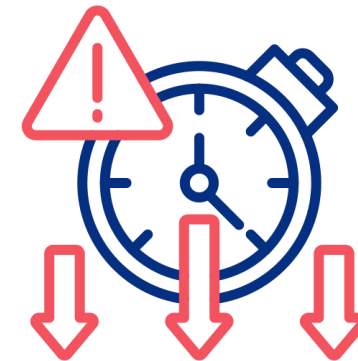


Need of Monitoring in Linux

➤ Early Detection of Problems



Monitoring Tools



Downtime



User Experience



Need of Monitoring in Linux

➤ **Early Detection of Problems**

✓ **Real-Time Visibility**

- Tools like top, htop, and iftop
- Quickly identify processes

✓ **Resource Optimization**

- Track and optimize system resources
- Make informed decisions

✓ **Log Analysis**

- Troubleshoot and analyze system health
- syslog or journalctl



Need of Monitoring in Linux

➤ **Early Detection of Problems**

✓ **Historical Data**

- Store historical performance data
- Helps in capacity planning, peak load analysis, and long-term performance tracking

✓ **Alerting and Automation**

- Send alerts
- Email, SMS & automated responses

✓ **Scalability and Distributed Monitoring**

- Prometheus, Nagios, or Zabbix
- Track the health of multiple servers, databases, and services



Need of Monitoring in Linux

➤ **Early Detection of Problems**

✓ **Security Monitoring**

- Track security events
- Early catch potential security breaches & rectification





Linux | Monitoring tools in Linux



Monitoring tools in Linux

➤ Popular monitoring tools available on Ubuntu 24.04 LTS:

- ✓ **ntopng (ntop Next Generation)** : A network monitoring tool that shows network traffic in real-time. A modern, next-generation version of ntop with enhanced features
- ✓ **free**: Provides a quick overview of memory usage on the system
- ✓ **vmstat**: Displays information about processes, memory, paging, block IO, traps, and CPU activity
- ✓ **iostat**: Displays CPU and I/O statistics for devices
- ✓ **watch**: Executes the command periodically, showing output full screen
- ✓ **iftop**: Monitors network bandwidth and provides detailed information about network connections





Demonstration | Popular Monitoring Tools

Popular monitoring tools

1 KB = 1,000 bytes

1 kibibyte (KiB) = 1,024 bytes





Linux | Advanced Monitoring tools



Advanced Monitoring

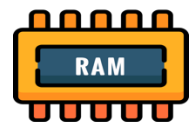
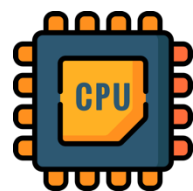
tools

➤ sar and journalctl command

❑ sar

✓ sar (System Activity Reporter) is another command-line utility that is part of the sysstat package in Linux

- Collecting
- Reporting
- Saving system activity



Monitoring



Advanced Monitoring tools

Real-time monitoring

Customizable reporting



Historical data

Long-term trend analysis



Advanced Monitoring

tools

➤ sar and journalctl command

❑ journalctl

- ✓ journalctl another powerful tool for **querying** and **analyzing logs** from systemd-based systems
- ✓ It's useful for **real-time monitoring** and **diagnostics**



Advanced Monitoring tools

Name	Number	Description
emerg	0	System is unusable
alert	1	Immediate action required
crit	2	Critical conditions
err	3	Error messages
warning	4	Warnings
notice	5	Significant but normal events
info	6	Informational logs
debug	7	Debugging messages



Summary



- ❖ Importance of Monitoring - System performance & Issue detection
- ❖ Popular Monitoring Tools - Track system and Network performance
- ❖ Demonstration - Monitor real-time system metrics
- ❖ Overview of Advanced Monitoring Tools





The Ultimate Linux Bootcamp for DevOps, SRE and Cloud Engineers (Hands-On)





System Monitoring



User & Group
Management



File
Permissions



Process & Service
Management



Package
Management



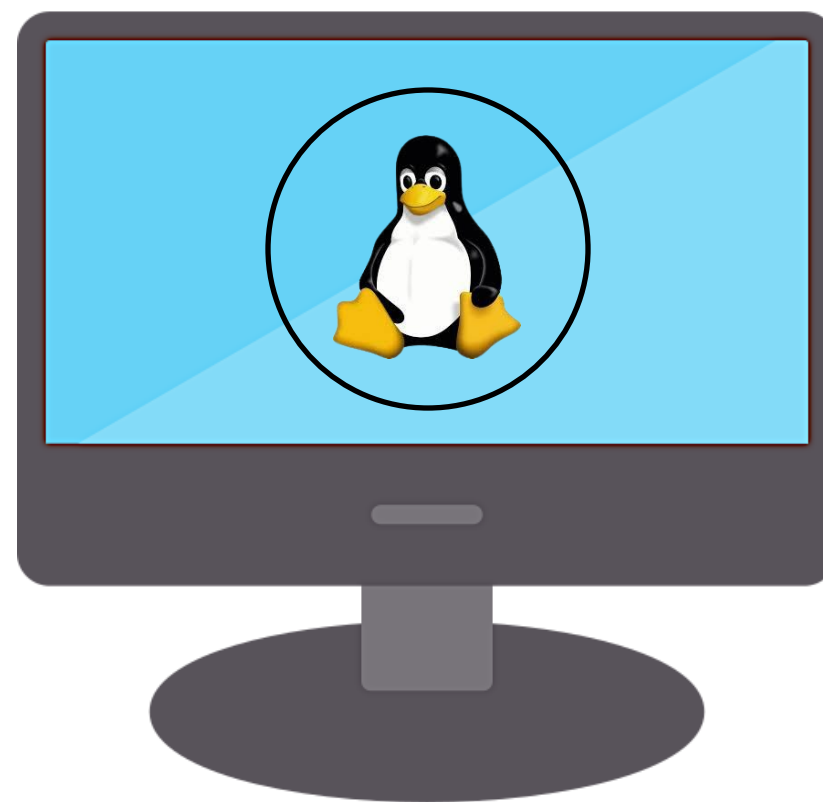
SSH Configuration



Networking



Disk
Partitioning





Follow us on:



@thinknyx



@thinknyx



@thinknyx-technologies



@thinknyx



@thinknyx-technologies